

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI, INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ

Praca dyplomowa

Analiza porównawcza architektury mikroserwisowej i monolitycznej
Comparative analysis of microservice and monolithic architecture

Autor: *Szymon Berdzik*
Kierunek studiów: *Informatyka i Systemy Inteligentne*
Opiekun pracy: *dr inż. Paweł Skrzyński*

Kraków, 2024

Spis treści

1 Wstęp *4*

1.1 Cele i założenia pracy **5**

1.2 Geneza i rozwój inżynierii oprogramowania **5**

1.3 Struktura pracy **6**

2 Koncepcja architektury oprogramowania *8*

2.1 Wprowadzenie do architektury oprogramowania **8**

2.1.1 Definicja architektury oprogramowania 8

2.1.2 Znaczenie architektury w inżynierii oprogramowania 9

2.2 Wzorce projektowe w inżynierii oprogramowania **10**

2.3 Wzorce architektoniczne w inżynierii oprogramowania **10**

2.4 Prawo Conway'a **17**

3 Przegląd popularnych architektur oprogramowania *19*

3.1 Architektura monolityczna **19**

3.1.1 Charakterystyka architektury monolitycznej 19

3.1.2 Modularny monolit 21

3.1.3 Zalety i wady architektury monolitycznej 22

3.2 Architektura mikroserwisowa **26**

3.2.1 Charakterystyka architektury mikroserwisowej 26

3.2.2 Wzorce projektowe w architektury mikroserwisowej 29

3.2.3 Wyzwania i trudności architektury mikroserwisowej 35

3.2.4 Zalety i wady architektury mikroserwisowej 46

4 Opis realizacji projektu referencyjnego w architekturze monolitycznej *51*

4.1 Wymagania funkcjonalne i niefunkcjonalne **51**

4.1.1 Wymagania niefunkcjonalne 51

4.1.2 Wymagania funkcjonalne 52

4.1.3 Diagram przypadków użycia 54

4.2 Projekt bazy danych **55**

4.3 Wykorzystane technologie **56**

4.4 Opis implementacji **57**

4.5 Konteneryzacja **61**

5 Projektowanie i implementacja systemu w architekturze mikroserwisowej *62*

5.1 Zmiana technologiczna **62**

5.2 Dekompozycja aplikacji na mikroserwisy **63**

5.2.1 Identyfikacja granic kontekstu 63

5.2.2 Podział bazy danych 65

5.2.3 Zarządzanie statycznymi i współdzielonymi danymi 70

5.2.4 Projektowanie komunikacji pomiędzy serwisami 71

5.2.5 Definiowanie granic transakcji 75

5.2.6 Zarządzanie zasobami pamięci i danych 75

5.2.7 Komunikacja z interfejsem użytkownika 76

5.3 Uwierzytelnianie i autoryzacja **77**

5.4 Implementacja wybranych mikroserwisów 78
5.4.1 Serwis powiadomień 78
5.4.2 Serwis profilu użytkownika 84

5.5 Infrastruktura systemu mikroserwisowego 88
5.5.1 Wykrywanie usług 88
5.5.2 Równoważenie obciążenia 88
5.5.3 Konfiguracja klastra 89
5.5.4 Zewnętrzne usługi 92

6 Testy 93

6.1 Testy obciążeniowe 95
6.1.1 Testy pod stałym obciążeniem 95
6.1.2 Testy pod zmiennym obciążeniem 100

6.2 Testy przeciążeniowe 104
6.2.1 Operacje odczytu danych 104
6.2.2 Operacje zapisu danych 109

6.3 Wnioski 112

7 Podsumowanie 114

7.1 Wnioski końcowe 115

8 Bibliografia 117

1 Wstęp

Architektura oprogramowania [12] jest jednym z kluczowych aspektów projektowania systemów informatycznych, która przez lata ewoluowała prowadząc do powstania różnych podejść do tworzenia oprogramowania. Kształtuje ona szkielet aplikacji, definiując sposób w jaki poszczególne komponenty systemu będą ze sobą współpracować. Te początkowe założenia mają wpływ na to jak aplikacja będzie się rozwijać i skalować w przyszłości, ale i również na to w jaki sposób projekt informatyczny jest zarządzany. Dlatego wybór odpowiedniej architektury, dostosowanej do rozwiązywanego problemu jest kluczowy dla sukcesu projektu. Dwa z najbardziej powszechnie stosowanych podejść do tworzenia oprogramowania to architektura monolityczna oraz mikroserwisowa.

Tradycyjne, monolityczne podejście do budowania oprogramowania przez lata było standardem w branży informatycznej, będąc uznawane za prostsze do implementacji i utrzymania w początkowych fazach projektu. Natomiast w miarę rozwoju, aplikacje monolityczne przeobrażają się w duży fragment oprogramowania, który jest trudny do zarządzania i adaptacji do zmieniających się wymagań biznesowych. [101]

W raz z rozwojem infrastruktury chmurowej, na popularności zyskała architektura mikroserwisowa, która doskonale pozwala na wykorzystanie korzyści płynących z przetwarzania w chmurze. W odpowiedzi na problemy związane z zarządzaniem oraz wydajnością dojrzałych aplikacji monolitycznych, firmy często decydują się na podział tych aplikacji na mniejsze komponenty i migracje do architektury mikroserwisowej. Niezależne usługi odpowiadające za konkretne funkcjonalności biznesowe są prostsze do skalowania oraz zarządzania, ponieważ praca może zostać wydzielona do niezależnych zespołów, a cały system zyskuje większą elastyczność w dostosowywaniu się do zmian. [101]

W kontekście dynamicznych zmian w dziedzinie inżynierii oprogramowania, istotne staje się zrozumienie różnic i podobieństw między architekturą mikroserwisową, a monolityczną. Niniejsza praca ma na celu przybliżenie tych zagadnień, poprzez omówienie kluczowych aspektów obu podejść.

1.1 Cele i założenia pracy

Celem pracy jest szczegółowe zbadanie i porównanie dwóch podejść do tworzenia i zarządzania aplikacjami programistycznymi: architektury mikroserwisowej i architektury monolitycznej, na podstawie aplikacji wspomagającej znajdowanie zleceń dla programistów.

Cele szczegółowe pracy to:

1. Przeanalizowanie ewolucji pojęcia architektury oprogramowania od jej początków do współczesności, w tym kluczowych definicji i koncepcji wraz z istniejącymi wzorcami architektonicznymi.
2. Omówienie podstawowych zagadnień architektury mikroserwisowej i monolitycznej, w tym ich kluczowych cech i różnic.
3. Analiza najczęściej występujących ograniczeń i problemów architektury mikroserwisowej oraz przedstawienie możliwych rozwiązań.
4. Dekompozycja oraz implementacja aplikacji monolitycznej w architekturze mikroserwisowej.
5. Przeprowadzenie eksperymentu mającego na celu porównanie wydajności aplikacji w obu architekturach.

Praca dostarczy kompleksowego przeglądu i analizy obu architektur, umożliwiając zrozumienie ich mocnych i łatwych stron. Na podstawie wyników przeprowadzonej analizy możliwe będzie sformułowanie wniosków oraz rekomendacji dotyczących wyboru odpowiedniej architektury, dostosowanej do specyficznych wymagań i kontekstu aplikacji.

1.2 Geneza i rozwój inżynierii oprogramowania

W artykule Manfreda Broya pt. *Yesterday 50 Years of Software Engineering* [15], który został opublikowany w pięćdziesięciolecie wydzielenia inżynierii oprogramowania jako nowej gałęzi informatyki. Autor przedstawia historię tej dyscypliny, rozpoczynając od konferencji NATO, która odbyła się w Niemczech w 1968 roku i była punktem wyjścia dla inżynierii oprogramowania jaką znamy. Artykuł przedstawia retrospektywny przegląd inżynierii oprogramowania w ciągu ostatnich pięćdziesięciu lat, wskazując na postępy jakie zostały poczynione. Dyskusje obejmowały wszystkie aspekty oprogramowania znane w tamtym czasie, włączając w to relację oprogramowania z sprzętem komputerowym, produkcję i wdrożenie oprogramowania, dystrybucję oprogramowania i serwisowanie. Dodatkowo według [13] na

konferencji pojawiły się pierwsze sformułowania architektury oprogramowania jako koncepcji Iana Sharpa dotyczącej modelowania systemów oprogramowania.

Autor omawia również przyszłe wyzwania stojące przed inżynierią oprogramowania, które możemy zaobserwować w aktualnym czasie takie jak rozwój sztucznej inteligencji, systemów cyberbezpieczeństwa i Internetu rzeczy (ang. *Internet of Things*). Dodatkowo autor zaznacza o potrzebie posiadania bardziej abstrakcyjnych widoków na architekturę i struktury metodologicznej. Odnosi się również do potrzeby projektowania oprogramowania, które byłoby bardziej adaptacyjne i autonomiczne, przygotowane na różne przypadki użycia i ich specjalne wymagania, które mogą się pojawić.

Autor artykułu podkreśla, że współcześnie inżynieria oprogramowania odgrywa kluczową rolę oraz jest bardzo obszerna, a jej postęp jest nadzwyczaj szybki. Wraz z dynamicznym rozwojem technologicznym, dziedzina ta musi elastycznie reagować na nowe wyzwania, jakie niesie przyszłość. Podkreśla się konieczność ciągłego rozwoju i adaptacji inżynierii oprogramowania do zmiennych potrzeb i wyzwań, aby sprostać wymaganiom dzisiejszego świata.

1.3 Struktura pracy

Praca składa się z siedmiu rozdziałów, które przedstawiają przegląd obu podmiotów badań, implementacje projektu referencyjnego w obu architekturach oraz przeprowadzenie badań wydajności architektur na podstawie projektu referencyjnego. Struktura pracy jest następująca:

Rozdział 1 – Wstęp: Wprowadzenie w tematykę pracy. W rozdziale zostały określone cele oraz założenia pracy wraz ze strukturą kolejnych rozdziałów.

Rozdział 2 – Koncepcja architektury oprogramowania: Rozdział przedstawia obecny stan wiedzy na temat architektury oprogramowania, oparty na podstawię dostępnej literaturze naukowej.

Rozdział 3 – Przegląd popularnych architektur oprogramowania: W tym rozdziale przeprowadzono szczegółowy przegląd dwóch podmiotów badań: architektury monolitycznej i mikroserwisowej. Zaprezentowane zostały kluczowe aspekty wraz z charakterystyką obu architektur, skupiając się również na zaletach oraz wadach obu rozwiązań.

Rozdział 4 – Opis realizacji projektu referencyjnego w architekturze monolitycznej: Rozdział ten przedstawia opis projektu w architekturze monolitycznej. Przedstawione zostały

wymagania funkcjonalne i niefunkcjonalne systemu wraz z opisem implementacji aplikacji monolitycznej.

Rozdział 5 – Opis realizacji projektu referencyjnego w architekturze mikroserwisowej: W rozdziale przedstawiony został proces dekompozycji aplikacji wraz z opisem implementacji oraz konfiguracją infrastruktury.

Rozdział 6 – Porównanie architektur na podstawie zrealizowanych projektów: W rozdziale przeprowadzone zostały testy obciążeniowe oraz wydajnościowe porównywając działanie aplikacji w architekturze monolitycznej oraz mikroserwisowej w różnych warunkach.

Rozdział 7 – Podsumowanie: W tym rozdziale zawarto podsumowanie pracy, z uwzględnieniem osiągniętych celów oraz wniosków.

2 Koncepcja architektury oprogramowania

W niniejszym rozdziale przedstawiony zostanie aktualny stan wiedzy na temat architektury oprogramowania, który został oparty na przeglądzie dostępnej literatury naukowej. Rozdział zaczyna się od przeglądu definicji architektury oprogramowania, ukazując jej ewolucję od początków jej istnienia oraz przedstawia istotną rolę tej nauki w inżynierii oprogramowania. Następnie omawiane są wzorce projektowe oraz architektoniczne, z uwzględnieniem przeglądu najważniejszych wzorców architektonicznych wykorzystywanych w inżynierii oprogramowania. Na zakończenie rozdziału przedstawione zostało Prawo Conway'a, które opisuje związek między strukturą organizacyjną a architekturą oprogramowania.

2.1 Wprowadzenie do architektury oprogramowania

2.1.1 Definicja architektury oprogramowania

W stosunkowo krótkim okresie istnienia nauki o architekturze oprogramowania pojawiło się wiele różnych definicji. Pierwsza formalna definicja została zawarta przez F. Brookes'a i K. Iverson'a w 1969 roku w swojej książce *Automatic Data Processing. (System 360 Edition).* [2] Zdefiniowali oni ją wówczas jako „pojęciową strukturę komputerową, jaką widzi programista". Natomiast kilka lat później w 1975 F. Brookes w swojej kolejnej książce *The Mithical Man-Month: Essays in Software Engineering* [3] skorygował tą definicję dochodząc do wniosku, że architektura oprogramowania to „pełne i szczegółowe przedstawienie interfejsu użytkownika. (...) Podczas gdy architektura mówi nam co się dzieje, implementacja mówi jak to ma się dziać". Z podanej definicji można dojść do konkluzji, że Brooks w tamtych czasach odróżniał architekturę od implementacji, co często nawet w obecnych czasach jest mylone i postrzegane jako to jedno.

Kolejną definicją architektury podał w roku 1989 R.W. Shwanke „Architektura - struktura oprogramowania to lista komponentów i połączenia pomiędzy nimi". [4] Definicję te rozszerzyli w roku 1992 D. Perry i A. Wolf, wprowadzając rozróżnienie pomiędzy elementami przetwarzania, elementami danych i elementami łączącymi. Ta taksonomia w zasadzie utrzymuję się w większości kolejnych definicji i podejść. [5] Brzmi ona następująco „Architektura opisuje wybrane elementy architektoniczne, połączenia między nimi oraz ograniczenia tych elementów architektonicznych i ich wzajemnych związków niezbędnych do zdefiniowania modelu spełniającego wymagania i będącego podstawą dla projektu systemu informatycznego". Rok później M. Shaw i D. Garlan w artykule *An Introduction to Software Architecture, Advances in Software Engineering and Knowledge Engineering, vol. 1,* który

uznano za przełomowy artykuł o architekturze oprogramowania [5] uprościli definicję D. Perry'ego i A. Wolf'a jako „Architekturę oprogramowania danego systemu można traktować jako kolekcję komponentów obliczeniowych – lub prościej komponentów - wraz z opisem interakcji pomiędzy komponentami.".

W roku 1995 D. Garlan i D. Perry podali nową definicję „Architektura oprogramowania to struktura komponentów programu/systemu, ich wzajemne powiązania oraz zasady i wytyczne dotyczące ich projektowania i rozwoju w czasie". Ten sam rok był również przełomowy dla nauki o architekturze oprogramowania za sprawą artykułu Philippe Kruchten'a *Architectural Blueprints – The „4+1" View Model of Software Architecture* [6], w którym autor wprowadził bardziej wszechobejmującą definicję, która uwzględnia abstrakcję i potrzeby systemu informatycznego. Brzmi ona następująco: „Architektura oprogramowania dotyczy projektu i regułami ustawiającymi sposób jej budowy i rozwoju". W 2010 P. Clements et al. w swojej książce [10] wyprowadzili kolejną definicję na podstawie tej z książki *Software Architecture in Practice*, która brzmi następująco: „Architektura jest to zbiór struktur potrzebnych do uzasadnienia systemu, który zawiera elementy oprogramowania, relacje między nimi i ich właściwości". Definicja podkreśla wielkość struktur obecnych w każdym systemie oprogramowania. Struktury te, starannie wybrane i zaprojektowane przez architekta, są kluczem do osiągnięcia i rozumowania na temat celów projektowych systemu. Warto zauważyć, że definicja jest obojętna co do tego, czy architektura systemu jest dobra czy zła, co oznacza, że może ona umożliwiać lub uniemożliwiać spełnienie wymagań dotyczących zachowania, wydajności i cyklu życia systemu.

Kolejna definicja zawiera się w standardzie ISO/IEC 42010:2011 [11], który wyparł wersje 42010:2007. Standard opisuje architekturę jako „Podstawowe (systemowe) pojęcia lub właściwości systemu działającego w swoim środowisku wyrażone przez jego elementy, relacje oraz zasady ich projektowania i ewolucji". Ostatnia, a zarazem najnowsza definicja, która została przedstawiona do czasu publikacji tej pracy zawiera się w najnowszym standardzie ISO/IEC/IEE 43010:2022 [12], który mówi, że architektura to „Podstawowe pojęcia lub właściwości jednostki w jej środowisku oraz zasady rządzące realizacją i ewolucją tej jednostki oraz powiązanych z nią procesów cyklu życia". [13,14]

2.1.2 Znaczenie architektury w inżynierii oprogramowania

Architektura oprogramowania odgrywa kluczową rolę w procesie inżynierii oprogramowania oraz jest fundamentalnym elementem każdego systemu informatycznego. Jest to stosunkowo młoda dziedzina informatyki i od kilku dekad cały czas dynamicznie się rozwija.

Zajmuje się ona definiowaniem struktury systemu oraz odpowiada za wiele różnych aspektów każdego modułu aplikacji, biorąc pod uwagę bezawaryjność i skalowalność systemu oraz łatwość zarządzania. Ponadto określa, jakie komponenty będą istniały, jak będą ze sobą współpracować i jakie technologie zostaną wykorzystane.

Architektura oprogramowania odnosi się do krytycznych wymagań niefunkcjonalnych, w tym wydajność, bezpieczeństwo i niezawodność, zapewniając, że system spełnia i przewyższa oczekiwania użytkowników. Właściwa architektura zoptymalizuje proces rozwoju, co pomoże spełnić wymagania i oczekiwania klienta. To główny powód, dla którego właściwy wybór architektury oprogramowania odgrywa kluczową rolę w rozwoju firmy. [1,16]

2.2 Wzorce projektowe w inżynierii oprogramowania

Wzorzec projektowy to udokumentowana najlepsza praktyka lub rdzeń rozwiązania, które zostało skutecznie zastosowane w wielu środowiskach do rozwiązania problemu, który występuje cyklicznie w określonym zestawie sytuacji. Architekt Christopher Alexander opisuje wzorzec jako „powtarzające się rozwiązanie dla wspólnego problemu w danym kontekście i systemie sił". W jego definicji termin „kontekst" odnosi się do zestawu sytuacji, w których dany wzorzec jest stosowany, a termin „system sił" odnosi się do zestawu ograniczeń występujących w określonym kontekście. Wzorzec projektowy jest skutecznym środkiem do przekazywania tego, co nauczono się o wysokiej jakości projektów. [18]

2.3 Wzorce architektoniczne w inżynierii oprogramowania

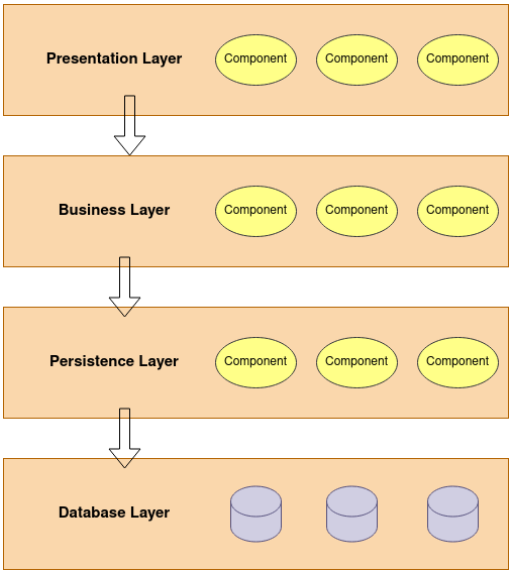
Wzorzec architektoniczny to ustrukturyzowany zestaw konkretnych decyzji projektowych, które zostały uznane za skuteczne w odniesieniu do określonych klas systemów oprogramowania lub podsystemów i mają zastosowanie przy powtarzających się problemach projektowych. Te decyzje projektowe są konfigurowalne i muszą być dostosowane do komponentów i łączników specyficznych dla konkretnej aplikacji. [17]

Poniżej zostaną przedstawione oraz opisane wzorce architektoniczne wykorzystywane w inżynierii oprogramowania:

- **Wzorzec architektury warstwowej** (ang. *Layered Architecture Pattern*)

Jest to jeden z najbardziej popularnych wzorców, zwany inaczej wzorcem architektury wielowarstwowej. Charakteryzuje się tym, że komponenty są zorganizowane na poziome warstwy i zajmują się tylko logiką związaną z daną warstwą. Każda warstwa ma określoną rolę w ramach całego systemu. Paradygmat wzorca warstwowego nie określa dokładnej liczby

i typów warstw, natomiast zazwyczaj składa się on z trzech lub czterech. Na poniższym rysunku przedstawiony został schemat architektury czterowarstwowej z warstwą prezentacji, biznesową, persystencji i bazy danych.

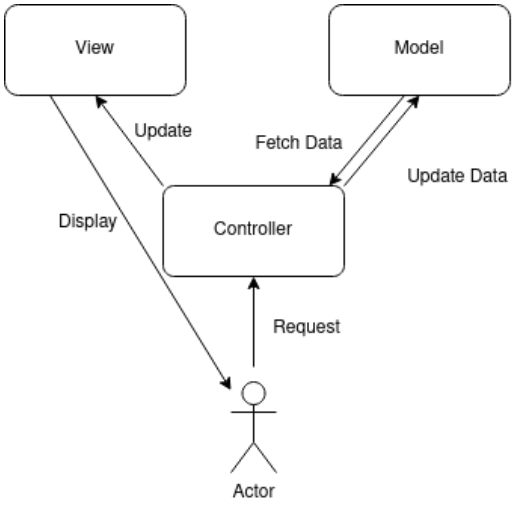


Rys 2. Schemat architektury czterowarstwowej. Źródło: P. Kuchana [19]

Bardzo ważnym aspektem w tym wzorcu jest izolacja warstw, która mówi, że każda z warstw jest „zamknięta". Oznacza to, że każde żądanie użytkownika przechodzące przez system, musi przejść przez wszystkie poprzednie warstwy, aby dostać się do kolejnej. Bazując na rys 2. zapytanie wpadające do warstwy prezentacji musi przejść przez warstwę biznesową zanim dostanie się do warstwy persystencji. Dzięki tej koncepcji zmiany wprowadzone w jednej warstwie architektury nie wpływają na komponenty w innych warstwach. [19]

- **Wzorzec MVC** (ang. *Model-View-Controller Pattern*)

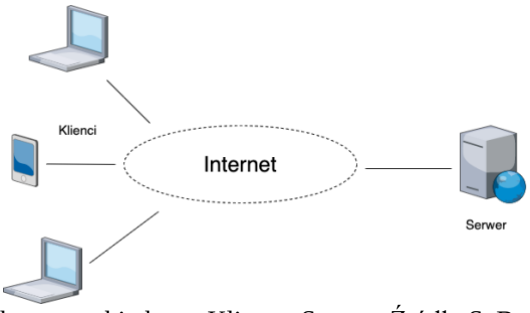
Wzorzec, który dzieli aplikację na trzy odrębne komponenty takie jak: model (ang. M*odel*), widok (ang. *View)* oraz kontroler (ang. C*ontroller*). Każdy z nich obsługuje określony aspekt funkcjonalności systemu. Warstwa modelu odpowiada rekordom w bazie danych, zarządza danymi i logiką specyficzną dla domeny aplikacji. Warstwa widoku reprezentuje elementy GUI (ang. *Graphical User Interface*), w których wyświetlane są dane reprezentowane przez model. Dzięki widokom użytkownicy mogą wejść w interakcje z systemem. Kontroler jest warstwą separacyjną pomiędzy modelem a widokiem. Kontroluje przepływem danych między tymi dwoma warstwami oraz zarządza danymi wejściowymi użytkownika aktualizując odpowiednio warstwę modelu i wyświetlając użytkownikowi odpowiedni widok. Schemat MVC został przedstawiony na poniższej ilustracji. [20]



Rys 3. Schemat przedstawiający wzorzec MVC. Źródło: Strona internetowa geeksforgeeks [94]

- **Wzorzec Klient – Serwer** (ang. *Client-Server Pattern*)

Architektura składa się z dwóch podmiotów: klienta oraz serwera. Klient oddziałuje jako niezależny proces, który oprócz wykonywania własnej pracy, komunikuje się z drugim procesem, nazywanym serwerem. Komunikacja odbywa się zazwyczaj poprzez protokół TCP/IP (ang. *Transmission Control Protocol/Internet Protocol*) na zasadzie przesłania żądania do danej usługi lub zasobu danych. Serwer natomiast oczekuje na wybranym porcie na zapytania przesłane przez klienta, a po otrzymaniu żądania, obsługuje je i przesyła odpowiedź zwrotną. Klientów może być wiele. [22] Na rysunku 4 przedstawiony został schemat tej architektury.

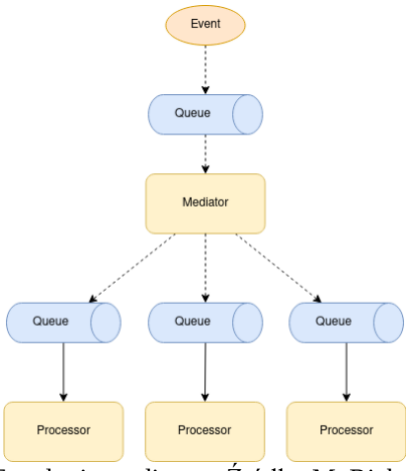


Rys 3. Schemat architektury Klient – Serwer. Źródło S. Berdzik [63]

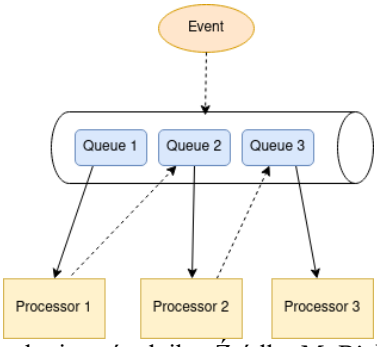
- **Wzorzec architektury skierowanej na zdarzenia** (ang. *Event-Driven Architecture Pattern*)

W celu przedstawienia tego wzorca potrzebne jest zdefiniowanie pojęcia *zdarzenia*. Według Wikipedii [21] zdarzenie jest to niemutowalna akcja, która nastąpiła w wyniku zmiany stanu aplikacji. Wzorzec architektury sterowanej zdarzeniami jest bardzo elastyczny i może być używany za równo do małych jak i dużych, rozproszonych projektów. Polega on na

opieraniu akcji serwisów o zdarzenia, która są produkowane i odczytywane przez różne serwisy w systemie. Wzorzec składa się z dwóch głównych topologii mediatora (ang. *Mediator)* oraz pośrednika (ang. *Broker*). Topologia mediatora opiera się na centralnym serwisie (mediatorze), który jest wykorzystywany do koordynowania nowych zdarzeń w ramach jednego, głównego zdarzenia. Natomiast w topologii pośrednika zdarzenia są ze sobą złączone łańcuchowo. [19]



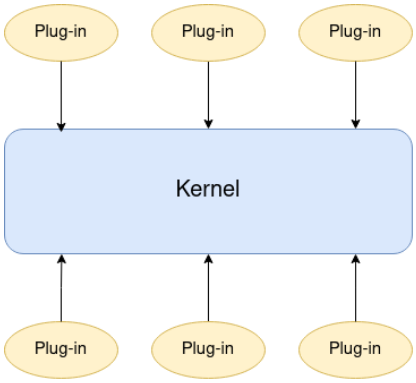
Rys 5. Topologia mediatora. Źródło: M. Richards [19]



Rys 6. Topologia pośrednika. Źródło: M. Richards [19]

- **Wzorzec architektury mikrojądrowej** (ang. *Microkernel Architecture Pattern*)

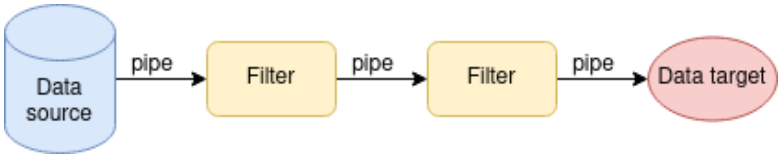
Wzorzec architektury mikro jądrowej opiera się na podziale architektury na dwie niezależne części: jądro systemu oraz niezależne wtyczki. Założeniem jest, że jądro systemu zawiera wyłącznie tą część oprogramowania, która jest niezbędna do prawidłowego funkcjonowania systemu. Dodatkowa logika biznesowa dołączana jest za pomocą wtyczek, które są samowystarczalne i niezależne od innych wtyczek. Wtyczki zawierają kod adaptowalny z jądrem i rozszerzający je o dodatkowe funkcjonalności. Schemat architektury został przedstawiony na rysunku 7. [19]



Rys 7. Schemat architektury mikrojądrowej. Źródło: M. Richards [19]

- **Wzorzec potoków i filtrów** (ang. *Pipe-filter Pattern*)

Wzorzec potoków i filtrów wykorzystywany jest w przypadku systemów przekształcających strumienie danych z wejścia na wyjście. Polega na podzieleniu komponentów systemu na niezależne komponenty nazywane filtrami, które przetwarzają otrzymane dane na wejściu i za pomocą potoków przekazują je do kolejnych encji. Dane wyjściowe danego filtra odpowiadają danym wejściowym kolejnego filtra, co można zaobserwować na poniższym schemacie architektonicznym.



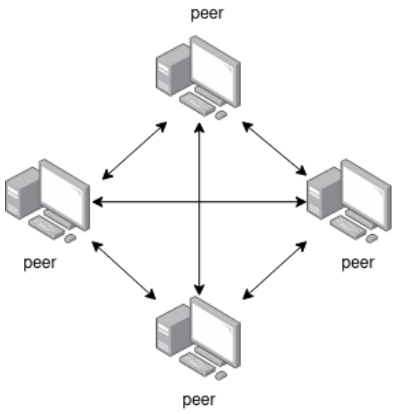
Rys 8. Schemat architektury z wykorzystaniem wzorca potoków i filtrów. Źródło: Strona internetowa Learn Microsoft [23]

Kluczową zaletą wzorca potoków i filtrów jest zdolność do uruchamiania równoległych instancji filtrów, co pozwala na równomierne rozłożenie obciążenia i poprawę przepustowości systemu. Dodatkowo filtry mogą być wykorzystane ponownie w ramach tego samego przepływu danych. [8, 23]

- **Wzorzec architektury peer-to-peer** (ang. *Peer-to-peer Architecture Pattern*)

Wzorzec peer-to-peer opiera się na zdecentralizowanej sieci, w której rozproszone komponenty komunikują się ze sobą jak równy z równym. W fachowym języku nazywane są peer'ami, co z języka angielskiego oznacza rówieśnik. Autor artykułu *A definition of peer-to-peer networking for the calssification of peer-to-peer architectures and applications* [24] nazywa encje w sieci peer-to-peer słowem „Servent", które jest połączeniem pierwszej sylaby słowa serwer (ang. *Server*) oraz drugiej sylaby słowa klient (ang. *Client*) w języku angielskim. Autor porównuje w ten sposób architekturę peer-to-peer do architektury klient - serwer

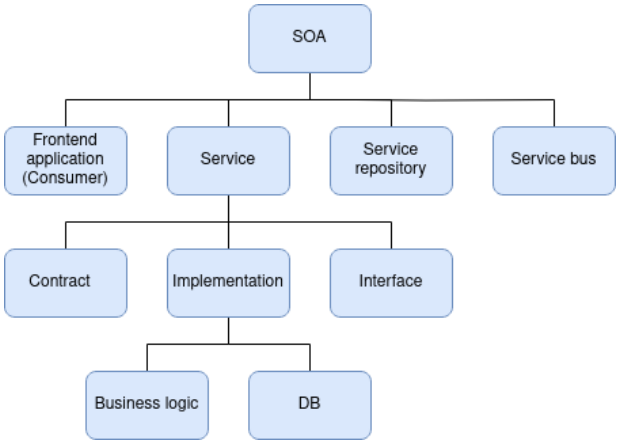
zaznaczając różnice pomiędzy nimi. W sieci peer-to-peer komponenty komunikują się ze sobą dwukierunkowo. Oznacza to, że mogą zachować się jednoczenie jako klient oraz serwer. Natomiast w architekturze klient - serwer jeden komponent może przyjmując postać tylko klienta lub serwera. Dodatkowo zasoby w sieci są współdzielone pomiędzy komponentami i są one dostępne bezpośrednio bez pośrednictwa innych podmiotów. [8]



Rys 9. Schemat architektury peer-to-peer. Źródło: Strona internetowa geeksforgeeks [94]

- **Wzorzec architektury zorientowanej na usługi** (ang. *Service-Oriented Architecture Pattern*)

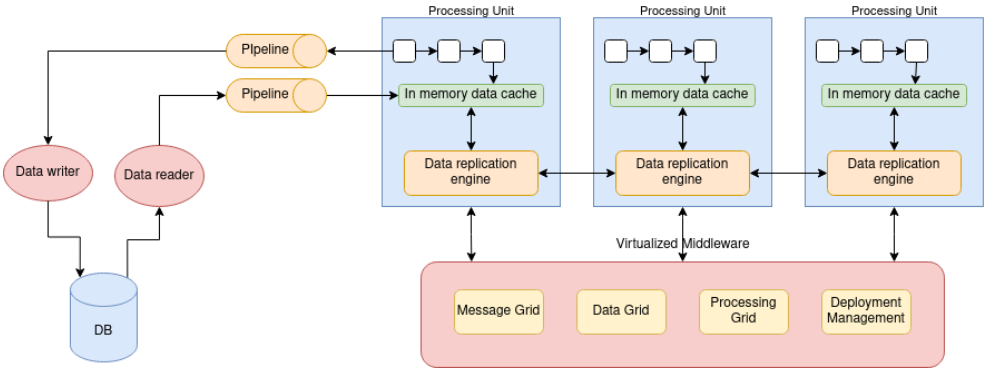
Główną ideą tego wzorca architektonicznego jest projektowanie rozproszonych systemów jako sieci współdziałających ze sobą biznesowych usług. Każdy serwis składa się z jednego lub więcej luźno powiązanych interfejsów oraz ich implementacji, które stanowią kontrakt przedstawiający specyfikacje oraz funkcjonalności danej usługi. Podstawowo wzorzec wykorzystuje cztery kluczowe elementy: konsumenci (ang. *Consumers*), usługi (ang. *Services*), rejestr usług (ang. *Service Repository*) oraz dostawca usług (ang. *Service Bus*) przedstawione na poniższym diagramie.



Rys 10. Struktura architektury zorientowanej na usługi. Źródło: Wikipedia [25]

- **Wzorzec architektury opartej na przestrzeni kosmicznej** (ang. *Space-Based Architecture Pattern*)

Wzorzec architektury opartej na przestrzeni kosmicznej skupia się na maksymalizacji skalowalności aplikacji poprzez minimalizacje zcentralizowanych ograniczeń. Polega to na wyeliminowaniu bezpośredniej komunikacji ze zcentralizowaną bazą danych i przerzucenie odpowiedzialności w przechowywaniu danych na pamięć wewnętrzną aplikacji. W celu dynamicznego skalowania aplikacji wprowadzana jest usługa replikacji danych pomiędzy aktywnymi jednostkami, aby w przypadku dużych obciążeń ruch mógł zostać przekierowany na inne instancje.



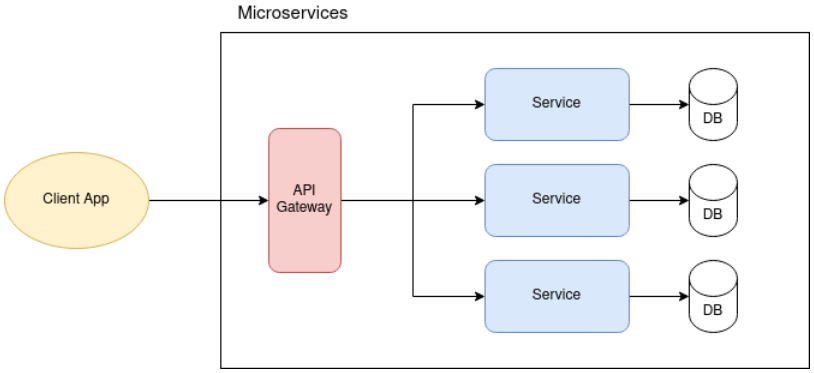
Rys 11. Struktura architektury opartej na przestrzeni kosmicznej. Źródło: P. Kuchana [19]

Na rysunku 11 można zauważyć, że oprócz jednostek aplikacji (ang. *Processing Unit*) w skład wzorca wchodzi również zwirtualizowane oprogramowanie pośrednie (ang. *Virtualized Middleware*), które zawiera komponenty kontrolujące obsługę żądań oraz synchronizacje danych.

- Siatka przesyłania wiadomości (ang. *Message Grid*) - odpowiada za równoważenie obciążenia i rozdzielania ruchu pomiędzy instancjami aplikacji,
- Siatka danych (ang. *Data Grid*) - odpowiada za poprawą replikacje danych pomiędzy serwisami,
- Siatka przetwarzania (ang. *Processing Grid*) - jest opcjonalnym komponentem, wykorzystywanym do przetwarzania żądań w przypadku, gdy każda z jednostek obsługuje inną cześć aplikacji,
- Menadżer wdrażania (ang. *Deployment Manager*) - zarządza cyklem życia jednostek aplikacji. [19]

- **Wzorzec architektury mikroserwisowej** (ang. *Microservice Architecture Pattern*)

Wzorzec powstał jako rozwiązanie ograniczeń wynikających z innych paradygmatów architektonicznych. Kluczem do zrozumienia wzorca architektury mikroserwisowej jest koncepcja modułów jako drobnoziarnistych serwisów. Architektura zachęca do porzucenia koncepcji ściśle powiązanych komponentów występujących w architekturze monolitycznej, na rzecz mikroserwisów, przedstawiając je jako niezależne, luźno powiązane jednostki. Dodatkowo upraszcza koncepcje usług i eliminuje nacisk związany z orkiestracją oferowaną przez wzorzec architektury opartej na usługach. Każda z mikrousług stanowi samodzielną aplikację skupiając się na jednej domenie biznesowej, dzięki temu wzorzec pozwala na wysoką elastyczność oraz skalowalność swoich rozwiązań. [19]



Rys 12. Struktura architektury mikroserwisowej. Źródło: opracowanie własne

2.4 Prawo Conway'a

W 1968 r. Melvin Conway w artykule pt. *How Do Committees Invent* [49] sformułował swoje prawo, które brzmi następująco: „Każda organizacja, która projektuje system (szeroko określony), nieuchronnie stworzy projekt, którego struktura będzie kopią struktury komunikacji w organizacji". Artykuł Conway'a został pierwotnie odrzucony przez redakcje „Harvard Business Review", ponieważ według redakcji nie bronił postawionej tezy. Jednak prawo Conway'a zostało dowiedzione na przestrzeni lat poprzez różne badania. [50]. Próbując interpretować słowa Conway'a, organizacja będzie wytwarzała produkt, który jest odwzorowaniem jej wewnętrznej struktury komunikacyjnej. Oprogramowanie to nie tylko zbiór zaprojektowanych funkcjonalności oraz wymagań, lecz są to również preferencje oraz opinie osób obsługujących i implementujących dany system.

Według Morgana Bruce'a i Paulo A. Pereira'y [34] odwrotność prawa Conway'a jest również prawdziwym stwierdzeniem. Jak sami piszą w swojej książce „Możemy skorzystać ze zmian w strukturze zespołu, aby uzyskać pożądaną architekturę". Autorzy podkreślają, że

struktura zespołu oraz architektura mikroserwisowa są powiązane ze sobą symbiotycznie oraz że oba czynniki mogą i powinny wpływać bezpośrednio na siebie.

Autorzy magazynu *Conway's Law Revisited: The Evidence for Task-Based Perspective* [62] wspominają o długotrwałym podejściu do zarządzania złożonymi projektami, które jest zgodne z prawem Conway'a. Mianowicie, podział systemu na mniejsze części i przekazanie odpowiedzialności za poszczególne komponenty konkretnym zespołom zmniejsza obciążenie komunikacyjne w zespole, co przyczynia się do stworzenia bardziej autonomicznych zespołów, a w konsekwencji bardziej autonomicznych usług.

3 Przegląd popularnych architektur oprogramowania

Architektura oprogramowania określa sposób w jakim dane oprogramowania będzie ewoluowało w kolejnych fazach życia projektu, dlatego ważnym elementem projektowania oprogramowania jest wybór odpowiedniej architektury. W niniejszym rozdziale przyjrzymy się dwóm najpopularniejszym podejściom do architektury oprogramowania, architekturze monolitycznej oraz w drugim kroku architekturze mikroserwisowej przedstawiając kluczowe aspekty oraz wady i zalety obu rozwiązań.

3.1 Architektura monolityczna

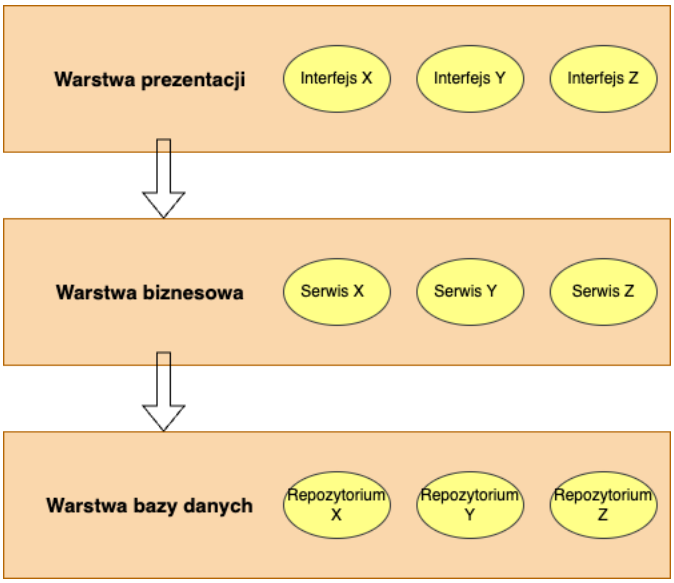
3.1.1 Charakterystyka architektury monolitycznej

W inżynierii oprogramowania architektura monolityczna uznawana jest za tradycyjną metodę budowania aplikacji. Ten styl architektoniczny, porównując do architektur rozproszonych od zawsze był preferowany przez programistów jako punkt startowy projektu ze względu na swoje proste podejście, które ułatwia inżynierom zarządzanie cyklem życia projektu. Aplikacje monolityczne są powszechnie nazywane monolitem, co w ogólnym znaczeniu oznacza dużą, niepodzielną jednostkę. Początkowo termin ten opisywał duże, samodzielne aplikacje, które poprzez liczbę dochodzących komponentów na przestrzeni lat, stają się coraz trudniejsze do utrzymania. W kontekście programistycznym, odnosi się to do aplikacji, której kod znajduje się w jednym repozytorium, jest skomplikowany i tworzy pojedynczy artefakt. Główną ideą aplikacji monolitycznych jest obsługa każdego kroku niezbędnego do wykonania danej funkcji, a nie tylko jednego określonego zadania. Oznacza to, że cała logika aplikacji jest zcentralizowana, co uniemożliwia rozdystrybuowanie procesów między różne usługi. Dzięki temu aplikacje unikają problemów wydajnościowy związany z komunikacją pomiędzy procesami (ang. *IPC*). Dodatkowo wszystkie dane są zarządzane przez jedną bazę danych, zwalniając architekta z problemu synchronizacji danych oraz obsługi rozproszonych transakcji ACID. [28]

Aplikacja monolityczna jest bardzo prosta do uruchomienia, ponieważ tak jak zostało już wcześniej wspomniane składa się tylko z jednego artefaktu. Zazwyczaj jest to jeden plik binarny w przypadku języków kompilowanych lub jedna aplikacja uruchomieniowa w przypadku języków interpretowanych. Dlatego możliwe jest uruchomienie jej jako pojedynczy proces na systemie operacyjnym lub bezpośrednio jako usługę chmurową (ang. *Serverless*). Ułatwia to również uruchomienie aplikacji przez programistę na lokalnym środowisku, co pozwala na szybkie debugowanie oraz testowanie. Proces wdrażania jest

również trywialny, ponieważ polega jedynie na podmianie poprzedniej wersji całego monolitu. Oznacza to, że nowa wersja aplikacji zastępuje starą w całości, bez konieczności rozdzielania i wdrażania poszczególnych modułów. Zmiany aplikacji mogą być wprowadzone szybko, co może być korzystne szczególnie na etapie rozwoju projektu. Natomiast wraz ze wzrostem rozmiaru aplikacji wzrasta też jej złożoność oraz staję się coraz trudniejsza do zarządzania. Duży rozmiar aplikacji wpływa na jej czas uruchamiania, w rezultacie spowalnia proces tworzenia oraz testowania oprogramowania. Długi czas uruchomienia wpływa również na czas wdrożenia, staję się to uciążliwą i kosztowną operacją, ponieważ wydając najmniejszą zmianę dla jednego z modułów musimy przebudować całą aplikację. Dodatkowo z czasem jakiekolwiek zmiany w jednym miejscu poprzez duże powiązania w kodzie mogą wywoływać błędy w innych, co może prowadzić do kaskady innych błędów. [27]

Wzorowym przykładem architektury monolitycznej jest aplikacja wykorzystująca architekturę trójwarstwową z warstwami: bazy danych, biznesu oraz prezentacji. Aplikacje korporacyjne często budowane są na podstawie tego modelu. Pierwsza warstwa odpowiedzialna jest za graficzny interfejs aplikacji oraz interakcje użytkownika z warstwą biznesową. Warstwa biznesowa dostarcza logikę biznesową po stronie serwera dla systemu. Jest to warstwa najbardziej obciążona na zmiany w raz z czasem trwania rozwoju projektu. Odpowiada za wykonanie logiki biznesowej, obsługę zapytań HTTP oraz przesyłanie danych do warstwy prezentacji. Ostatnia warstwa odpowiedzialna jest za komunikację z aplikacji z bazą danych. Przykładowa architektura została zaprezentowana na poniższej ilustracji.



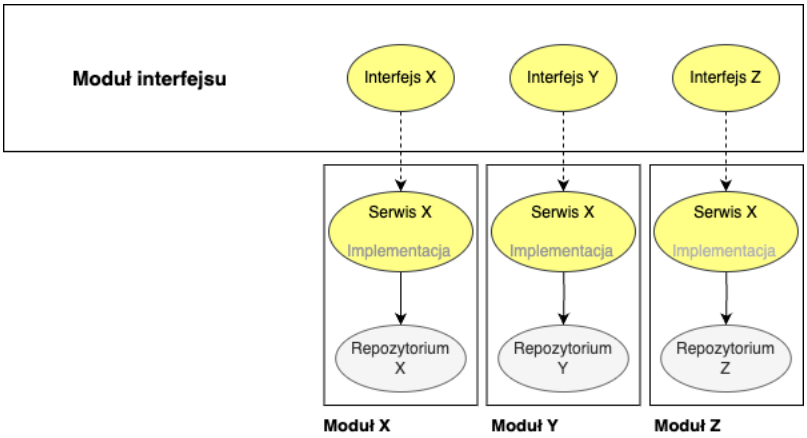
Rys 13. Schemat architektury trójwarstwowej. Źródło: P. Kuchana [19]

3.1.2 Modularny monolit

Architektura monolityczna cechuje się brakiem jawnego wyznaczenia modularności na poziomie projektowania oraz występowaniu szerokich powiązań i współzależności pomiędzy komponentami wchodzącymi w skład oprogramowania. Nie oznacza to jednak, że w systemach monolitycznych modularność nie jest pożądana, ponieważ umożliwia ona lepszą organizację oraz zarządzanie kodem, a także uskutecznia wymianę lub naprawę wadliwych komponentów. Istnieje również podejście, które zakłada zaprojektowanie systemu monolitycznego w sposób modularny, w którym występują autonomiczne moduły biznesowe przy jednoczesnym zachowaniu niskiej złożoności projektu. Moduły mogą być wydzielone przy użyciu przestrzeni nazw lub oddzielnych wtyczek i bibliotek dołączanych do aplikacji. Zapewniają one luźno powiązane moduły oraz dobrze zdefiniowane interfejsy z hermetyzowanym dostępem do implementacji. Takie podejście ułatwia przejście na architekturę rozproszoną, ponieważ mamy już wydzieloną infrastrukturę, która w przyszłości może zostać uruchomiona jako oddzielny serwis. [29]

Bazując na przykładzie z rys. 13 przedstawiającego architekturę trójwarstwową i zakładając, że każda warstwa zawiera się w oddzielnym pakiecie nie mamy możliwości kontroli, kto tak naprawdę ma i powinien mieć dostęp do poszczególnych interfejsów. Jedyną możliwość nadzoru dostępów do interfejsów jest stosowanie się do pewnej konwencji wyznaczonej wewnątrz organizacji, natomiast architektura systemu nie jest w stanie tego zapewnić. Dodatkowo, gdy aplikacja się rozrasta, utrzymywanie zmian przez programistów w konkretnych modułach, tak aby nie rozprzestrzeniały się na inne może być stosunkowo trudne do utrzymania, szczególnie przy dużych rotacjach osób w projekcie. [29]

Poniższy przykład prezentuje podział architektury z rys 13 na moduły, wzorując się na modularnym monolicie. Kolorem szarym zaznaczone zostały elementy zagnieżdżone w poszczególnych modułach, takie jak warstwa bazy danych oraz implementacja logiki biznesowej i do których dostęp nie jest możliwy spoza modułu. W taki sposób skutecznie otrzymujemy luźno powiązane moduły, z hermetyzowaną implementacją.



Rys 14. Schemat architektury trójwarstwowej z wydzieleniem modułów. Źródło: opracowanie własne

3.1.3 Zalety i wady architektury monolitycznej

Systemy monolitycznie kojarzone są z przestarzałymi systemami (ang. *System legacy*). Często jest to niesłuszne skojarzenie, ponieważ dobrze ustrukturyzowany i systematycznie udoskonalany monolit może przynosić korzyści biznesowe przez lata. W tej części pracy skupimy się na zaletach oraz wadach tej architektury. Dokładne zrozumienie zarówno mocnych, jak i słabych stron architektury monolitycznej pozwoli lepiej ocenić, w jakich kontekstach jest ona odpowiednia, a kiedy warto rozważyć alternatywne podejścia. Na początku skupimy się na przedstawieniu pozytywnego aspektu omawianej architektury.

- **Dobry wybór na początkowy etap projektu**

  W przypadku nowych projektów, startupów, gdzie wymagania mogą się klarować podczas tworzenia projektu, architektura monolityczna jest bardzo efektywnym rozwiązaniem. Sprawdza się ona doskonale w sytuacjach, gdzie potrzebna jest szybka iteracja i łatwe dodawanie nowych funkcji, ponieważ umożliwia łatwe zarządzanie jednym kodem źródłowym, jednym modelem danych i jednym procesem wdrożeniowym. Dzięki temu startupy mogą szybko reagować na zmieniające się wymagania rynkowe i użytkowników, co jest kluczowe na początkowym etapie rozwoju. Wiele dużych korporacji odnoszących ogromne sukcesy, takich jak Netflix, Amazon czy Uber zaczynały swoje projekty od monolitu, stosując zasadę najpierw monolit (ang. *Monolith first)*, mówiącą o tym, że projekt informatyczny powinien zaczynać się od aplikacji monolitycznej przed przejściem na architekturę rozproszoną, ze względu na prostotę w początkowej fazie [30]. Na początku, przy stosunkowo niewielkiej liczbie procesów i łatwości określenia wymagań, architektura monolityczna była lepszym rozwiązaniem. Była wydajna i dobrze sprawdzała się przy wąskim zakresie funkcji, który często pozostawał stabilny przez dłuższy czas. Dopiero później, gdy firmy się rozwijały, zwiększały zakres swoich usług oraz liczbę

użytkowników i wymagały szybszego skalowania, przechodziły na architekturę mikroserwisów, która lepiej spełniała ich nowe potrzeby. [27]

- **Prostota i łatwość w zarządzaniu**

  Architektura monolityczna charakteryzuje się jednolitym i zwartym podejściem do budowy oprogramowania, które sprawia, że zarządzanie projektem informatycznym nie jest wcale tak skomplikowane. Tak jak już wcześniej zostało wspomniane, monolit jest zbudowany jako jedna jednostka, która zawiera cała logikę biznesową oraz wszystkie komponenty w jednym kodzie źródłowym. Dzięki temu, programista może pracować tylko nad jednym kodem źródłowym, bez konieczności koordynowania zmian między różne serwisy.

- **Łatwość w testowaniu**

  Przeprowadzanie testów w aplikacji monolitycznej jest stosunkowo proste, ponieważ testy jednostkowe, integracyjne oraz end-to-end mogą obejmować system w całości. Eliminuje to potrzebę tworzenia skomplikowanych środowisk testowych, które symulowałyby interakcje pomiędzy mikroserwisami. Ponadto testowanie lokalnie jest łatwiejsze, dlatego że środowisko wymaga uruchomienia tylko jednej aplikacji. Dzięki temu programiści mogą testować zmiany efektywniej, bez potrzeby uruchamiania wielu usług i zarządzania ich zależnościami.

- **Scentralizowany monitoring**

  Aplikacja monolityczna nie tylko ułatwia proces wytwarzania oraz testowania oprogramowania, ale również monitorowania systemu na produkcji. Logi w monolicie gromadzone są w jednym, centralnym miejscu, co znacząco przyśpiesza identyfikację i naprawę problemów. Brak konieczności integracji logów z wielu instancji lub różnych serwisów, wykorzystując przy tym specjalne identyfikatory w celu śledzenia danego zapytania między różnymi systemami redukuje złożoność monitorowania aplikacji.

- **Samowystarczalna jednostka wdrożeniowa**

  Aplikacja monolityczna jest kompletnym oprogramowaniem biznesowym, które samodzielnie jest gotowe do wdrożenia i działania produkcyjnego w izolacji, bez konieczności współpracy z innymi usługami. Monolit zawiera wszystkie niezbędne komponenty potrzebne do prawidłowego działania systemu. Przy tylko jednej jednostce wdrażania proces wdrażania aplikacji jest relatywnie prosty. Wystarczy tylko jeden proces

kompilacji i wdrożenia, co eliminuje potrzebę synchronizacji wielu serwisów i minimalizuje ryzyko błędów związanych z integracją.

Gdy rozmiar aplikacji wrasta, wrasta również liczba jej funkcjonalności oraz linii kodu. Aplikacje stają się bardziej skomplikowane i trudniejsze w zarządzaniu. Mimo przedstawionych wyżej zalet, architektura monolityczna posiada również wiele wad, które przy przedstawionych okolicznościach stają się niemałym problemem dla zespołów programistycznych.

- **Brak modularności i elastyczności**

  W architekturze monolitycznej często komponenty są ze sobą ściśle powiązane, przez co zmiany w jednym miejscu wymagają zmian w innych. Silne powiązania utrudniają modyfikację aplikacji, ponieważ każda zmiana musi być przetestowana pod kątem wpływu na cały system. Ogranicza to również elastyczność aplikacji pod względem zmian bibliotek lub technologii, ponieważ zmiana jednej części aplikacji może wymagać dostosowania całej aplikacji pod tą zmianę. Może być to czasochłonna i kosztowna operacja.

- **Złożoność rośnie wraz z rozmiarem**

  W miarę rozwoju funkcjonalności aplikacji monolitycznej liczba kodu wzrasta, a z tym procesy zarządzania stają się bardziej złożone. System stopniowo staję się kłopotliwy, a wprowadzanie kolejnych zmian oraz testowanie ich jest bardziej czasochłonne. Z czasem coraz trudniej jest zachować czysty kod, przez co zmiany w jednym miejscu mogą wpływać na działanie innego modułu. Utrudnia to utrzymanie zmian, które powinny wpływać na jedną cześć systemu i może prowadzić do niestabilności aplikacji. Dodatkowo każda najmniejsza zmiana w aplikacji wymaga kompilacji i wdrożenia całego monolitu. [28]

- **Trudność w pracy wielu zespołów nad jednym kodem**

  Podczas pracy nad oprogramowaniem w architekturze monolitycznej jest bardzo mała szansa, że zespół będzie miał autonomiczność nad projektem. W dużych projektach, najczęściej poszczególne komponenty są rozdystrybuowane pomiędzy różne zespoły. Często granice odpowiedzialności pomiędzy zespołami mogą być nie jasne i może to utrudniać utrzymanie jakości kodu oraz zarządzaniem projektem. Dodatkowo, kiedy wiele

zespołów pracuje nad jednym źródłem kodu, zmiany wprowadzone przez jeden zespół mogą powodować konflikty ze zmianami innych zespołów. Dlatego zmiany w kodzie muszą być starannie zsynchronizowane, aby uniknąć problemów podczas integracji kodu.

- **Jednolity stos technologiczny**

  Aplikacje monolityczne ograniczają się do korzystania z jednego stosu technologicznego. Takie rozwiązanie nie zawsze jest odpowiednie dla wszystkich domen i nakazuje korzystanie ze starszych technologii nawet w nowszych modułach. Dodatkowo w przypadku większych aplikacji ciągła aktualizacja stosu technologicznego do obowiązujących trendów i standardów, może być bardzo czasochłonna przez co często jest pomijanym krokiem. [31]

- **Ograniczona skalowalność**

  Aplikacje monolityczne uruchamiane są jako pojedynczy proces. Jest to znaczące ułatwienie podczas wytwarzania oprogramowania w początkowej fazie, natomiast wraz z wzrostem generowanego ruchu w aplikacji spada jej wydajność jak i również zadowolenie klienta. Architektura monolityczna ogranicza skalowalność aplikacji, ponieważ skalowanie poziome wymaga dodawania kolejnych instancji aplikacji, które z reguły są dosyć duże, co może prowadzić do większego zużycia zasobów. Często również tylko część aplikacji jest obciążona, dlatego duplikowanie całej aplikacji może nie mieć większego sensu. Warto wspomnieć, że można próbować skalować aplikacje w sposób pionowy, zwiększając zasoby serwera. Natomiast te serwery mają ograniczoną pojemność i większa liczba zasobów generuje większe koszty. [32]

Podsumowując, architektura monolityczna, dzięki swojej prostocie jest często wybierana przy planowaniu nowych produktów, zwłaszcza w mniejszych firmach produktowych, tak zwanych start-upach. Projekty monolityczne zapewniają szybki czas wdrożenia oraz relatywnie niskie koszty zarządzania i utrzymania projektu, porównując z bardziej skomplikowanymi architekturami. Natomiast, w miarę rozwoju projektu wady architektury monolitycznej mogą stać się poważnym ograniczeniem i hamulcem do dalszego rozwoju aplikacji. Może to powodować konieczność przejścia na bardziej rozproszony rodzaj architektury taki jak architektura mikroserwisowa.

3.2 Architektura mikroserwisowa

3.2.1 Charakterystyka architektury mikroserwisowej

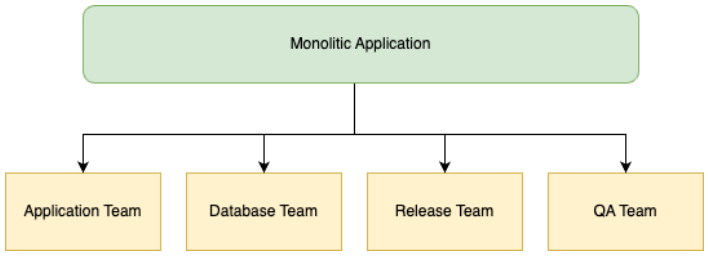
Tradycyjne podejście do tworzenia oprogramowania zaleca, aby dobrze zaprojektowany system charakteryzował się wysoką spójnością oraz luźnymi powiązaniami. System identyfikujący się z tymi wartościami jest z reguły prostszy i bardziej elastyczny, co przekłada się na łatwiejsze przeprowadzenie zmian w systemie. W kontekście projektów informatycznych spójność odnosi się do stopnia w jakim elementy danego modułu są ze sobą powiązane. Natomiast powiązanie oznacza stopień wiedzy danego elementu o wewnętrznym działaniu drugiego. W systemach opartych na architekturze monolitycznej dążymy do osiągnięcia takiego stanu na poziomie klasy lub szerzej, modułów i komponentów. W przypadku architektury mikroserwisowej, dążymy do sytuacji, w której każda usługa jest wysoce spójna, co oznacza, że jest odpowiedzialna w pełni tylko za jedną funkcjonalność oraz luźno powiązana, czyli każda usługa powinna znać tylko wymagane minimum o innych, z którymi współpracuje. Im mniej powiązane są usługi, tym łatwiej dokonać zmiany w jednej z nich bez wymuszania zmiany na powiązanych usługach. [34]

Jedna z pierwszych definicji architektury mikroserwisowej została przedstawiona w artykule J.Lewisa i M.Fowlera pt. *Microservices: A Definition of This New Architectural Term* [33]. Według definicji zawartej w tym artykule, architektura mikroserwisowa jest podejściem do tworzenia aplikacji jako zbioru małych, autonomicznych usług, które komunikują się ze sobą za pomocą dobrze zdefiniowanych interfejsów i wyznaczonych kontraktów. Architektura mikroserwisowa polega na podzieleniu jednego, dużego systemu na wiele mniejszych serwisów, które są odpowiedzialne za jedną funkcjonalność i współpracują ze sobą w celu wykonywania bardziej złożonych operacji. Może to być funkcjonalność zawierająca logikę biznesowa lub techniczną, integrująca się z innym systemem. [34]

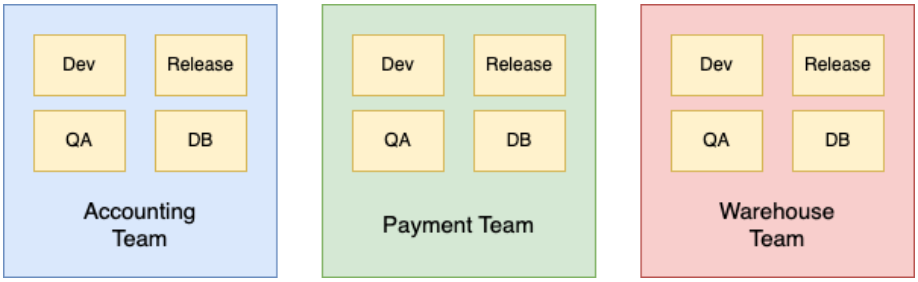
Jednym z głównych powodów, dla których architektura mikroserwisowa wykorzystuje komponenty aplikacji jako oddzielne serwisy jest fakt, że każdy z nich stanowi niezależną i wymienną jednostkę. W przypadku gdy aplikacja zawiera wiele komponentów, które zostały zintegrowane poprzez biblioteki, każda zmiana w pojedynczym komponencie wymaga przebudowania całej aplikacji. Natomiast w aplikacji opartej na mikroserwisach każdy mikroserwis może być wdrożony niezależnie, dlatego wystarczające jest przebudowanie jednej usługi, dokładniej tej w której zaszły zmiany. Należy jednak pamiętać, że niektóre zmiany w wyjściowych interfejsach usług, mogą wymagać modyfikacji w serwisach, które je

wywołują. Celem dobrego mikroserwisu jest zminimalizowanie takich przypadków poprzez poprawne zdefiniowanie granic oraz kontraktów pomiędzy usługami. [33]

Architektura mikroserwisowa utożsamia się z pięcioma zasadami, które określają poziom zaprojektowanej architektury. Zasady te nie tylko stanowią dobrą praktykę dla programistów, ale powinny wpływać na decyzję techniczne i organizacyjne podejmowane przez zarządzający zespół. Pierwszą zasadą jest autonomia, której temat został już poruszony w tym rozdziale. Aby osiągnąć autonomie w mikroserwisie, usługi powinny być ze sobą luźno powiązane oraz niezależne przy wdrażaniu. Zgodnie z Prawem Conwaya, struktura organizacyjna zespołu ma bezpośredni wpływ produkt. Dlatego ta zasada podkreśla również, że zespół programistyczny powinien być obarczony całkowitą odpowiedzialnością za swój produkt za równo w okresie rozwoju jak i na produkcji. Pozwala to na pełną kontrole nad jakością i szybsze reagowanie na problemy. Taki model nie jest stosowany w klasycznym podejściu wytwarzania oprogramowanie, gdzie zespół programistyczny odpowiedzialny za projekt dostarcza tylko oprogramowanie, które w dalszej fazie życia zarządzanie jest przez inną organizację. Powinniśmy się skupić na tworzeniu produktu, a nie projektu. Rysunki 15 oraz 16 przedstawiają strukturę organizacyjną zespołów w architekturze monolitycznej oraz mikroserwisowej. Autonomiczność może być również rozumiana w kontekście technologicznym, ponieważ każdy mikroserwis może być zbudowany w oparciu o technologię, która najlepiej wpasowuję się w dany problem. Ze względu na to, że każdy moduł stanowi oddzielny produkt bez problemu możemy zachować agnostyczność technologiczną. [35]



Rys 15. Struktura organizacyjna zespołów w architekturze monolitycznej. Źródło: V. Velupecha i P. Flores [37]



Rys 16. Struktura organizacyjna zespołów w architekturze mikroserwisowej. Źródło: V. Velupecha i P. Flores [37]

Architektura mikroserwisowa charakteryzuję się również odpornością na błędy. Podział systemu na wiele niezależnych serwisów daje nam większą gwarancję, że w przypadku awarii jednego z nich awaria będzie izolowana przez dany serwis i nie wpłynie to na resztę funkcjonalności. Niestety przez to, że każdy mikroserwis stanowi oddzielny punkt w architekturze to zwiększamy liczbę potencjalnych punktów wystąpienia awarii. Dlatego warto wdrożyć odpowiednie mechanizmy awaryjne, które pomogą zapobiec kaskadzie błędów i pomogą nam w śledzeniu błędów. Każdy serwis powinien udostępniać duże ilości metryk jak i dzienników logowań zarówno biznesowych jak i związanych z aplikacją oraz infrastrukturą systemu. Tak aby system był w każdym momencie przejrzysty i umożliwiał monitorowanie oraz szybką diagnozę problemu, co stanowi kolejną zasadę architektoniczną, przejrzystość. [35]

Kolejną ważna zasadą przy implementacji mikroserwisów jest automatyzacja. System zbudowany z wielu mniejszych jednostek jest bardziej skomplikowany i wymaga więcej czasu poświęconego na wdrożenie niż w przypadku aplikacji monolitycznej. Przez co proces staję się bardziej kosztowny, dlatego ważne jest, aby zapewnić automatyzacje przy wdrożeniach stosując procesy CI/CD (ang. *Continuous Integration/Continuous Delivery*) [47] jak i przy monitoringu stosując narzędzia do monitorowania systemu. Wraz z architekturą mikroserwisowa popularnym rozwiązaniem stało się podejście infrastruktury jako kodu (ang. *IaC*) [48]. Podejście to sprowadza proces zarządzania zasobami systemu do postaci kodu zamiast wykonywania ręcznej konfiguracji. Pozwala to na przechowywanie skryptów oraz szablonów konfiguracyjnych w repozytoriach kodu, dzięki czemu mogą być w łatwy sposób replikowane pomiędzy inne usługi. [35]

Ostatnią zasadą jest ukierunkowanie. W celu osiągniecia większej spójności, bardzo istotnie jest, aby dążyć do uporządkowania usług w systemie, a co za tym idzie również zespołów programistycznych zgodnie z koncepcjami biznesowymi. Architektura mikroserwisowa powinna być zorientowana na poziomą dekompozycje, w której każda usługa jest kompletna, tzn. obejmuję w pełni implementacje logiki biznesowej oraz warstwy techniczne. [34] Oprócz decentralizacji aplikacji na mniejsze, architektura mikroserwisowa pożąda również decentralizacji bazy danych. Mikroserwisy umożliwiają każdej z usług bezpośrednie zarządzanie własną bazą danych, które mogą korzystać z różnych technologii, zależnie od specyficznych potrzeb. Dzięki decentralizacji bazy danych serwisy są zhermetyzowane, a interakcja z nimi jest możliwa wyłącznie za pośrednictwem udostępnionego interfejsu API. [28] Rozwiązanie wykorzystujące jedną bazę danych jest również wykorzystywane, ze względu na mniejsze koszty infrastrukturalne, natomiast konieczne jest,

aby granice serwisów były dobrze zdefiniowane i nie wychodziły na inne usługi, tak aby zmiany w danych wpływały na pojęcie niezależności usługi.[35]

Architektura mikroserwisowa jest bardzo elastyczna, komponowanie systemu za pomocą małych autonomicznych usług sprawia, że każda usługa może być zastąpiona nową implementacją bez wpływu na komunikacje z innymi serwisami oraz na działanie całego systemu. Jednocześnie każda usługa, która nie jest używana w systemie może być w bezpieczny sposób wyłączona. [37]

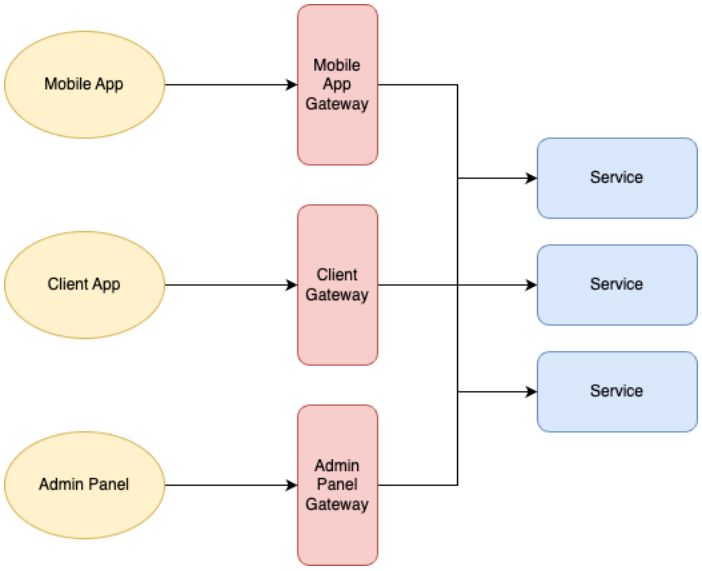
Diagram ilustrujący schemat architektury mikroserwisowej, w której każdy serwis wykorzystuje niezależną bazę danych, został przedstawiony na rysunku 12.

3.2.2 Wzorce projektowe w architektury mikroserwisowej

Wzorce projektowe odgrywają istotną role w inżynierii oprogramowania przy tworzeniu elastycznych i łatwo skalowalnych rozwiązań. Definicja wzorca projektowego przedstawiona została w rozdziale 2.2. W niniejszym rozdziale omówione zostaną kluczowe wzorce projektowe wykorzystywane przy projektowaniu architektury mikroserwisowej oraz dekompozycji aplikacji monolitycznych.

- **Wzorzec BFF** (ang. *Backend For Frontend Pattern*)

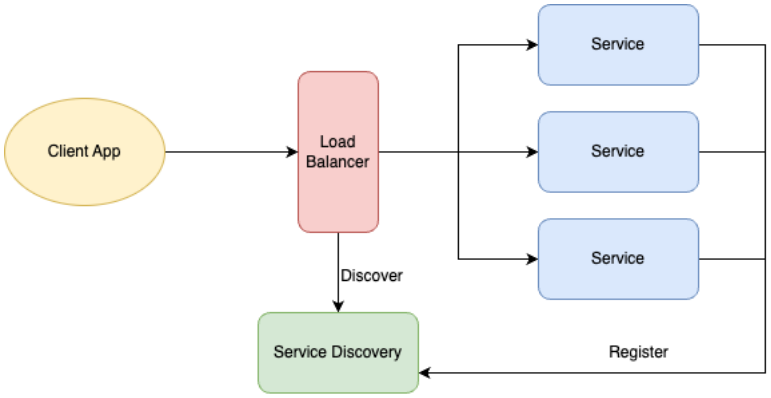
  Wzorzec BFF jest to wzorzec, który usprawnia sposób pozyskiwania danych między różnymi aplikacjami klienckimi. Celem tego wzorca jest oddzielenie aplikacji frontendowych od backendu, oddzielając każdego klienta dodatkową warstwą translacyjną, na przykład bramą interfejsu API (ang. *API Gateway*) [36]. Wzorzec ten zapewnia izolację aplikacji klienckich od backendu. Jest szczególnie przydatny w sytuacji, gdy aplikacja serwerowa jest wykorzystywana przez więcej niż jedną aplikację kliencką, a każda z nich używa innych struktur interfejsów. W takim przypadku wzorzec ten nie wymaga wprowadzania zmian w kodzie aplikacji serwerowej, a jedynie skonfigurowania nowej bramy i dostosowania wywołań interfejsów do nowej usługi. Wykorzystanie bramy API umożliwia również maskowanie struktury API, co podnosi poziom bezpieczeństwa systemu. Przykład architektury mikroserwisowej z wykorzystaniem wzorca BFF przedstawiony został na rysunku 17. [37]



Rys 17. Przykład architektury wykorzystującej wzorzec BFF. Źródło: V. Velupecha i P. Flores [37]

- **Wzorzec wykrywania usług** (ang. *Service Discovery Pattern*)

  W przypadku złożonych aplikacji, w których usługi są mnożone w celu uzyskania lepszej wydajności, system musi być świadomy, które instancje są aktywne i gdzie są hostowane, a które nie, aby móc odpowiednio przekierowywać zapytania. Wzorzec wykrywania usług opiera się na wykorzystaniu tzw. Rejestru Usług (ang. *Service Discovery*) [38], który przechowuję informacje o wszystkich aktywnych usługach wykorzystywanych w środowisku. Rejestracja usług polega na automatycznym wykrywaniu usług działających w systemie i ich rejestrowaniu w serwisie rejestrującym. Każda aplikacja podczas uruchomienia przesyła zapytanie do serwisu rejestrującego, który rejestruję tą usługę i następnie regularnie przesyłają swój status, aby rejestrator wiedział czy dana usługa jest nadal aktywna. Wzorzec ten doskonale współpracuje z metodą równoważenia obciążenia (ang. *load balancing*) [39], która odpowiada za równomierny podział obciążenia pomiędzy instancje usług. Rejestr usług dostarcza modułowi równoważenia obciążenia informacje na temat dostępności oraz lokalizacji usług oraz ich instancji. [37] Na poniższym rysunku 18 przedstawiona został schemat z użyciem omawianego wzorca.



Rys 18. Przykład architektury wykorzystującej wzorzec wykrywania usług. Źródło: V. Velupecha i P. Flores [37]

- **Wzorzec współdzielenia danych** (ang. *Shared Data Microservice Design Pattern*)

  Zgodnie z nazwą, wzorzec sugeruję, wykorzystanie jednej bazy danych dla wszystkich usług w systemie. Dekompozycja bazy danych na wiele baz danych może być procesem skomplikowanym i kosztownym, dlatego czasami architekci decydują się na pozostanie przy wspólnej instancji W tym przypadku należy zapewnić, że tabele i schematy nie są współdzielone pomiędzy różnymi serwisami oraz że nie występują pomiędzy nimi żadne powiązania na poziomie bazy danych. Jak wspomniano we wcześniejszym rozdziale, takie podejście jest możliwe, natomiast może być uznane jako antywzorzec, ponieważ mikroserwisy powinny być zamkniętymi jednostkami. Niemniej jednak, wzorzec ten może okazać się przydatny w procesie dekompozycji, gdzie utrzymanie współdzielonej instancji na czas transformacji, pozwala w pełni skupić się na przepisywaniu aplikacji na mikroserwisy i może sprawić, że proces ten będzie bardziej efektywny. [37]

- **Wzorzec dusiciela** (ang. *Strangler Pattern*)

  Wzorzec dusiciela jest strategią dekompozycji aplikacji monolitycznej na mikroserwisy. Podeście polega na stopniowym zastępowaniu modułów aplikacji monolitycznej na mikroserwisy wykorzystując strukturę strony internetowej i URI [40]. Kluczową cechą tego podejścia jest to, że zarówno mikroserwisy, jak i oryginalna aplikacja monolityczna są wystawione do Internetu [41] pod tą samą domeną i w miarę postępów dekompozycji konkretne URI przekazują odpowiedzialność na mikroserwisy. Wzorzec dusiciela składa się z trzech kroków:

    - Transformacja (ang. *Transform*) - polega na stworzeniu nowej witryny równoległej do istniejącej i stopniowym przenoszeniu na nią funkcjonalności w architekturze mikroserwisowej,

- Współistnienie (ang. *Coexist*) - w tej fazie nowy serwis wchodzi na produkcje i oryginalna aplikacja przekierowuje użytkowników do nowej witryny dla nowo przepisanych funkcjonalności,
- Eliminacja (ang. *Eliminate*) - polega na trwałym usunięciu starej części monolitu i zastąpienia jej nową.

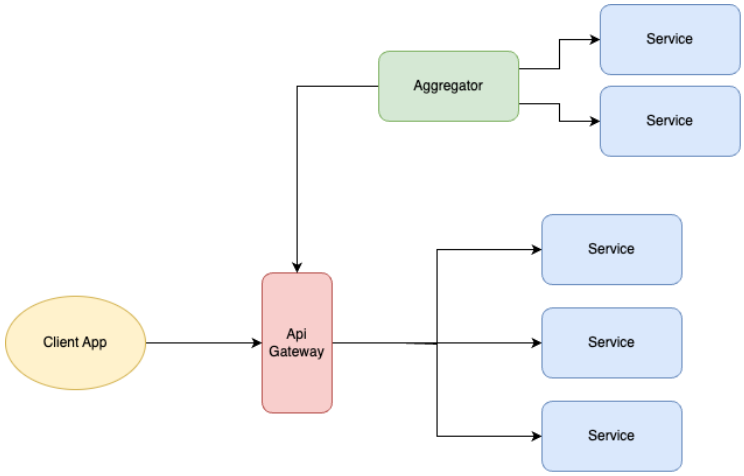
Wzorzec wykorzystuje iteracyjne podejście dekompozycji monolitu, powyższe kroki powinny być stosowane aż do momentu przepisania wszystkich wydzielonych modułów na mikroserwisy i pełnym zastąpieniu monolitu na produkcji. [37]

- **Wzorzec adapter** (ang. *Adapter Microservices Pattern*)

  Główna idea wzorca adaptera w kontekście mikroserwisów jest analogiczna do tradycyjnego podejścia, z różnicą, że zamiast dostosowywania interfejsów w kodzie skupiamy się na konkretnych serwisach w ramach architektury systemu. Wzorzec ten odpowiada za tworzenie mikroserwisów, których jedynym zadaniem jest pośredniczenie i mapowanie danych pomiędzy zewnętrznym API, a naszym systemem. Takie podejście może być szczególnie użyteczne w przypadku potrzeby dostosowania starego serwisu do standardów całego systemu, na przykład poprzez zmapowanie usługi opierającej się na protokole SOAP [42] na nowszy standard REST [43]. Równie powszechnym zastosowaniem wzorca adaptera jest integracja z zewnętrznym API. Uodparniamy się w ten sposób na nieprzewidziane zmiany w zewnętrznych usługach minimalizując ryzyko błędów w systemie. [37, 44]

- **Wzorzec Agregator** (ang. *Aggregator Pattern*)

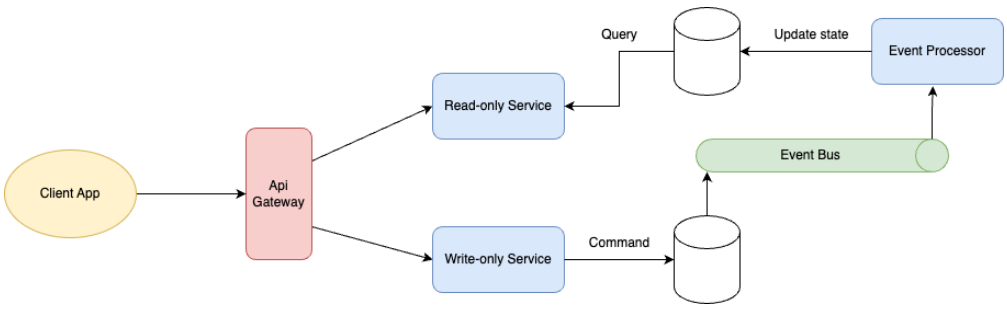
  Wzorzec Agregatora wprowadza niezależny serwis do architektury systemu, tzw. agregator, który odpowiada za konsolidacje danych lub funkcjonalności z różnych serwisów. Agregator jest przydatny, gdy istnieje potrzeba agregacji lub przetworzenia danych w systemie z wielu źródłem i zwrócenia ich do usługi końcowej podczas pojedynczego zapytania. Na rysunku 19 przedstawiona została architektura systemu z wykorzystaniem agregatora. [45]



Rys 19. Przykład architektury wykorzystującej wzorzec agregator. Źródło: V. Velupecha i P. Flores [37]

- **Wzorzec CQRS** (ang. *Command Query Responsibility Segregator*)

  Wzorzec CQRS jest wzorcem projektowym, który odseparowuje operacje odczytu od zapisu, operując na dwóch oddzielnych bazach danych. Separacja ta określa ilość danych, które trzeba przenieść z zapisu do odczytu, aby zachować ostateczną spójność między nimi. Wzorzec CQRS zapewnia, że dane mogą być odczytane lub zapisane, ale nie umożliwia wykonania obu operacji na raz. Aplikacja obsługująca operacje odczytujące z bazy danych, aktualizuje stan bazy po każdym zapisie w drugim komponencie. Komunikacja pomiędzy aplikacjami odbywa się poprzez zdarzenia, wykorzystując wzorzec architektury skierowanej na zdarzenia. Serwis zapisujący przesyła notyfikacje o nowym rekordem na kolejkę, która jest subskrybowana przez serwis uprawniony do odczytu. Następnie modyfikuję bazę o nowe elementy. [46] Przykład wykorzystujący wzorzec CQRS został przedstawiony na poniższej ilustracji.

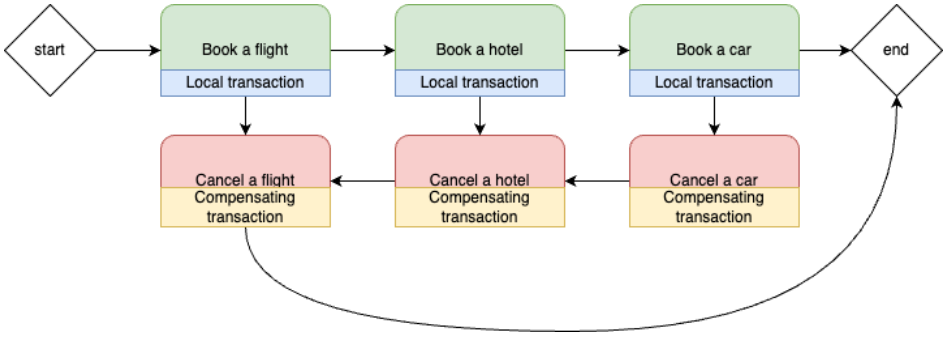


Rys 20. Przykład architektury wykorzystującej wzorzec CQRS. Źródło: D. Surya Sai Venkatesh i S. Agarwal [46]

- **Wzorzec Sagi** (ang. *Saga Pattern*)

  Wzorzec Sagi został wprowadzony w celu rozwiązania problemów transakcji rozproszonych wykorzystujących algorytm dwufazowego zatwierdzania (ang. *Two-phase*

commit) [54]. Wzorzec saga zastępuje jedną, blokującą transakcję rozproszoną skoordynowaną sekwencją indywidualnych transakcji lokalnych. W sadze każdy poprzedni krok uruchamia kolejny. Jeżeli jedna transakcja z sekwencji nie powiedzie się, uruchamiane są transakcje kompensacyjne w celu cofnięcia zmian i przywrócenia systemu do spójnego stanu. To czy operacje zostaną cofnięte do stanu pierwotnego zależy od implementacji semantyki biznesowej. Wadą sag w porównaniu do transakcji ACID jest brak izolacji. Wynik każdej lokalnej transakcji jest widoczny dla innych transakcji przed zakończeniem sagi. Oznacza to, że zasób może uczestniczyć w kilku sagach na raz.



Rys 21. Przykład wzorca sagi w podejściu choreograficznym. Źródło: M. Bruce et al. [34]

Saga może być sterowana na dwa sposoby, choreograficznie lub wykorzystując orkiestrowanie. Podejście choreograficzne jest podstawowym podejściem sagi w którym zdarzenia są wymieniane bezpośrednio pomiędzy uczestnikami sagi bez stosowania procesu koordynacyjnego. Każda kolejna transakcja jest uruchamiana przez komunikaty generowane przez poprzednie transakcje lokalne. Sprawia to, że serwisy są bardziej autonomiczne, ponieważ same odpowiadają za rozwiązanie transakcji. Natomiast w sadze wykorzystującej orkiestracje jedna z usług lub niezależna usługa jest koordynatorem procesu sagi. Orkiestrator jest automatem skończonym, który odpowiada za śledzenie i przechowywanie stanów zadań w odrąbie danej sagi oraz przywróceniu stanu prawidłowego stanu uczestnikom transakcji. Orkiestracja ułatwia śledzenie statusów oraz postępów lokalnych transakcji w sadze. [34,56]

- **DDD** (ang. *Domain Driven Design*)

  DDD jest to podejście do projektowania oprogramowania, które polega na głębokim zrozumieniu i modelowaniu domeny biznesowej dla której tworzymy system. Centralizacja wiedzy pomiędzy programistami, a analitykami domeny odbywa się poprzez podstawowe zasady: dyskusje, słuchanie, zrozumienie, odkrywanie i wartość biznesową. DDD przede wszystkim nie dotyczy technologii, ale zrozumienia domeny biznesowej

i odzwierciedlenia jej złożoności w kodzie. Do opisania modeli i procesów biznesowych wykorzystywany jest uniwersalny język, który powinien być zrozumiały zarówno dla programistów jak i analityków domeny, osób nietechnicznych. Jednym z celów DDD jest wyznaczenie podziału głównej dziedziny na poddziedziny. Dziedzina określa role biznesową jaką zajmuję się dana organizacja, natomiast poddziedzina jest oddzielnym procesem biznesowym, który składa się na główną dziedzinę. Kontekst ograniczony wyznacza granice w której istnieje model dziedziny. Terminologia wewnątrz kontekstu ograniczonego ma takie same znaczenie, a język wykorzystywany wewnątrz kontekstu należy do danego modelu. [60] W przypadku architektury mikroserwisowej, dziedzina może składać się na rzecz całego systemu, natomiast poddziedzina może być realizowana jako oddzielny mikroserwis wyznaczony za pomocą kontekstu ograniczonego. Stosowanie DDD do określania wymagań mikrousługi prowadzi do otrzymania wysokiej spójności w serwisie. [37]

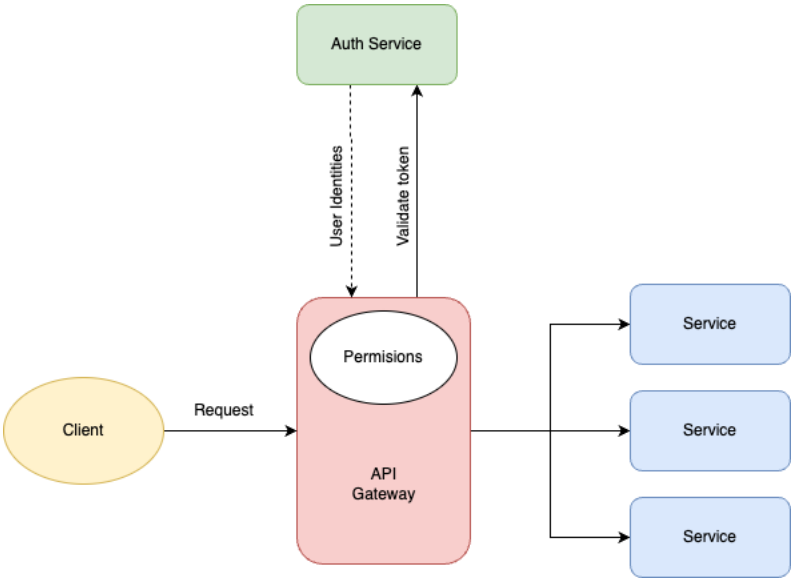
3.2.3 Wyzwania i trudności architektury mikroserwisowej

Architektura mikroserwisowa definiowana jest jako podejście, w którym wykorzystuje się niewielkie, niezależne usługi do tworzenia całości systemu. Jednakże, wiążę się ono również z wieloma wyzwaniami na etapie projektowania i działania mikroserwisów. W dalszych rozdziałach przyjrzymy się głównym trudnościom związanym z mikroserwisami, które muszą zostać wzięte pod uwagę podczas projektowania aplikacji w architekturze mikroserwisowej.

3.2.3.1 Uwierzytelnienie i autoryzacja

Uwierzytelnienie jest procesem weryfikacji tożsamości użytkownika w systemie informatycznym. Autoryzacja polega na sprawdzeniu uprawnień danego użytkownika do określonego zasobu lub akcji w systemie. Użytkownik musi być uwierzytelniony, aby wziąć udział w procesie autoryzacyjnym. [50] W systemach zbudowanych w architekturze rozproszonej bezpieczeństwo odgrywa jeszcze ważniejszą rolę, ponieważ im więcej usług składa się na jeden system tym łatwiej znaleźć lukę bezpieczeństwa w systemie. W monolitach powszechnym podejściem jest przeprowadzenie procesu uwierzytelniania i autoryzacji w tej samej aplikacji. W przypadku aplikacji mikroserwisowych proces musi zostać zdecentralizowany tak aby jedno potwierdzenie tożsamości zezwoliło na dostęp do wszystkich usług. Opisując ten problem skupimy się wyłącznie na metodach projektowania procesu uwierzytelnienia użytkownika w architekturze mikroserwisowej, pomijając szczegóły dotyczące różnych rodzajów implementacji tego procesu.

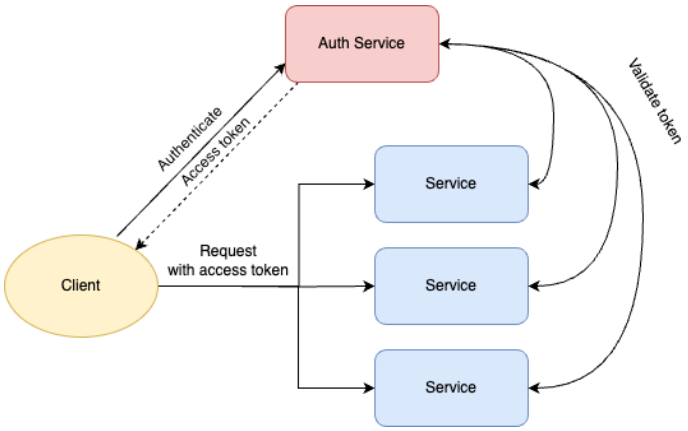
Jednym z popularnych podejść jest stosowanie globalnej autoryzacji na poziomie bramy API. Takie podejście sprawdza się w przypadku, gdy brama stanowi główny punkt wejściowy dla użytkowników aplikacji. Możemy wykorzystać bramę jako usługę, która w pełni weryfikuję użytkowników przeprowadzając proces uwierzytelniania oraz autoryzacji. Definiujemy zasady dla każdej usługi ustalając, które ścieżki URI wymagają uwierzytelnienia oraz jakie role potrzebne są do uzyskania dostępu do zasobu. Alternatywną opcją jest przekazanie odpowiedzialności autoryzacji na usługi końcowe. W tym przypadku brama API jedynie weryfikuje token uwierzytelniający oraz odczytuje zakodowane wartości identyfikujące użytkownika, takie jak unikalny identyfikator wykorzystywany w całym systemie oraz jego rolę. Dane te są następnie przekazywane w nagłówkach zapytania do usługi końcowej. Wykorzystanie pozycji bramy API jako serwisu przeprowadzającego uwierzytelnianie zapewnia nam niezależność serwisów od usługi uwierzytelniającej, gdyż weryfikacja odbywa się na poziomie bramy. Należy jednak pamiętać, że stosując tę technikę, naruszamy zasadę dogłębnej obrony (ang. *Defence in depth*), ponieważ usługi końcowe nie są w żaden sposób zabezpieczone. W związku z tym istotne jest, aby były one dostępne jedynie dla innych wewnętrznych serwisów w odrąbie jednego klastra i nie były wystawione do Internetu. [51]



Rys 22. Schemat przeprowadzania uwierzytelniania w systemie na poziomie bramy API. Źródło: C.H. Jack et al. [51]

Innym podejściem jest rozproszenie procesu uwierzytelnienia użytkowników na każdy serwis. W tym podejściu również występuję oddzielny serwis autoryzacyjny, który odpowiada za autoryzacje użytkowników w aplikacji. Do każdego zapytania do systemu dołączany jest token identyfikujący danego użytkownika. Każdy mikroserwis dekoduje kucz i weryfikuje jego sygnaturę indywidulnie lub wykorzystując serwis autoryzacyjny. Wykorzystując to podejście

nie musimy się martwić o nieautoryzowany dostęp do systemu, ponieważ każdy serwis z osobna posiada wprowadzony mechanizm zabezpieczający. W przypadku wykorzystania serwisu autoryzacyjnego do walidacji użytkowników, wprowadzamy powiązanie każdego serwisu z usługą autoryzacyjną, co sprawia, że aplikacja nie będzie mogła funkcjonować gdy serwis autoryzacyjny nie będzie uruchomiony. [51]



Rys 23. Schemat przeprowadzania uwierzytelniania w systemie na poziomie serwisów. Źródło: C.H. Jack et al. [51]

Proces uwierzytelniania i autoryzacji może być również zaimplementowany bezpośrednio w każdym serwisie, wykorzystując wspólną bazę danych przechowującą wrażliwe dane użytkowników. Takie podejście pozwala na autonomiczność usługi i zespołu w wyborze sposobu zabezpieczenia aplikacji. Jednakże serwisy stają się powiązane ze sobą na poziomie bazy danych, co narusza zasadę luźnego powiązania pomiędzy serwisami i sprawia, że zależą one od danych, które nie są ich własnością. Dodatkowo, jeżeli logika implementująca proces bezpieczeństwa jest powtarzalna, powoduje to duplikację kodu pomiędzy serwisami. [53]

Wszystkie powyższe przypadki przeprowadzają weryfikację użytkownika z każdym żądaniem klienta. Należy być świadomym, że awaria serwisu uwierzytelniającego lub wspólnej bazy danych może spowodować jeden punkt awarii (ang. *Single point of failure*), który pociągnie za sobą cały system. [52]

3.2.3.2 Zarządzanie transakcjami

Transakcje jest to zbiór operacji, które są traktowane jako pojedyncza jednostka pracy. Mają na celu zagwarantowanie, że dane systemowe pozostają spójne, nawet gdy wystąpią awarie systemu. Albo wszystko ulega zmianie albo nic. Aplikacje monolityczne, w zależności od swojej złożoności gwarantują wykonywanie operacji w ramach pojedynczej transakcji. W przypadku aplikacji opartych na architekturze rozproszonej, każda aplikacja utrzymuję swój własny stan. Oznacza to, że każda z operacji wykonywana jest osobno w granicach swoich

wewnętrznych transakcji. Jeżeli operacja wykonywana jest równocześnie w dwóch usługach i jedna z nich się nie powiedzie, nie można liczyć na wycofanie zmian z jednego serwisu w przypadku niepowodzenia operacji w drugim. W takim przypadku można zastosować alternatywną metodę osiągania końcowej spójności poprzez rezygnację z granic transakcji i przyjęcie założenia, że system osiągnie spójny stan w przyszłości stosując metodę ponawiania. Natomiast trzeba założyć, że ponawianie zawsze się powiedzie. Ta metoda jest szczególnie korzystna w kontekście długotrwałych operacji biznesowych.

Inną możliwością jest anulowanie całej operacji. Odrzucenie operacji może być wykonane przy wykorzystaniu transakcji kompensacyjnej, która jest nową transakcją stosowana do wycofania poprzedniej transakcji. Transakcja kompensacyjna może być wywołana ręcznie przez oddzielny ekran dostępny dla administratora systemu lub zautomatyzowana. Automatyzacja może być wykonana stosując wzorzec projektowy Sagi, który został szerzej opisany w rozdziale 3.2.2. Rozwiązuje on transakcje kompensacyjne w formie sekwencyjnej jak i orkiestracyjnej przy pomocy procesu koordynującego.

Odmiennym podejściem do obsługi transakcji w systemach rozproszonych jest stosowanie transakcji rozproszonych. Transakcja rozproszona gromadzi wieli wewnętrznych transakcji tworząc jedną blokującą transakcje. Cel transakcji rozproszonej nie odbiega od transakcji normalnej, w obu celem jest zapewnienie spójności w systemie. Do obsługi transakcji rozproszonych wykorzystywany jest algorytm dwufazowego zatwierdzania. Algorytm ten wykorzystuję menedżera transakcji, który wysyła dwa komunikaty do serwisów. Pierwszy z nich informuje o przygotowaniu do transakcji. Jeśli uczestnicy odpowiadają pozytywnie na to zapytanie, koordynator wysyła komunikat o zatwierdzeniu wszystkich lokalnych transakcji. W przypadku, gdy którykolwiek z uczestników odpowie negatywnie, koordynator niezwłocznie przesyła komunikat o wycofaniu lokalnych transakcji do wszystkich uczestników. W przypadku awarii menadżera transakcji lub gdy którykolwiek uczestnik przestanie odpowiadać podczas procesu głosowania, cała operacja zostaje zablokowana. Menadżer transakcji musi także zarządzać blokadami, co oznacza, że oczekujące transakcje mogą ustanawiać blokady na zasobach. Te blokady mogą prowadzić do konkurencji, co znacząco utrudnia skalowanie systemów, zwłaszcza w środowiskach rozproszonych.

Każde z zaproponowanych rozwiązań zwiększa poziom złożoności. Wprowadzenie transakcji rozproszonych komplikuje właściwe zarządzanie i skalowanie systemem. Systemy, które osiągają zbieżność poprzez kompensacyjną logikę ponawiania, mogą być trudniejsze do analizy i mogą wymagać zastosowania dodatkowej logiki kompensacyjnej do naprawy

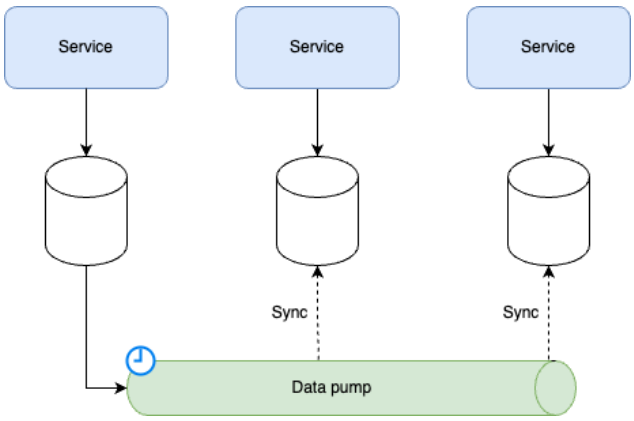
niespójności danych. Należy rozważyć, czy każda operacja biznesowa powinna być objęta transakcją rozproszoną Często operacja biznesowa, która w aplikacji monolitycznej jest realizowana jako pojedyncza transakcja, może być zamodelowana za pomocą lokalnych transakcji i końcowej spójności w architekturze mikroserwisowej. W przypadku, gdy wymóg spójności stanu uniemożliwia takie podejście, należy przede wszystkim skupić się na unikaniu podziału tej operacji na mniejsze. [50]

3.2.3.3 Synchronizacja danych

Krytycznym aspektem mikroserwisów jest interakcja oraz wymiana danych pomiędzy nimi. Decydując się na decentralizacje danych musimy brać pod uwagę potrzebę ich synchronizacji pomiędzy serwisami. Wymiana danych powinna być bezkonfliktowa oraz odbywać się w czasie rzeczywistym, aby nie zakłócać poprawnego działania systemu. Najprostszą metodą synchronizacji danych jest wykorzystanie protokołu HTTP, zarówno w podejściu synchronicznym, jak i asynchronicznym. Natomiast nie jest to rozwiązanie ani wydajne, ani wysoce skalowalne. Liczba przesyłanych zapytań do podrzędnych serwisów wzrasta liniowo, co w przypadku dużej liczby serwisów może wymagać znacznych zasobów. Dodatkowo, wpływa to na złożoność zarządzania systemem, ponieważ każdorazowe dodanie nowej usługi wymagającej synchronizacji wiąże się z koniecznością dodania kolejnego zapytania. Analogicznie, w przypadku usunięcia jednej z usług, konieczne będzie usunięcie odpowiedniego zapytania. Tworzą się również narzuty z koniecznością tworzenia dedykowanych interfejsów API, które są potrzebne wyłącznie na potrzeby synchronizacji. Ponadto, wykorzystanie protokołu HTTP powoduje ścisłe powiązanie usług, co jest sprzeczne z zasadą luźnego powiązania. [55]

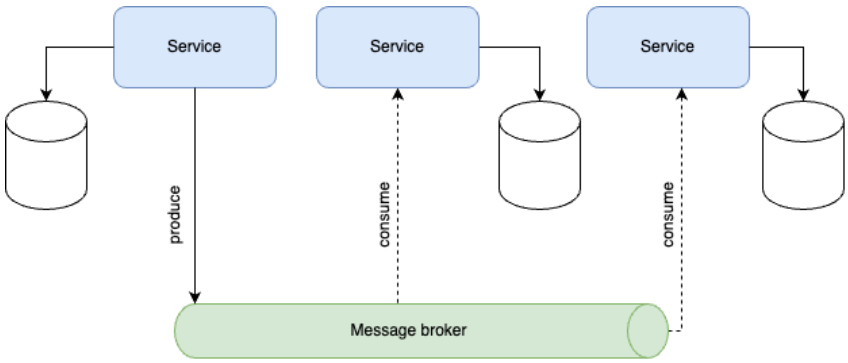
Odpowiedzialność za synchronizacje danych może zostać przeniesiona na poziom baz danych serwisów wykorzystując do tego pompę danych. Pompa danych jest odrębną usługą, której zadaniem jest cykliczna synchronizacja danych pomiędzy bazami danych poprzez mapowanie schematów zasobów. Pompa powinna posiadać wiedzę na temat schematów udostępnianych przez bazę źródłową oraz schematów baz docelowych Należy zauważyć, że zastosowanie pompy danych powoduje powiązania serwisów na poziomie danych. Jednakże, odpowiednio zaimplementowana pompa, która ułatwia synchronizację danych pomiędzy serwisami, minimalizuje wady wynikające ze sprzężeń. Ważnym aspektem jest to, że synchronizacja, jak wspomniano, jest cykliczna, co oznacza, że dane nie są synchronizowane pomiędzy serwisami automatycznie po każdej zmianie, a jedynie w określonych odstępach

czasowych. W związku z tym, omawiane rozwiązanie może być wyłącznie dla operacji biznesowych, które mogą zaakceptować pewne opóźnienia w spójności danych.



Rys 24. Wykorzystanie pompy danej w celu okresowej synchronizacji danych. Źródło: S. Newman [50]

Kolejna metoda wykorzystuje pompę danych sterowaną zdarzeniami. Zastosowano tutaj brokera wiadomości, który pełni rolę pośrednika komunikatów między serwisami w systemie. Podejście wykorzystuję wzorzec wydawcy-subskrybenta (ang. *Publish-subscribe*), w którym wiele serwisów może odbierać komunikaty od tego samego dostawcy. Po zaktualizowaniu zasobu w serwisie, serwis generuje zdarzenie zawierające zaktualizowany rekord i przesyła je na dedykowany temat (ang. *Topic*). Każdy serwis, który chce korzystać z tych danych, przyjmuje rolę konsumenta, nasłuchując określony temat, odczytuje i mapuje dane, dostosowując je do swojego kontekstu. Takie rozwiązanie pomaga uniknąć sprzężeń na poziomie serwisów oraz bazy danych. Dzięki temu podejściu otrzymujemy bardziej aktualne dane w porównaniu do poprzedniej metody. Jedyną wadą tego podejścia jest konieczność publikowania każdej aktualizacji zasobu w formie zdarzenia, co w przypadku większych ilości danych może nie skalować się tak dobrze, jak pompa danych na poziomie bazy danych. [55, 50]



Rys 25. Wykorzystanie pompy danych sterowanej zdarzeniami. Źródło: opracowanie własne

3.2.3.4 Komunikacja pomiędzy usługami

Komunikacja pomiędzy mikroserwisami jest kluczowym aspektem w aplikacjach mikroserwisowych, ponieważ serwisy muszą komunikować się ze sobą w celu realizacji konkretnych zadań w systemie. Komunikacja może odbywać się dwoma kanałami synchronicznie oraz asynchronicznie. Komunikacja synchroniczna polega na bezpośrednim odwoływaniu się do innych serwisów w blokującym połączeniu, natomiast komunikacja asynchroniczna pozwala na zrównoleglenie wykonywanych zadań wykonywanych w formie nieblokującej. W niektórych sytuacjach nie interesuję nas nawet czy operacja się powiodła. W tej sekcji omówione zostaną najpopularniejsze metody komunikacji synchronicznej oraz asynchronicznej wykorzystywane w architekturze mikroserwisowej oraz wady jakie za sobą niosą oba rozwiązania.

Komunikacja synchroniczna jest prostsza do zaprojektowania, ponieważ w każdym przypadku wiemy z jakim statusem oraz odpowiedzią zakończy się dana operacja. Do przesyłania synchronicznych pakietów pomiędzy usługami zwykle wykorzystywany jest protokół HTTP. Paradygmat REST jest jednym z najpopularniejszych stylów architektonicznych implementacji interfejsów API, który wykorzystuję protokół HTTP w wersji 1.1. Jego filarem jest 6 zasad, które zostały zdefiniowane przez Roya Rieldinga w swojej rozprawie doktorskiej pt. *Architectural Styles and Design of Network-based Software Architectures* [57]. Wykorzystanie HTTP daje dużą elastyczność w wyborze typu konsumowania zasobów, ponieważ jest on agnostyczny w zakresie stosowania typów komunikatów. Natomiast to format JSON ze względu na swoją prostotę i wydajność jest najczęściej wykorzystywanym typem danych w tym paradygmacie. Komunikacja z wykorzystaniem REST odbywa się poprzez zdefiniowane interfejsy API w każdej z usług. [58]

Zdalne wywołanie procedur (ang. *Remote Procedure Call - RPC*) jest protokołem umożliwiającym uruchomienie procedur na innym, zdalnym serwerze, które dla klienta inicjującego wywołanie wygląda jak wywołanie lokalne. [50] Jedną z implementacji protokołu RPC jest gRPC. Jest to otwarto-źródłowy projekt stworzony przez Google na potrzeby budowy wydajnych systemów w architekturze rozproszonej. gRPC wykorzystuje protokół HTTP/2 do komunikacji, który umożliwia przysyłanie i otrzymywanie wielu pakietów podczas jednego połączenia TCP oraz dostarcza bardziej efektywną binarną serializacje, co znacząco redukuję opóźnienia w komunikacji. W gRPC kontrakt pomiędzy klientem, a serwerem definiowany jest

za pomocą technologii Protobuf (ang. *Protocol Buffers*) która pozwala na generowanie interfejsów API klienta i serwera w niezależnych językach programowania. [58]

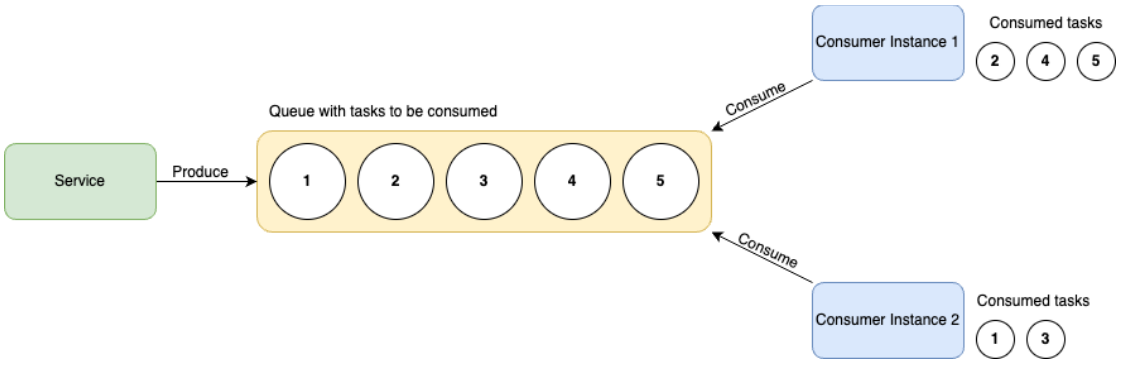
Komunikacja synchroniczna mimo swojej łatwości w projektowaniu ma też swoje ograniczenia. Przede wszystkim sprawia, że usługi stają się ze sobą powiązane, ponieważ każda z usług odwołuje się do bezpośrednio do innych przez co jest świadoma usługi, z którą współpracuje. Serwis, który do poprawnego funkcjonowania potrzebuje drugiej działającej usługi, traci swoją niezależność, ponieważ nie może działać w pełni samodzielnie bez wsparcia powiązanej usługi. Nadmierne stosowanie komunikatów synchronicznych może również skutkować tworzeniem głębokich łańcuchów zależności, co zwiększa podatność na błędy w ścieżce wywołań. Ponadto komunikacja blokująca może powodować problemy wydajnościowe aplikacji i wyzwalać awarie kaskadowe. [34]

W przypadku komunikacji asynchronicznej obiekt wywołujący operacje nie czeka na jej zakończenie. Znacząco wpływa to na wydajność systemu, która w przypadku żądań blokujących jest narażona na opóźnienia związane z zainicjowaniem połączenia i oczekiwaniem na odpowiedź. Komunikacja asynchroniczna ze względu na swój nieblokujący charakter jest szczególnie efektywna w przypadku długotrwałych operacji, pozwalając im wykonywać się w tle. [34]

Komunikacja asynchroniczna może być realizowana na dwa sposoby. Pierwsza metoda polega na zastosowaniu wzorca żądanie–odpowiedź, który jest wykorzystywany w komunikacji synchronicznej. Przykładowo załóżmy, że mamy dwie usługi A oraz B. Usługa A przesyła żądanie do usługi B, która potwierdza jedynie przyjęcie żądania, a dalsze operacje wykonuje w tle. Usługa A może być zainteresowana informacją na temat statusu wykonania tej operacji, natomiast nie uzyska jej bezpośrednio w tym samym połączeniu. Usługa B może poinformować usługę A o statusie operacji poprzez dedykowaną ścieżkę (ang. *Endpoint*) na którą usługa B prześle odpowiedź zgodnie z kontraktem ustalonym pomiędzy usługami. Na przedstawionym przykładzie można zauważyć, że mimo pozbycia się opóźnień związanych z blokowaniem żądań, usługi nadal są ze sobą ściśle powiązane. W rezultacie problem, który występował przy podejściu synchronicznym nadal występuje. [50]

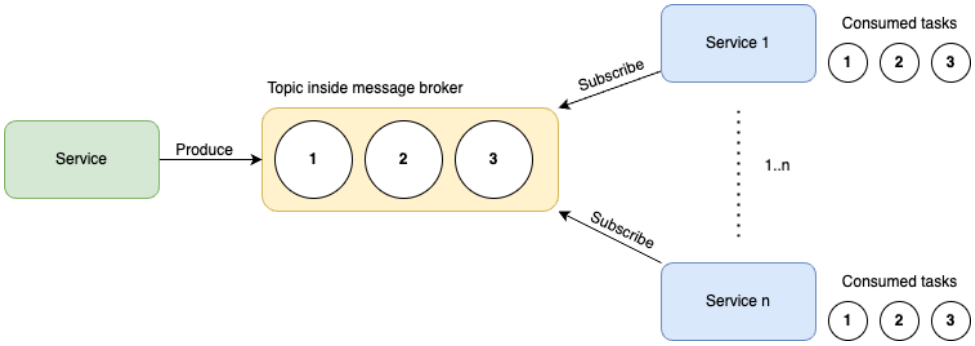
Drugim podejściem jest wykorzystanie zdarzeń do współpracy pomiędzy mikroserwisami. W tym podejściu serwis przesyła zdarzenia informujące o tym co się wydarzyło na kolejkę lub temat (ang. *Topic*) i na tym kończy się jego zadanie. Po wygenerowaniu zdarzenia proces kończy się lub przechodzi dalej. To czy zdarzenie zostanie skonsumowane i przez kogo nie jest w zakresie zainteresowania usługi. Mówiąc abstrakcyjnie,

w podejściu sterowanym zdarzeniami mówimy usługom co się wydarzyło i oczekujemy, że będą wiedziały jakie operacje powinny być wykonane dalej zamiast przekazywać im co mają zrobić tak jak w podejściu synchronicznym. Dzięki temu serwisy współpracujące ze sobą nie są świadome swojego istnienia, co gwarantuje mniejsze powiązania pomiędzy serwisami i elastyczność systemu. W architekturze opartej na zdarzeniach łatwiej jest wprowadzić nowe rozszerzenia w systemie, ponieważ usługi nie muszą wiedzieć o swoim istnieniu oraz o tym kto konsumuje ich zdarzenia. Komunikacja odbywa się zazwyczaj poprzez dodatkowy komponent nazwany brokerem komunikacji (ang. *Message broker*), który staję się centralnym punktem komunikacyjnym w aplikacji. Broker służy do odbierania żądań i delegowania ich do konsumentów. Zdarzenia mogą być konsumowane jedno lub wielokrotnie, w zależności od konfiguracji brokera. Na rysunku 26 przedstawiono schemat prezentujący przepływ komunikatów w systemie bazującym na kolejce zadań. W tym podejściu stosowana jest zasada „kto pierwszy ten lepszy". Jeden komunikat może zostać pobrany oraz przetworzony z kolejki tylko przez jednego konsumenta, niezależnie od liczby instancji. Ten wzorzec jest szczególnie przydatny, gdy zdarzenie może być wykorzystane wyłącznie w jednej operacji i kolejność ich wykonywania nie ma znaczenia. W tej sytuacji zwiększenie liczby konsumentów zapewnia wykonanie większej ilości zadań w krótszym czasie mając przy tym gwarancje, że żadne zdarzenie nie zostanie przetworzone więcej niż jeden raz.



Rys 26. Schemat prezentujący komunikacje poprzez kolejkę zadań. Źródło: M. Bruce et al. [34]

Na ilustracji 27 przedstawiono podejście publikuj-subskrybuj (ang. *Publish-subcribe*), które charakteryzuje się tym, że publikowane zdarzenie może być skonsumowane przez dowolną liczbę odbiorców subskrybujących dany kanał. Jest to typowy wzorzec komunikacji asynchronicznej w architekturze mikroserwisowej, ponieważ przesyłając zdarzenie, nie interesuje nas, ile usług je przetworzy.



Rys 27. Schemat prezentujący komunikacje poprzez wzorzec publikuj-subskrybuj. Źródło: M. Bruce et al. [34]

Pomimo wielu benefitów aspekt asynchroniczności dodaje złożoności do projektu i m.in utrudnia analizę operacji w systemie ze względu na to, że operacje nie są wykonywane jawnie oraz sekwencyjnie przez co trudniej jest śledzić żądania wykonane w trakcie jednego procesu. Dlatego ważnym punktem jest zadbanie o dobry monitoring w systemie. [34, 50]

3.2.3.5 Monitorowanie systemów mikroserwisowych

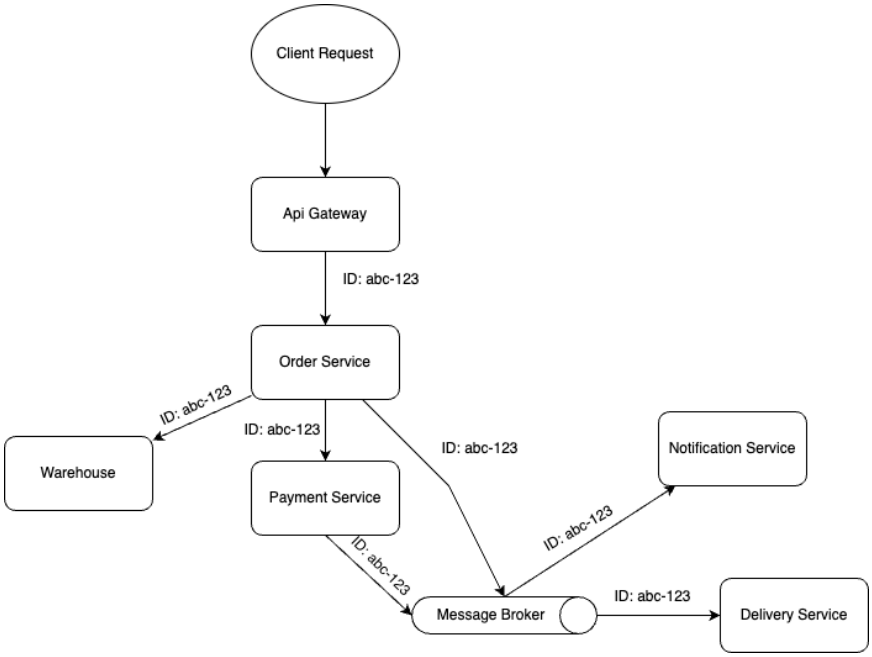
Systemy mikroserwisowe, poprzez swoją drobnoziarnistą, rozproszoną strukturę wprowadzają złożoność w obserwowalności i identyfikacji problemów w systemie. Monitoring systemów mikroserwisowych dodaje kolejną warstwę bezpieczeństwa do systemu, pozwalając na identyfikacje potencjalnych ataków oraz planowanie strategii ochrony przed zagrożeniami bezpieczeństwa. Systemy mikroserwisowe mogą posiadać wiele usług i każda z usług może posiadać wiele instancji, których liczba może zmieniać się w zależności od obciążenia. Monitorowanie każdego hosta oddzielnie jest problematyczne i trudne do osiągnięcia przy ciągle zmieniającej się liczbie replik, dlatego ważna jest centralizacja zagregowanych dzienników oraz metryk w jednym miejscu. Zgromadzone dane powinny zostać przesyłane w czasie rzeczywistym do odpowiedniego systemu AKP (ang. *Application Performance Management*) zdolnego do ich agregacji oraz analizy.[61]

Według [34] podczas zbierania danych z systemu powinniśmy się skupić na 4 złotych sygnałach, które są kluczowym wskaźnikiem wydajności i zdrowia dla systemu:

- **Opóźnienie** - mierzy czas trwania żądania w danej usłudze. Pozwala stwierdzić nastąpienie degradacji usługi, w przypadku, gdy opóźnienia są nieadekwatne do oczekiwanych,
- **Błędy** - sygnał określa liczbę żądań, które zakończyły się niepowodzeniem,
- **Ruch** - określa natężenie ruchu np. sieciowego w monitorowanym systemie,

- **Nasycenie** - określa stopień wykorzystania zasobów infrastrukturalnych wykorzystywanych przez usługi.

W architekturze mikroserwisowej śledzenie poszczególnych kroków wywołanych podczas jednego zapytania przez użytkownika może być szczególnie problematyczne w przypadku interakcji z innymi usługami, które generują podrzędne zapytania lub zdarzenia asynchroniczne. Koncept identyfikatora korelacji polega na dołączeniu unikalnego identyfikatora, do każdego zapytania. Identyfikator generowany jest podczas głównego zapytania, np. na poziomie bramy API i przekazywany do każdej podrzędnej akcji na przykład wykorzystując nagłówki HTTP lub te oferowane przed brokera komunikacyjnego. Mechanizm logowania powinien dołączać identyfikator korelacji do każdego wpisu generowanego w dzienniku zdarzeń. Po odpowiedniej agregacji logów jesteśmy w stanie prześledzić akcje, które wpłynęły na niepowodzenie danej operacji. Schemat takiego rozwiązania został przedstawiony na rysunku 28. [50]



Rys 28. Schemat prezentujący wykorzystanie identyfikatora korelacji w systemie mikroserwisowym. Źródło: S. Newman [50]

Monitorowanie syntetyczne, powszechnie znane również jako monitorowanie semantyczne polega na symulowaniu żądań użytkownika zwanych transakcjami syntetycznymi w celu monitorowania dostępności aplikacji oraz zachowań poszczególnych funkcjonalności w systemie. Testy syntetyczne są zautomatyzowanym rozwiązaniem i są przeprowadzane regularnie w zaplanowanych odstępach czasu. Testy nie zastępują potrzeby metryk niższego szczebla, ale uzupełniają je o dodatkowy mechanizm wykrywania problemów. [50] Dodatkowo

dobrą praktyką jest wystawienie API do kontroli zdrowia usługi (ang. *Healthcheck API*), które informuję o statusie zdrowia usługi oraz jej zależności. API powinno być odpytywane cyklicznie w celu weryfikacji działania aplikacji. [61]

3.2.4 Zalety i wady architektury mikroserwisowej

Architektura mikroserwisowa jest coraz popularniejszym rozwiązaniem wśród nowoczesnych systemów informatycznych. Jej struktura przynosi wiele korzyści, które mają istotne znaczenie w dążeniu organizacji do zwiększenia efektywności oraz elastyczności ich rozwiązań. W tej części pracy przeprowadzona zostanie analiza zalet oraz wad architektury mikroserwisowej, która pozwoli na przedstawienie pełniejszego obrazu omawianej architektury. W pierwszej kolejności omówione zostaną pozytywne aspekty mikroserwisów.

- **Samozarządzające się zespoły**

  Paradygmat architektury mikroserwisowej polega na tworzeniu produktu zamiast projektu, za który odpowiedzialne są małe, niezależne zespoły. Daje to zespołowi pełna autonomiczność nad usługami, umożliwiając modyfikacje i udoskonalanie niezależnie od innych zespołów. Małe zespoły specjalizujące się w konkretnej funkcjonalności biznesowej są w stanie szybciej oraz częściej dostarczać zmiany w oprogramowania niż w organizacji, w której odpowiedzialność za usługi jest rozrzucona pomiędzy różne, techniczne zespoły. Dzięki temu, że zespół jest w pełni odpowiedzialny za jeden lub kilka mikroserwisów przez cały cykl życia: implementacja, testowanie, utrzymanie oraz monitoring, wzrasta zaangażowanie zespołu oraz dbanie o końcową jakość, ponieważ zespół będzie musiał sam odpowiadać za wsparcie produktu na produkcji. Samozarządzające zespoły sprzyjają również innowacyjności i dają swobodę w doborze algorytmów oraz technologii dostosowanych do danego problemu. [35]

- **Łatwość zarządzania oraz wdrażania**

  Niewielki rozmiar oraz niezależność mikroserwisów sprawia, że usługi są łatwiejsze do zarządzania oraz naprawiania błędów. W przypadku aplikacji monolitycznych wdrożenie jednej zmiany wymaga opublikowania nowej wersji całej aplikacji, która zazwyczaj zawiera relatywnie dużo linii kodu. Takie operacje są obarczone wysokim ryzykiem, ze względu na to, że w przypadku dużych powiązań pomiędzy komponentami, jedna zmiana może wpłynąć negatywnie na funkcjonowanie systemu w innym miejscu. Z tego powodu wdrażanie nowych wersji w monolitach odbywa się rzadziej i zmiany wdrażane są w większych paczkach. W architekturze mikroserwisowej zmiana w jednym

mikroserwisie może zostać wdrożona niezależnie od innych usług, dzięki czemu unikamy kolizji pomiędzy komponentami. Pozwala to na częstsze wdrażanie nowych funkcjonalności, ale i szybszą reakcje na błędy powstałe w wyniku nowych zmian. Wycofanie jednej małej zmiany w niewielkiej aplikacji jest znacznie prostsze niż wycofanie wielu zmian w większej. [50]

- **Skalowalność**

  Dekompozycja aplikacji monolitycznej na mikrousługi pozwala na efektywniejsze skalowanie systemu w sposób poziomy. W przypadku aplikacji monolitycznej, gdy jedna z funkcjonalności była znacząco obciążona ruchem użytkowników, konieczne byłoby skalowanie całej aplikacji poprzez dodanie kolejnej instancji, która zazwyczaj jest duża i zużywa dużo zasobów generując przy tym większe koszty. W przypadku architektury mikroserwisów skalowanie poziome pozwala na dołożenie kolejnych instancji tylko dla konkretnych aplikacji. Dzięki temu możliwe jest, skalowanie wyłącznie aplikacji, które obsługują większy ruch i są przeciążone, aby poprawić wydajność tylko tej części systemu. Natomiast usługi mniej obciążone mogą być uruchomione na słabszych maszynach jako jedna instancja. Wiele usług chmurowych pozwala na automatyczne skalowanie usług w zależności od obciążenia. W przypadku przekroczenia wyznaczonych granic, uruchamiane są dodatkowe instancje usług, aby przejąć ruch. [50]

- **Heterogoniczność technologiczna**

  Ekosystem architektury mikroserwisowej nie wymaga stosowania jednej uniwersalnej technologii w każdej z aplikacji, natomiast pozwala na budowanie usług za pomocą różnych technologii dostosowanych do napotkanego problemu. Różne narzędzia odnajdują się lepiej lub gorzej w konkretnych zadaniach, w mikroserwisach jesteśmy w stanie dostosować narzędzie do problem. Brak uzależnienia od jednej technologii daje również możliwość tworzenia nowych serwisów w nowszych technologiach, bez konieczności przepisywania całego systemu. Dodatkowo testowanie oraz eksperymentowanie z nowymi technologiami wiąże się z mniejszym ryzykiem, jako że robimy to w obrębie konkretnej usługi, która jest autonomiczną częścią systemu. Szybciej jesteśmy w stanie wdrażać nowe technologie i lepiej zrozumieć, w jaki sposób mogą nam pomóc nowe usprawnienia. [50]

- **Odporność na błędy**

  Tak jak zostało wspomniane wcześniej w rozdziale 3.2.1, odpowiednio zaprojektowana architektura mikroserwisowa, w której awaria nie rozprzestrzenia w sposób kaskadowy na

inne usługi, jest w stanie izolować awarie w granicach swojego kontekstu, co pozwala reszcie systemu na kontynuowanie działania. Dzięki temu jesteśmy w stanie budować systemy, odpornych na awarie, które w przypadku ich wystąpienia wyłączają daną funkcjonalność do czasu naprawienia błędu.

Architektura mikroserwisowa choć posiada wiele zalet, jak każde podejście ma również swoje wady i ograniczenia. Ze względu na swoją rozproszoną strukturę, mikroserwisy mogą wprowadzać dodatkową złożoność do systemu i wymagać wypracowania pewnych kompromisów. Ponadto, konieczność zarządzania wieloma niezależnymi komponentami może wymagać zaawansowanej infrastruktury systemowej do utrzymania wysokiej jakości oprogramowania. W dalszej części rozdziału dokonamy szczegółowej analizy wad architektury mikroserwisowej, co pozwoli na ocenę ryzyka związanego z przejściem na to rozwiązanie.

- **Duplikacja danych**

  Ze względu na to, że architektura mikroserwisowa zaleca korzystanie z odizolowanej bazy danych dla każdego serwisu, wymagane jest wprowadzenie mechanizmu synchronizacji danych pomiędzy usługami, aby zapewnić płynność w systemie. Brak synchronizacji może prowadzić do niespójności w całym systemie mikroserwisowym. Duplikacja danych jest pewnego rodzaju kompromisem pomiędzy, wydajnością systemu oraz powiązaniami pomiędzy usługami, a zwiększeniem zasobów konsumowanych przez system. Zamiast za każdym razem pobierać dane z konkretnych serwisów decydujemy się na ich powielenie. Oczywiście większa ilość przetrzymywanych danych oznacza większe koszty operacyjne.

- **Zmiany funkcjonalności obejmujących wiele usług wymagają starannej koordynacji**

  Zmiany w funkcjonalnościach obejmujących wiele usług mogą być problematyczne i prowadzić do awarii kaskadowych. Dlatego kluczowa jest komunikacja pomiędzy zespołami odpowiedzialnymi za poszczególne serwisy. W przypadku modyfikacji współdzielonego kodu, takiego jak wewnętrzne biblioteki organizacji, czy też zmian w kontraktach komunikacyjnych pomiędzy usługami, zawsze należy stosować wersjonowanie oraz zachować kompatybilność wsteczną, tak aby inne usługi miały czas na adaptacje nowych zmian, co pomoże w uniknięciu zmian.

- **Systemy rozproszone są z natury skomplikowane**

  System składający się z wielu niezależnych komponentów, które współpracują ze sobą na różne sposoby, asynchronicznie lub synchronicznie jest o wiele bardziej złożony niż jedna

aplikacja monolityczna. Diagnozowanie błędów oraz debugowanie w przypadku systemów rozproszonych może wymagać większego nakładu pracy, jako że błąd nie zawsze jest jednoznaczny. Testowanie oraz wdrażanie systemu rozproszonego również nie należy do prostych czynności, jako że musimy brać pod uwagę wiele zintegrowanych aplikacji. [37] Dodatkowo system powinien być odpowiednio skonfigurowany, biorąc pod uwagę wszystkie trudności przedstawione w tym rozdziale. Rozwój pojedynczej aplikacji mikroserwisowej jest prostszy, jednak zarządzanie całym systemem opartym na mikroserwisach jest skomplikowaną czynnością.

- **Wyższe koszty sieciowe**

  W architekturze mikroserwisowej usługi komunikują się ze sobą przez sieć lub za pomocą zdarzeń. Integracje poprzez sieć generują dodatkowy ruch sieciowy, powodując opóźnienia komunikacyjne pomiędzy usługami i użytkownikiem końcowym. W systemach, w których liczba połączeń pomiędzy serwisami jest stosunkowo wysoka, sumaryczne opóźnienia mogą znacząco wpłynąć na wydajność systemu. Większość systemów rozproszonych wykorzystuje bramę API jako punkt wejściowy do aplikacji, która również może stanowić „wąskie gardło" zwiększając koszty sieciowe. W niektórych przypadkach należy rozważyć optymalizacje, poprzez przeniesienie komunikacji na wywołania asynchroniczne w celu redukcji ruchu sieciowego. [37]

- **Wyższe koszty wdrożenia i utrzymania**

  W idealnym systemie mikroserwisowym każdy mikroserwis ma swoje odizolowane środowisko z własną bazą danych oraz innymi zasobami które potrzebuje. Takie rozwiązanie kosztuję i jest o wiele droższe w utrzymaniu niżeli jedna aplikacja monolityczna. W eksperymencie przeprowadzonym w artykule *Performance Evaluation of Monolithic and Microservice Architecture for an E-commerce Startup* [59] wynika, że koszty infrastruktury chmurowej dla mikroserwisów były ponad dwa razy większe niż w przypadku architektury monolitycznej.

Podsumowując, architektura mikroserwisowa poprzez dekompozycje systemu na mniejsze podzespoły pozwala wprowadzić bardziej przejrzysty obraz procesów biznesowych w oprogramowaniu, rozdzielając przy tym odpowiedzialność za moduły na różne zespoły. Architektura mikroserwisowa oferuje również większą elastyczność rozwiązań oraz lepszą skalowalność w porównaniu do wcześniej omawianej architektury monolitycznej, natomiast

wprowadza dodatkową złożoność wynikającą z natury systemów rozproszonych, której trzeba być świadomym przy wyborze tego typu rozwiązania.

4 Opis realizacji projektu referencyjnego w architekturze monolitycznej

W niniejszym rozdziale przedstawiony zostanie projekt referencyjny w architekturze monolitycznej, który powstał w ramach pisania pracy inżynierskiej zatytułowanej *Projekt aplikacji wspomagającej znajdowanie zleceń dla programistów* [63]. Rozdział ten ma na celu przedstawienie kluczowych informacji dotyczących projektu, umożliwiających zrozumienie ogólnego sensu oraz aspektu technologicznego prezentowanej aplikacji. W rozdziale uwzględniono przegląd zapotrzebowań w postaci wymagań funkcjonalnych i niefunkcjonalnych, projekt bazy danych oraz streszczony opis implementacji, co pozwoli na zrozumienie działania aplikacji w architekturze monolitycznej. Opis implementacji koncentruje się wyłącznie na części serwera aplikacji, który jest przedmiotem badań niniejszej pracy. Szczegółowy opis procesu projektowania oraz implementacji aplikacji znajduje się w pracy inżynierskiej.

Celem aplikacji jest wsparcie programistów w znajdowaniu dodatkowych zleceń. Aplikacja została stworzona w odpowiedzi na brak istniejących rozwiązań, które spajałyby społeczność programistów, jednocześnie pomagając w znajdowaniu dodatkowych lub krótkookresowych zleceń.

4.1 Wymagania funkcjonalne i niefunkcjonalne

4.1.1 Wymagania niefunkcjonalne

Jak opisano w pracy inżynierskiej aplikacja musi spełniać następujące wymagania niefunkcjonalne: „

- Responsywność aplikacji.
- Włączona obsługa JavaScript w przeglądarce.
- Wsparcie dla przeglądarek:
  - Chrome ver min 107,
  - Safari ver min 15.4,
  - Firefox ver min 107.
- Poprawna obsługa do 1000 użytkowników na minutę.
- Wczytanie strony w czasie do 4 sekund.”

4.1.2 Wymagania funkcjonalne

W pracy inżynierskiej zostały również określone poniższe wymagania funkcjonalne dla aplikacji:

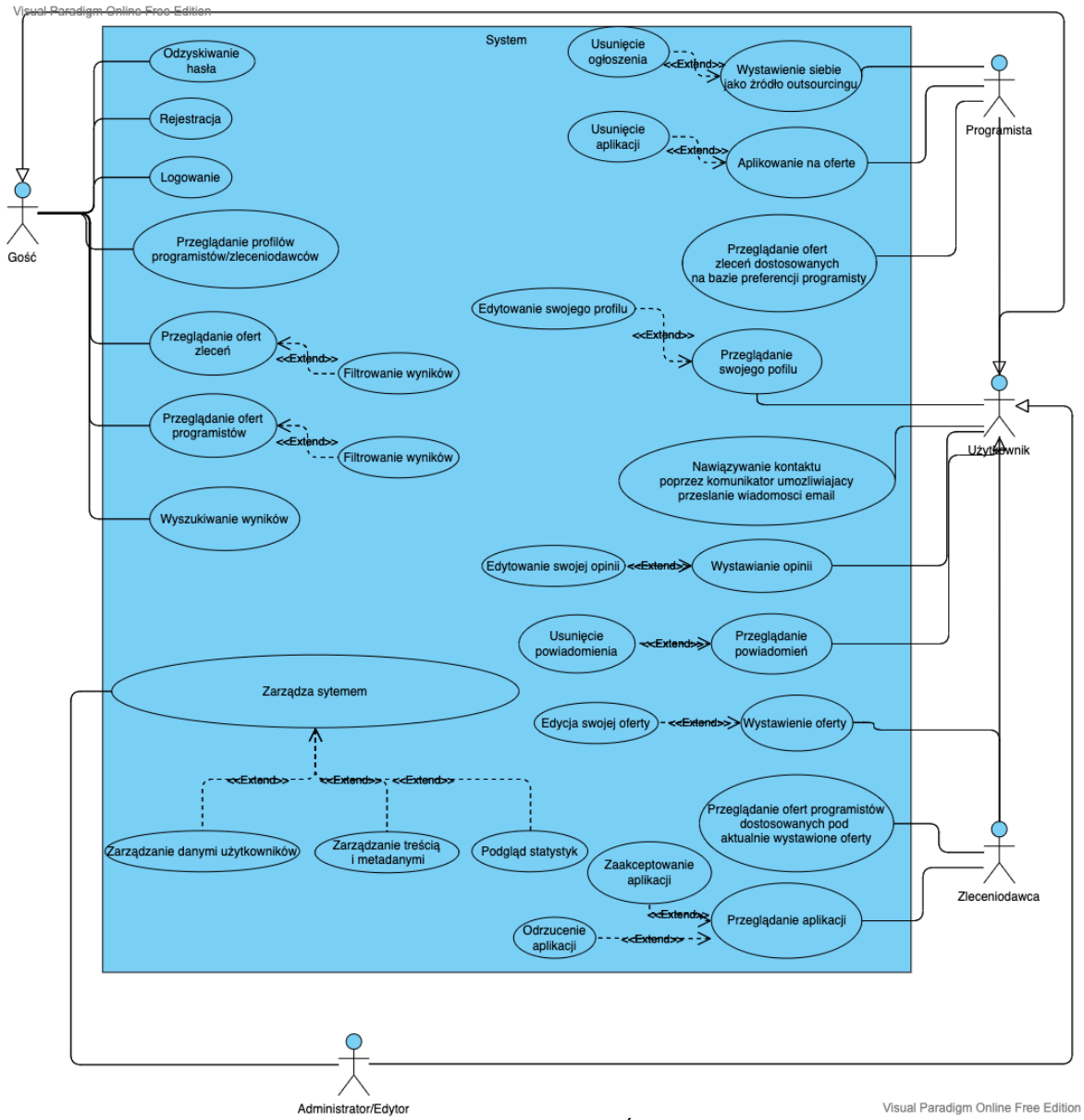
- Możliwość rejestracja konta:
  - Użytkownik może założyć konto jako zleceniodawca lub programista,
  - Na podany adres e-mail przesyłana jest wiadomość z linkiem do potwierdzenia rejestracji,
  - Po potwierdzeniu rejestracji użytkownik otrzymuje powiadomienie w aplikacji oraz wiadomość e-mail z podziękowaniem.
- Możliwość uwierzytelniania:
  - Użytkownik ma możliwość zalogowania się za pomocą zarejestrowanego konta i hasła,
  - Istnieje możliwość odzyskania hasła poprzez link wysyłany na podany adres e-mail,
  - Aplikacja umożliwia logowanie za pomocą konta Google.
- Możliwość przeglądanie podstawowych informacji o aplikacji:
  - Zakładka *O nas* zawiera podstawowe informacje na temat działania aplikacji,
  - Zakładka *Kontakt* umożliwia użytkownikom kontakt z administratorami aplikacji.
- Możliwość przeglądanie ofert:
  - Użytkownicy mogą przeglądać oferty zleceń oraz oferty programistów,
  - Aplikacja umożliwia filtrowanie ofert według technologii, rodzaju oferty, lokalizacji, typu zlecenia oraz doświadczenia,
  - Oferty mogą być sortowane według popularności oraz daty dodania.
- Możliwość przeglądania spersonizowanych ofert:
  - Po zalogowaniu, użytkownik ma możliwość wyświetlenia ofert dostosowanych do jego preferencji, uwzględniających lokalizację, technologie, doświadczenie oraz opinie.
- Możliwość wyszukiwania i przeglądania użytkowników:
  - Aplikacja posiada wewnętrzną wyszukiwarkę, umożliwiającą wyszukiwanie ofert zleceń oraz użytkowników,
  - Użytkownicy mogą przeglądać profile innych użytkowników oraz swój własny profil.

- Możliwość edycji swojego profilu:
  - Użytkownicy mogą edytować swoje dane osobiste, zdjęcie profilowe, opis, wykształcenie, miejsca pracy oraz znane technologie.
- Możliwość zarządzanie ofertami:
  - Użytkownicy mogą dodawać, aktualizować oraz usuwać oferty,
  - Oferty mogą automatycznie wygasać, a powiadomienia o ich wygaśnięciu są wysyłane do aplikantów oraz zleceniodawców.
- Możliwość aplikacji na oferty pracy:
  - Na podstawie profilu użytkownika generowane jest CV, które przesyłane jest jako aplikacja na ofertę,
  - Użytkownicy mają możliwość dodawania załączników oraz wiadomości do zleceniodawcy w formularzu aplikacyjnym,
  - Powiadomienia o nowych zgłoszeniach są przesyłane do zleceniodawcy.
- Możliwość zarządzania aplikacjami:
  - Użytkownicy mogą przeglądać swoje aktywne oraz wygasłe aplikacje, pobierać załączone pliki oraz usuwać aktywne aplikacje,
  - Zleceniodawcy mogą przeglądać aplikacje złożone na ich oferty, akceptować lub odrzucać aplikacje oraz pobierać załączone pliki.
- Możliwość kontaktu z użytkownikami:
  - Możliwe jest przesyłanie wiadomości kontaktowych na adres e-mail użytkownika poprzez formularz w aplikacji.
- Możliwość zarządzania oraz przeglądania opinii:
  - Użytkownicy mogą dodawać oraz przeglądać opinie, wybierając ocenę w skali od 1 do 5 oraz dodając komentarze.
- Użytkownicy mają możliwość przeglądania i usuwania powiadomień.
- Możliwość zarządzania aplikacją poprzez panel administracyjny:
  - Administratorzy mogą zarządzać treścią zakładki *O nas*, dodawać powiadomienia, przeglądać dane użytkowników oraz uzupełniać meta dane,

- Możliwe jest usuwanie oraz dodawanie nowych administratorów i edytorów serwisu.

4.1.3 Diagram przypadków użycia

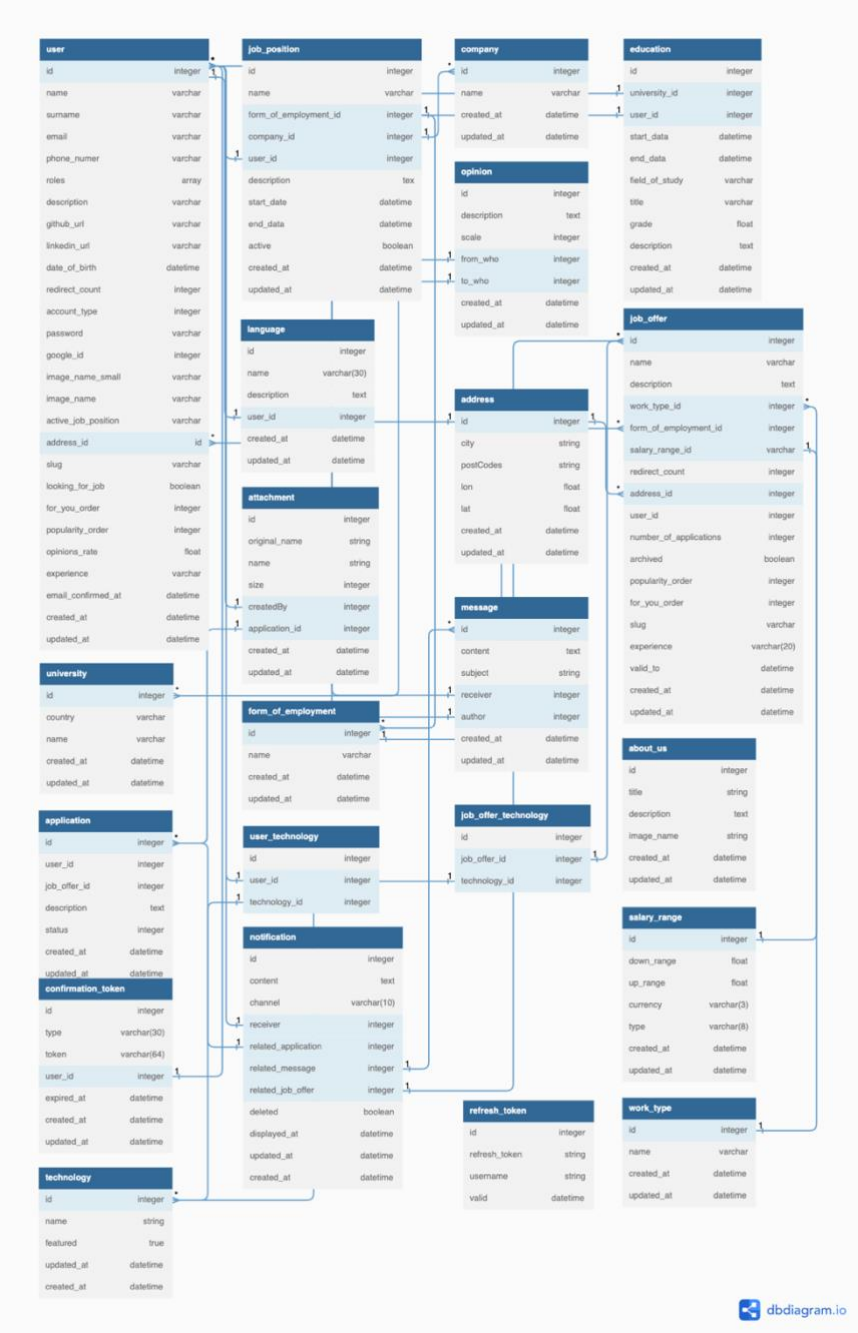
Diagram przypadków użycia (ang. *Use Case*) przedstawiony na rysunku 29 w sposób graficzny przedstawia wymagania funkcjonalne systemu. Diagram obrazuję przede wszystkim typy użytkowników, którzy mogą korzystać z aplikacji oraz scenariusze działań, które mogą być przez nich zostać realizowane. Każdy scenariusz użycia reprezentuję konkretną funkcję aplikacji, którą użytkownik może zainicjować, a linie łączące aktorów z przypadkami pokazują interakcje użytkownika o określonych rolach z systemem, wyznaczając jednocześnie granice tych interakcji.



Rys 29. Diagram przypadków użycia. Źródło: S. Berdzik [63]

4.2 Projekt bazy danych

W projekcie inżynierskim zdecydowano się na relacyjny model bazy danych ze względu naturalne związki występujące pomiędzy relacjami i lepszą wydajność operacji agregujących oraz złączających. Całkowite przejście na nierelacyjną bazę danych byłoby w tym przypadku nieoptymalne. Diagram związków encji (ang. *Entity Relationship Diagram*) został przedstawiony na rysunku 30. Cały model diagramu opiera się na dwóch głównych relacjach user oraz job_offer, dokładniejszy opis związków pomiędzy relacjami znajduję się z pracy inżynierskiej.



Rys 30. Diagram ERD bazy danych. Źródło: S. Berdzik [63]

4.3 Wykorzystane technologie

W niniejszym rozdziale przedstawione zostaną narzędzia technologiczne wykorzystane do implementacji części serwera aplikacji w architekturze monolitycznej.

- **PHP 8.1** [68]

  Skryptowy język programowania, uruchamiany po stronie serwera, stosowany do tworzenia aplikacji webowych.

- **Nginx** [69]

  Wysokowydajny serwer WWW oraz serwer pośredniczący dla HTTP, który zdecydowanie lepiej radzi sobie przy obsłudze dużego ruchu sieciowego od równie popularnego serwera Apache. [44]

- **Docker** [67]

  Platforma do tworzenia oraz uruchamiania aplikacji w kontenerach. Konteneryzacja umożliwia pakowanie aplikacji w oddzielnym systemie operacyjnym, który posiada tylko warstwy potrzebne do funkcjonowania aplikacji, dzięki temu jest lżejszy od konteneryzacji.

- **Symfony** [70]

  Środowisko zintegrowane (ang. *framework*) stworzone na podstawie wzorca projektowego MVC (ang. *Model-View-Controller*) dla języka PHP. Środowisko działa w cyklu zapytanie – odpowiedź.

- **API Platform** [71]

  Biblioteka oparta na szkielecie środowiska Symfony, która umożliwia szybkie tworzenie interfejsów API zgodnych REST wraz z dokumentacją.

- **Elasticsearch** [72]

  Rozproszony silnik wyszukiwania napisany w języku Java umożliwiający przeszukiwania dużych ilości danych w czasie rzeczywistym.

- **Redis** [73]

  Narzędzie służące do przechowywania danych w pamięci RAM w strukturze klucz – wartość. Najczęściej stosowany jako nierelacyjna baza danych lub pamięć podręczna aplikacji.

- **Mysql** [74]

  Otwarto źródłowy silnik relacyjnej baza danych.

- **Varnish** [75]

  Serwer pośredniczący pomiędzy klientem, a serwerem HTTP, służący do przechowywania pamięci podręcznej.

4.4 Opis implementacji

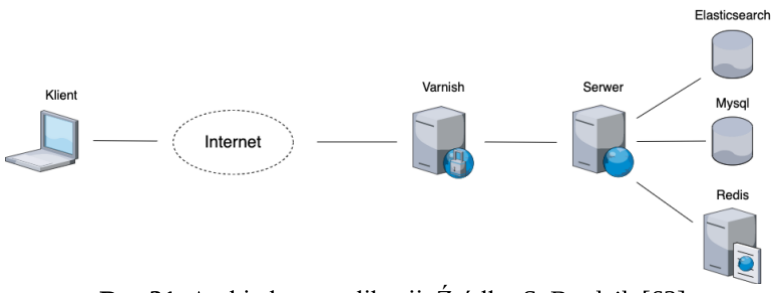
Aplikacja została zaprojektowana wykorzystując architekturę klient-serwer (ang. *Client-Server*). Architektura klient-serwer pozwala na rozdzielenie aplikacji interfejsu użytkownika od serwera. Obie aplikacje oddziaływają jako niezależne procesy, które komunikują się ze sobą. Aplikacja kliencka komunikuję się z serwerem na zasadzie przesyłania żądania poprzez interfejsy API zdefiniowane przez aplikacje serwerową. Poniższy opis skupia się na implementacji aplikacji serwerowej zaprojektowanej w architekturze monolitycznej.

Aplikacja serwerowa została zamodelowana w architekturze trójwarstwowej identyfikując warstwę bazy danych, biznesową oraz interfejsy API. Struktura katalogów w projekcie została przedstawiona na rysunku 32. Jak można zauważyć, aplikacja nie została podzielona na moduły i nie implementuje podejścia modularnego monolitu. Podejście jakie zostało przyjęte nie wyznacza granic funkcjonalności, natomiast dzieli poszczególne klasy w zależności od jej typu, przez co komponenty są ze sobą bardziej powiązanie, a implementacje interfejsów dostępne w zakresie całego projektu. Utrudnia to podział aplikacji na mikrousługi, co zostanie zaprezentowane w kolejnym rozdziale. Schemat architektoniczny aplikacji został przedstawiony na rysunku 31.

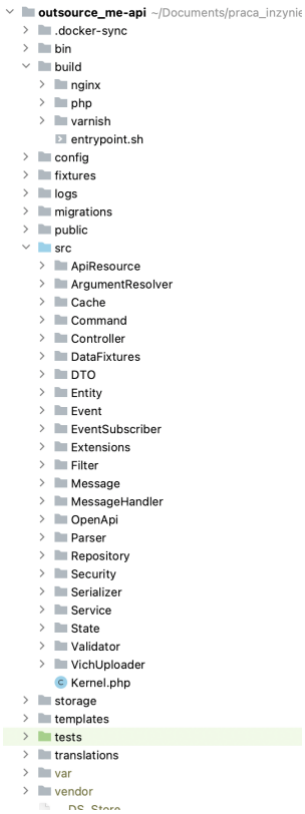
Serwer został napisany w języku PHP z wykorzystaniem środowiska zintegrowanego Symfony oraz uruchomiony na serwerze HTTP firmy Nginx. Aplikacja serwerowa udostępnia interfejsy API utylizując protokół REST. Biblioteka API Platform pozwala na generowanie operacji CRUD (ang. *Create, Read, Update Delete*) jak i bardziej skomplikowaną logikę biznesową wraz z dokumentacją OpenAPI [65] za pomocą atrybutów (adnotacji) dołączanych do klasy encji. Na poniższym listingu 1 przedstawiona została klasa encji `Education` z nałożonymi atrybutem `ApiResource`, który odpowiada za definicje operacji CRUD.

Listing 1. Kod źródłowy klasy Education w języku PHP

```
#[ApiResource(
   operations: [
      new Post(
         denormalizationContext: ['groups' => 'education:write'],
         security: "is_granted('ROLE_USER')",
         processor: EducationProcessor::class
      ),
      new Patch(
         denormalizationContext: ['groups' => 'education:edit'],
         security: "is_granted('ROLE_USER') and is_granted('USER_EDIT', object.getIndividual())"
      ),
      new GetCollection(
         uriTemplate: '/users/{userId}/educations',
         uriVariables: [
            'userId' => new Link(toProperty: 'individual', fromClass: User::class)
         ]
      ),
      new Delete(
         uriTemplate: '/users/{userId}/educations/{id}',
         uriVariables: [
            'userId' => new Link(toProperty: 'individual', fromClass: User::class),
            'id' => new Link(fromClass: Education::class),
         ],
         security: "is_granted('ROLE_USER') and is_granted('USER_EDIT', object.getIndividual())"
      )
   ],
   normalizationContext: ['groups' => 'education:read'],
)]
#[DateTimeRange]
#[ORM\Entity(repositoryClass: EducationRepository::class)]
class Education extends AbstractEntity
{ … }
```



Rys 31. Architektura aplikacji. Źródło: S. Berdzik [63]



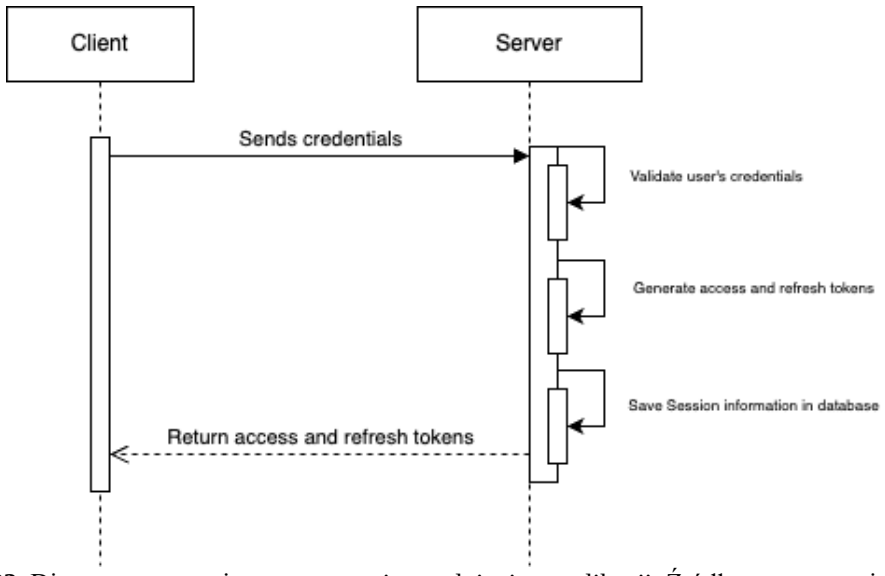
Rys 32. Struktura katalogów w aplikacji monolitycznej. Źródło: S. Berdzik [63]

Aby sprostać wymaganiom związanym z wyszukiwaniem danych w aplikacji, przeprowadzono integracje z silnikiem Elasticsearch. Został on wykorzystany w celu optymalizacji wyszukiwania pełno tekstowego z wykorzystaniem operacji LIKE na relacyjnej bazie danych. Dane wykorzystywane do przeszukiwania są grupowane w indeksy, gdzie każda encja w ERD odpowiada jednemu indeksowi, a każdy wiersz danych w tabeli odpowiada dokumentowi. Dane są duplikowane i z każdą aktualizacją synchronizowane z klastrem Elasticsearch'a, w celu zapewnienia spójności w systemie. Elasticsearch przechowuje wszystkie dane, które są wykorzystywane w zapytaniach wyszukujących, co eliminuje potrzebę dodatkowej integracji z bazą danych w celu pobrania większej ilości informacji o rekordach, optymalizując tym samym czas realizacji zapytań.

Na ilustracji 31, można zauważyć serwer pośredniczący Varnish umieszczony pomiędzy warstwą Internetu, a serwerem. Varnish w tym przypadku pełni rolę pamięci podręcznej, która przechowuje odpowiedzi API serwera, dzięki czemu kolejne takie same zapytania zostaną zwrócone z pamięci podręcznej zamiast z serwera. Efektywne korzystanie z Varnish wymaga precyzyjnej synchronizacji z serwerem, tak aby każda operacja aktualizująca zasoby automatycznie odświeżała pamięć podręczną dla powiązanych interfejsów API.

Dodatkowa usługą widoczną na schemacie architektonicznym jest Redis. Redis pełni funkcje pamięci podręcznej drugiej linii. Jest wykorzystywany m.in. do wsparcia algorytmu dopasowania ofert do preferencji użytkownik, który obciążał aplikacje ze względu na przeprowadzanie skomplikowanych operacji na dużej ilości danych w czasie rzeczywistym.

Proces uwierzytelniania i autoryzacji w aplikacji został zaimplementowany wykorzystując komponent bezpieczeństwa Symfony (ang. *Symfony Security*), który pozwala na zdefiniowania zasad bezpieczeństwa dla poszczególnych funkcjonalności za pomocą konfiguracji YAML [64]. Uwierzytelnianie użytkowników odbywa się poprzez wymianę tokenu dostępu JWT (ang. *JSON Web Token*) [100] pomiędzy klientem, a aplikacją serwerową, JWT jest standardem służącym do wymiany danych pomiędzy podmiotami. Wraz z tokenem dostępu (ang. *Access Token*) generowany jest token odświerzający (ang. *Refresh Token*), który umożliwia odświeżenie sesji użytkownika przez określony czas. Token odświeżający przechowywany jest w bazie danych w tabeli `refresh_token` w celu weryfikacji tożsamości użytkownika podczas próby wygenerowania nowego tokenu bezpieczeństwa. Schemat uwierzytelnienia w aplikacji monolitycznej został przedstawiony na rysunku 33.

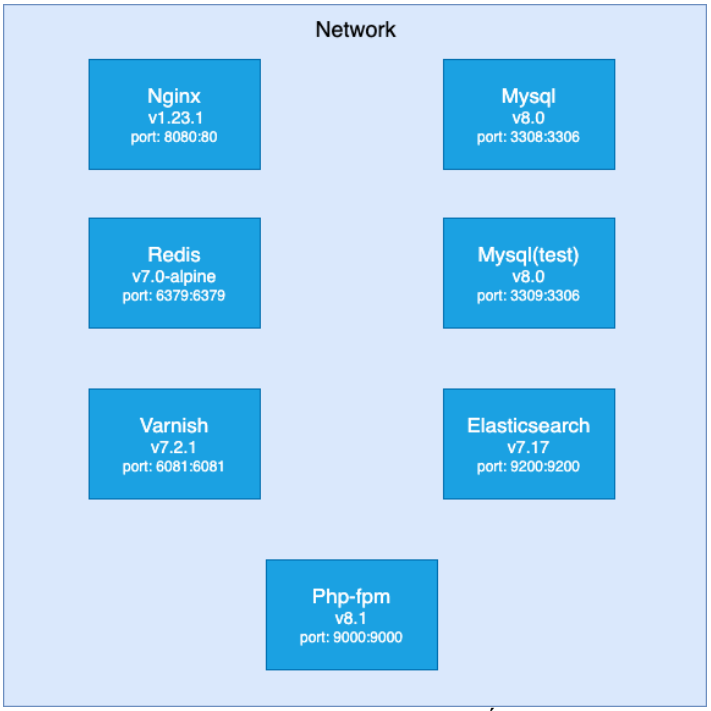


Rys 33. Diagram prezentujący proces uwierzytelnienia w aplikacji. Źródło: opracowanie własne

Język PHP jest językiem uniwersalnym, umożliwiającym programowanie zarówno w paradygmacie obiektowym, jak i proceduralnym. Jednakże nowoczesne biblioteki takie jak Symfony oraz API Platform preferują i często wymuszają stosowanie paradygmatu obiektowego. Podczas implementacji aplikacji wykorzystano paradygmat programowania obiektowego, stosując się do wszystkich popularnych praktyk programowania takich jak: SOLID, TDD (ang. *Test-Driven Development*), KISS (ang. *Keep It Simple Stupid*), DRY (ang. *Don't Repeat Yourself*).

4.5 Konteneryzacja

Docker jest platformą, która umożliwiającą wdrażanie aplikacji w kontenerach, które są lekkimi oraz przenośnymi jednostkami zawierającymi wszystko co aplikacja potrzebuje do uruchomienia. Aplikacja monolityczna wraz z wykorzystanymi zasobami zostały skonteneryzowane i uruchomione w wewnątrz jednej sieci za pomocą narzędzia Compose [66], który pozwala uruchamiać aplikacje składające się z wielu kontenerów. Konteneryzacja została zastosowana ze względu na proste wdrożenie infrastruktury na różnych środowiskach oraz agregacja logów każdego kontenera w jednym miejscu. Pliki konfiguracyjne do budowania obrazów znajdują w się katalogu projektów, a uruchomienie aplikacji na nowym środowisku obejmuje w zasadzie zainstalowanie narzędzia Docker oraz Compose i uruchomienie polecenia budującego infrastrukturę. Dodatkowym atutem jest izolacja, każdy kontener jest izolowany od zewnętrznego systemu operacyjnego serwera. Umożliwia to dostosowanie każdego kontenera pod oddzielne wymagania, bez ingerencji w globalne ustawienia systemu operacyjnego. Kontenery mogą być również uruchamiana bezpośrednio w infrastrukturze chmurowej, bez potrzeby ręcznej konfiguracji serwera. Wspólna sieć zapewnia, że kontenery widzą się nawzajem i mogą ze sobą współpracować. Schemat skonteneryzowanej aplikacji został przedstawiony na rysunku 34.



Rys 34. Schemat sieci kontenerów w aplikacji. Źródło: S. Berdzik [63]

5 Projektowanie i implementacja systemu w architekturze mikroserwisowej

W niniejszym rozdziale zostanie szczegółowo przedstawiony proces projektowania oraz implementacji wcześniej opisanej aplikacji w architekturze mikroserwisowej. Rozdział zawiera informacje dotyczące wykorzystanych technologii do implementacji mikroserwisów, a także szczegółowy opis dekompozycji aplikacji monolitycznej na niezależne usługi. Podczas dekompozycji szczególną uwagę poświęcono na identyfikacje granic kontekstu wraz ze strategią podziału bazy danych i komunikacją pomiędzy usługami.

Ponadto, w rozdziale zawarto projekt procesów uwierzytelniania i autoryzacji w systemie mikroserwisowym oraz opis implementacji dwóch wybranych usług, aby zaprezentować przeprowadzone prace oraz zastosowane rozwiązania techniczne i konfiguracje systemu w klastrze.

5.1 Zmiana technologiczna

Podczas implementacji aplikacji monolitycznej wykorzystany został język PHP. W przypadku systemu mikroserwisowego zdecydowano się na przejście na technologię JVM [76]. Zmiana ta jest podyktowana kilkoma czynnikami. Języki Java [77] oraz Kotlin [78] oferują lepszą wydajność oraz skalowalność, co jest szczególnie istotne w kontekście mikroserwisów. Ponadto, ekosystem narzędzi wspierających rozwój drobnoziarnistych usług jest znacznie bardziej rozwinięty i oferuję gotowe rozwiązania dla problemów związanych z konfiguracją, komunikacją pomiędzy serwisami oraz wiele innych. PHP został stworzony do budowania prostych stron internetowych, a nie wysoce skalowalnych systemów mikroserwisowych.

Java to obiektowy język programowania, który jest jednym z najdłużej używanych języków w branży. Java jest często wybierana do tworzenia złożonych systemów rozproszonych ze względu na swoją stabilność, skalowalność oraz dojrzały ekosystem. W projekcie została wykorzystana Java w wersji 21, w której m.in. zostały wprowadzone wirtualne wątki. [102]

Kotlin jest statycznie typowanym językiem, który pozwala na programowanie zarówno obiektowe jak i proceduralne. Porównując do Javy, oferuje bardziej zwięzłą oraz czytelną składnie. Wprowadza również mechanizmy, które pomagają zapobiegać typowym błędom w programowaniu w Javie, na przykład wyrażenia typów nullowanych i nienullowanych.

Kotlin może być uruchamiany w JVM i jest interoperacyjny z Javą, co oznacza, cały ekosystem bibliotek Java jest dostępny z poziomu Kotlin ‘a. W projekcie został wykorzystany Kotlin w wersji 1.9.22, kompilowany do Javy 21.

Aplikacje napisane w obu językach mogą być uruchamiane na różnych systemach operacyjnych bez konieczności modyfikacji w kodzie, ponieważ są izolowane w maszynie wirtualnej.

Mikroserwisy zostały zaimplementowane w środowisku zintegrowanym Spring Framework [79] wykorzystując rozszerzenie Spring Boot. Spring Framework to popularne środowisko zintegrowane w języku Java, służące do tworzenia aplikacji webowych i nie tylko. Zarządza cyklem życia i konfiguracją obiektów w aplikacji umożliwiając iniekcje zależności. Moduł Spring Cloud wprowadza zestaw narzędzi wspierających tworzenie mikroserwisów. Są to komponenty do obsługi takich aspektów jak: konfiguracja rozproszona, wykrywanie usług, równoważenie obciążenia, bramy API. Spring Boot jest dodatkiem do środowiska Spring, który na celu uproszczenie procesu tworzenia oraz konfigurowania aplikacji, dzięki czemu proces wdrażania nowych funkcjonalności staje się bardziej efektywny.

5.2 Dekompozycja aplikacji na mikroserwisy

W tej części rozdziału omówiony zostanie proces dekompozycji aplikacji monolitycznej na mikroserwisy. Dekonstrukcja monolitu na mniejsze, niezależne komponenty stanowi najważniejszy krok podczas migracji do architektury mikroserwisowej.

5.2.1 Identyfikacja granic kontekstu

Ze względu na to, że aplikacja monolityczna nie została zaimplementowana w sposób modularny, ani z wykorzystaniem wzorca DDD, wszystkie komponenty są ze sobą ściśle powiązane i wymagane jest ich rozdzielenie w celu wyznaczenia granic kontekstów. Pierwszym podjętym krokiem przyczyniającym się do dekompozycji aplikacji na mikroserwisy jest identyfikacja szwów. Szew jest fragmentem kodu, który może być utrzymywany osobno w kodzie, nie ingerując w działanie aplikacji w innych miejscach. Szwy pozwalają na utrzymanie kodu w czystości, ale również na identyfikacje granic danego kontekstu. Kontekst ograniczony z podejścia DDD opisanego w rozdziale 3.2.2 ze względu na luźno sprzężone granice i spójność w organizacji przedstawia idealny szew. Tak jak został opisane we wcześniejszej części pracy, spójność oraz luźne powiązania to cechy pożądane w mikroserwisach.

Identyfikacja szwów w kodzie jest możliwa wykorzystując przestrzenie nazw w języku PHP lub pakiety w Javie. Ze względu kompletną zmianę technologiczną zdecydowano się na wykonanie abstrakcyjnego podziału na konteksty ograniczone, jako że kod i tak musiałby być przepisany na inny język i nie mógłby być wykorzystany. Bazując na powiązaniach komponentów w kodzie oraz dokumentacji interfejsów API zidentyfikowano poniższe konteksty:

- Bezpieczeństwa,
- Profil użytkownika,
- Języki,
- Doświadczenie zawodowe,
- Edukacja,
- Technologie,
- Oferty zleceń,
- Aplikacje,
- Powiadomienia,
- Wiadomości,
- Opinie,
- Panel administracyjny.

W wyniku powyższego podziału zauważono, że podział kontekstów został przeprowadzony zbyt szczegółowo, ponieważ nadal występują ścisłe powiązania pomiędzy modułami. Konteksty ograniczone takie jak: Języki, Doświadczenie zawodowe, Edukacja oraz Technologia są powiązane z modułem profilu użytkownika i nie mają większego zastosowania osobno, dlatego znajdują się w kontekście ograniczonym Profil użytkownika. Kolejnym przypadkiem jest moduł Oferty zleceń, który jest głównym kontekstem dla Aplikacji oraz Technologii. Zauważono, również, że panel administracyjny jest częścią każdego z wyznaczonych kontekstów tworząc ścisłe powiązania pomiędzy modułami, dlatego został on włączony do każdego z nich.

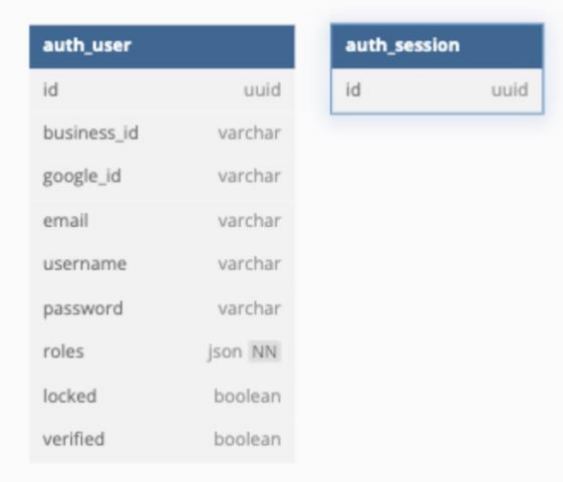
Powyższa analiza doprowadziła do wyznaczenia 6 kontekstów ograniczonych wchodzących w skład domeny aplikacji. Każdy kontekst ograniczony reprezentuje oddzielną usługę w architekturze mikroserwisowej:

- Uwierzytelnianie – obejmuję funkcjonalności takie jak rejestracja, logowanie oraz zarządzanie rolami użytkownika,
- Profil użytkownika – zarządza profilami użytkownika,
- Oferty zleceń – obejmuje funkcjonalności dotyczące ofert zleceń w tym aplikacje,
- Powiadomienia – odpowiada za powiadomienia wysyłane do użytkowników,
- Wiadomości – umożliwia komunikacje pomiędzy użytkownikami,
- Opinie – funkcjonalności opinii użytkowników.

5.2.2 Podział bazy danych

Kolejnym krokiem dekompozycji jest wydzielenie szwów na poziomie bazy danych. Schemat bazy danych przedstawiające relację pomiędzy tabelami został zaprezentowany na rysunku 30. Wydzielenie szwów na poziomie bazy danych jest trudniejsze, ze względu na to, że jedna tabela może być wykorzystywana przez wiele kontekstów ograniczonych.

Rozpoczynając od usługi odpowiedzialnej za autoryzacje. Dotychczas dane uwierzytelniające były przechowywane w tabeli users wraz z danymi dotyczycącymi profilu użytkownika. Oznacza to, że z tabeli korzystają dwa konteksty: uwierzytelnianie oraz profil użytkownika, które muszą zostać rozdzielone, aby nie tworzyć integracji na poziomie bazy danych. W tym celu dla kontekstu uwierzytelniania wydzielono tabele auth_users przechowującą dane dotyczące uwierzytelniania, a do tabeli users dodana została kolumna external_id, która identyfikuję użytkownika z tabeli auth_users. Relacja jest oczywiście abstrakcyjna, ponieważ obie tabele znajdują się w innych bazach danych. Więcej na temat wykorzystania tej relacji zostanie opisane w rozdziale 5.2.6. Do bazy danych usługi uwierzytelniającej przeniesiona została również tabela auth_sessions, która jest zmodyfikowaną wersją tabeli refresh_token z wersji monolitycznej. Schemat bazy danych usługi uwierzytelniającej po transformacji tabeli users przedstawiony został na ilustracji 35. Ze względu na brak występujących relacji pomiędzy tabelami zdecydowano się na wykorzystanie nierelacyjnej bazy danych.



Rys 35. Schemat bazy danych usługi uwierzytelniania. Źródło: opracowanie własne

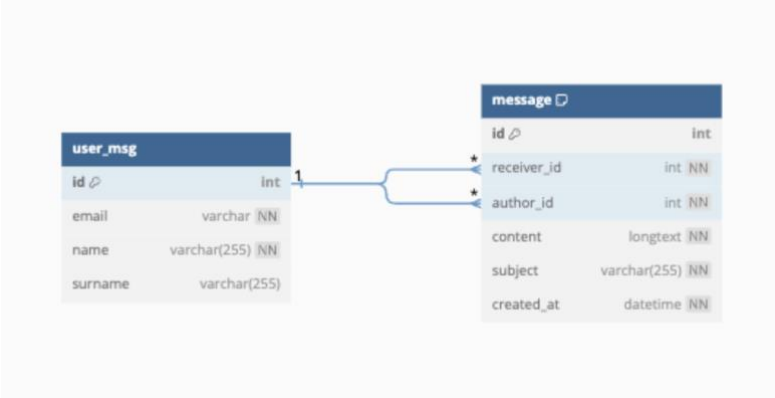
Kontekst powiadomień wykorzystuję tabele notifications. Na schemacie ERD bazy danych można zauważyć relacje występujące pomiędzy tabelami: users, messages, job_offers, applications tworząc powiązania z trzema innymi kontekstami. W związku z tym tabela notifications została zgeneralizowana, aby serwis był bardziej wszechstronny oraz zachował autonomiczność. Z tabeli usunięte zostały powiązane kolumny zostawiając kolumny recipient oraz recipient_id, który przechowują informacje o odbiorcy. Kolumna recipient jest wymagana i reprezentuje kontakt adresata w zależności od kanału komunikacji (adres email, numer telefonu, identyfikator użytkownika). Kolumna recipient_id nie jest wymagana i reprezentuje identyfikator użytkownika w systemie wywołującym. Baza danych usługi powiadomień została również przeniesiona do nierelacyjnej bazy danych, a schemat przedstawiono na rysunku 36.



Rys 36. Schemat bazy danych usługi powiadomień. Źródło: opracowanie własne

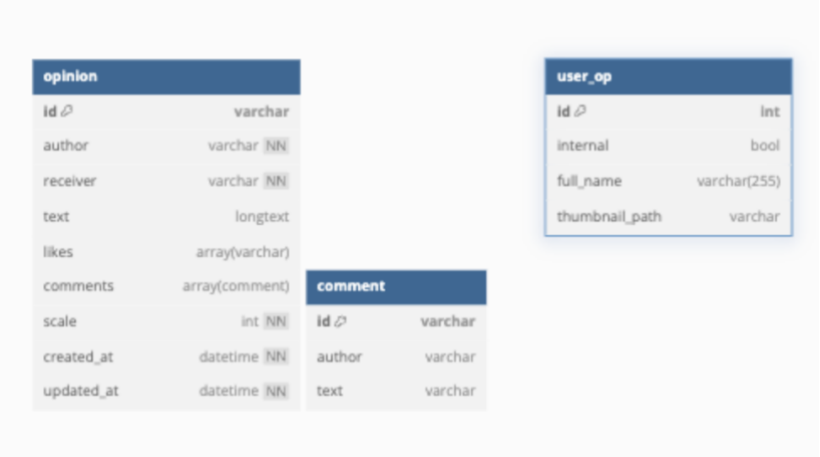
Kolejnym kontekstem jest kontekst wiadomości. Główną encją domenową dla tego kontekstu jest tabela message, która jest w relacji z tabelą user. Jak wspomniano wcześniej tabela user należy do innego kontekstu ograniczonego. Relacja pomiędzy tabelami musi zostać

zachowana z perspektywy biznesowej, dlatego zdecydowano się na duplikację potrzebnych danych z tabeli user z profilu użytkownika w tabeli user z wiadomości. Inną opcją byłoby usunięcie relacji i pozostawienie identyfikatorów autora oraz odbiorcy, a następnie pobieranie danych dotyczących użytkowników za pomocą interfejsu API usługi profilu użytkownika. Jednakże, rozwiązanie to wiązałoby się z głębokim powiązaniem usług oraz osłabieniem wydajności aplikacji. W przypadku tej bazy danych wykorzystany został silnik relacyjnej bazy danych. Schemat został przedstawiony na rysunku 37.



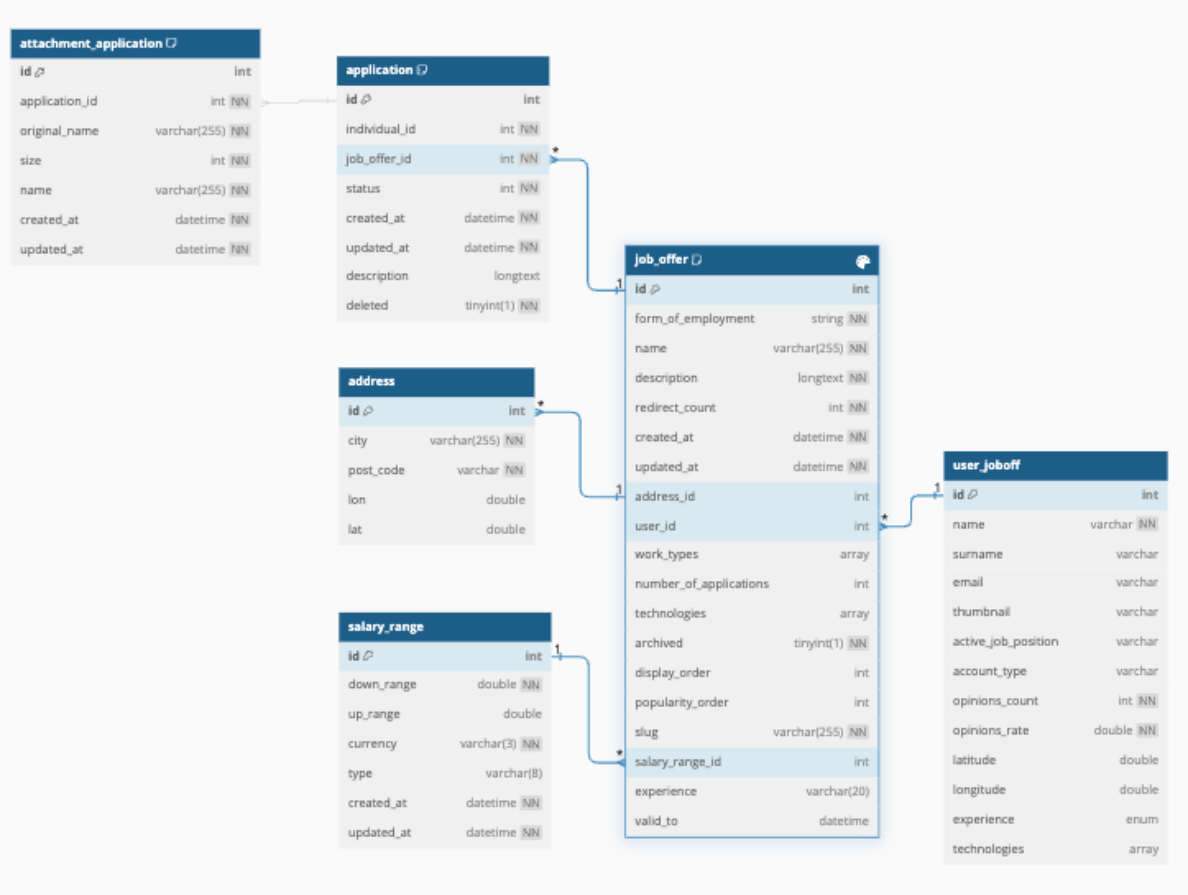
Rys 37. Schemat bazy danych usługi wiadomości. Źródło: opracowanie własne

Kolejnym kontekstem w procesie podziału bazy danych jest moduł opinii. W warstwie bazy danych kontekst ten korzysta z tabel opinion oraz user. Jest to przypadek podobny do poprzedniego, jednak różnią się dane z tabeli user, które są wykorzystywane przez kontekst opinii. W tym przypadku również zastosowano podejście z duplikacją potrzebnych danych z tabeli user, natomiast relacja pomiędzy tabelami została zachowana jedynie w sposób abstrakcyjny. Wynika to z przejścia na model nierelacyjnej bazy danych, w którym relacje nie istnieją. Decyzja o zastosowaniu modelu nierelacyjnego zapadła ze względu na dodanie nowych funkcjonalności pozwalających na dodawanie komentarzy oraz polubieni. W modelu nierelacyjnym dane te mogą zostać zagnieżdżone wewnątrz dokumentu nierelacyjnej kolekcji. Schemat dokumentów w nierelacyjnej bazie danych został przedstawiony na rysunku 38.



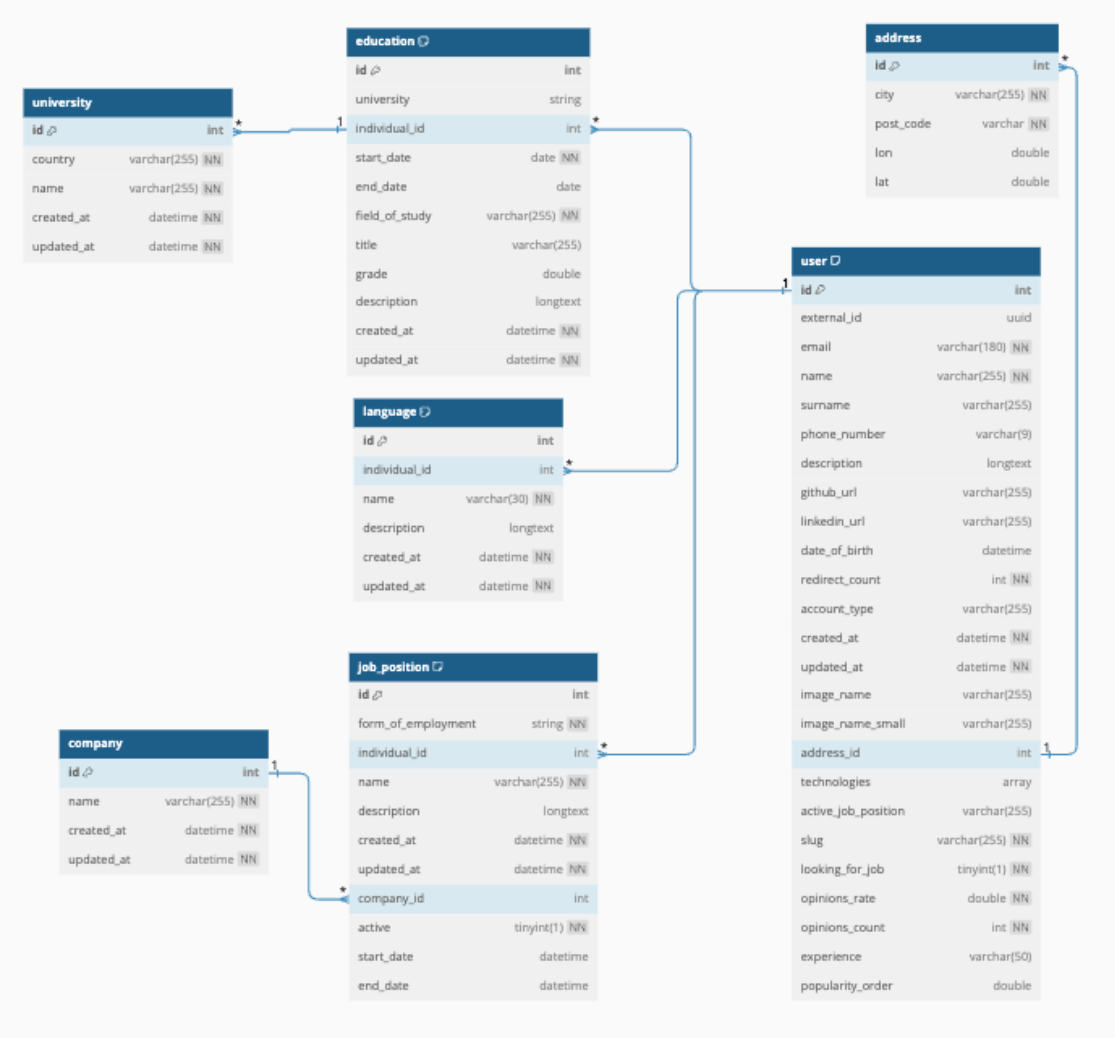
Rys 38. Schemat bazy danych usługi opinii. Źródło: opracowanie własne

Jak wspomniano we wcześniejszym rozdziale w skład kontekstu ograniczonego ofert zleceń wchodzi kontekst aplikacji. Z diagramu ERD zaprezentowanego na rysunku 30 można zaobserwować powiązanie z tabeli job_offer z tabelą user relacją wiele do jednego. W przypadku tego kontekstu użytkownik również odgrywa ważną rolę biznesową. Aby uniknąć powiązania kontekstów na poziomie bazy danych oraz uzależnienia usługi ofert pracy od dostępności usługi profilu użytkownika zdecydowano się na duplikacje danych z tabeli user potrzebnych do samodzielnego funkcjonowania usługi. Pozostałe tabele pozostaje bez zmian, a baza danych pozostaje w formie relacyjnej. Schemat bazy danych dla usługi ofert zleceń został przedstawiony na rysunku 39. Można zauważyć zniknięcie tabel form_of_employment, work_type, technology oraz zmianę formatu tabeli address. Dwie pierwsze zostały zastąpione statycznymi wartościami zapisanymi na poziomie kodu źródłowego. Tabela technology została zdenormalizowana do postaci tablicy przechowywanej w kolumnie technologies, natomiast tabela address przechowuję jeden kod pocztowy zamiast wszystkich dla jednego miasta. Więcej szczegółów o statycznych oraz współdzielonych danych zostanie opisane w kolejnym podrozdziale dotyczącym dekompozycji.



Rys 39. Schemat bazy danych usługi ofert zleceń. Źródło: opracowanie własne

Po wyznaczeniu szwów dla kontekstów, które po podziale wskazywały powiązania z modułem profilu użytkownika łatwiej jest wyznaczyć szwy dla samego kontekstu profilu użytkownika. Funkcjonalność profilu użytkownika opiera się na głównej tabeli user, która gromadzi szczegółowe dane użytkowników dotyczące ich edukacji, doświadczenia zawodowego, znajomości języków. Na poniższym schemacie przedstawiono kontekst profilu użytkownika, uwzględniający zmiany wynikające z poprzednich transformacji. Ponownie można zauważyć brak tabeli form_of_employment i technology oraz powielenie tabeli address, która została zaprezentowana również w schemacie dla kontekstu ofert zleceń.



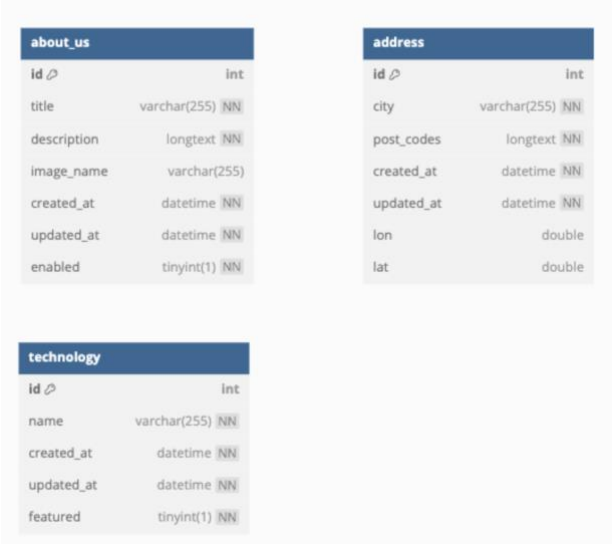
Rys 40. Schemat bazy danych usługi profilu użytkownika. Źródło: opracowanie własne

5.2.3 Zarządzanie statycznymi i współdzielonymi danymi

Porównując schematy baz danych przed i po podziale, można zauważyć, że niektóre tabele takie jak: work_type, form_of_employment, technology, about_us oraz address nie zostały wymienione w żadnym z schematów lub zmieniły swój format. Wszystkie dane zawarte w tych tabelach reprezentują dane statyczne, które po wyznaczeniu kontekstów były współdzielone pomiędzy różnymi kontekstami, ale nie należały do żadnego z nich bezpośrednio. Ponadto wszystkie dane są eksponowane na zewnątrz poprzez interfejs API. W związku z tym zdecydowano o stworzeniu oddzielnej usługi odpowiedzialnej za przechowywanie danych statycznych wykorzystywanych w aplikacji.

Tabela work_type oraz form_of_employment zostały usunięte, a wartości statyczne są teraz przechowywane w typie enumerycznym. Dane, te nie będą zmieniane znacznie częściej niż publikowana będzie nowa wersja aplikacji, dlatego zdecydowano się na takie rozwiązanie.

Słowniki są zduplikowane pomiędzy usługami korzystającymi z tych danych, a usługa statyczna jako jedyna udostępnia słownik poprzez interfejsy API. Tabele technology, about_us oraz address zachowały swój pierwotny kształt, jednak ze względu na brak relacji zostały przeniesione do nierelacyjnej bazy danych. W przypadku tabeli about_us dane są wypełniane poprzez panel administracyjny. Dane do tabeli technology ładowane są podczas startu aplikacji z pliku konfiguracyjnego, a do tabeli address z zewnętrznego API. Schemat bazy danych dla usługi danych statycznych został przedstawiony na rysunku 41.



Rys 41. Schemat bazy danych usługi danych statycznych. Źródło: opracowanie własne

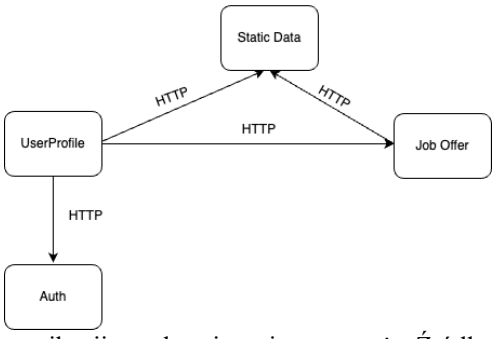
Kolumna technologies w tabelach job_offer i user oraz tabela address w usługach ofert zleceń oraz profilu użytkownika nie są powiązane z tabelami technology oraz address w omawianej usługi. Taki sposób pozwala na ekspozycje danych statycznych poprzez dedykowaną usługę i zarządzanie danymi wewnątrz oddzielnych kontekstów. Choć dane udostępniane przez usługę statyczną będą źródłem informacji dla usług ofert zleceń oraz profilu użytkownika, żadna z tych usług nie jest tego świadoma. Użytkownik, korzystając z API, ma możliwość wprowadzenia dowolnych danych.

5.2.4 Projektowanie komunikacji pomiędzy serwisami

W powyższych trzech podrozdziałach przedstawiony został proces wyznaczenia kontekstów ograniczonych w aplikacji monolitycznej oraz proces podziału bazy danych. W tym podrozdziale skupimy się na przedstawieniu sposobu komunikacji pomiędzy mikroserwisami w systemie.

Komunikacja w systemie odbywa się w sposób synchroniczny oraz asynchroniczny. Komunikaty synchroniczne przesyłane są za pomocą protokołu HTTP poprzez zdefiniowane interfejsy API wykorzystując REST. Starano się jak najbardziej wykluczyć komunikacje

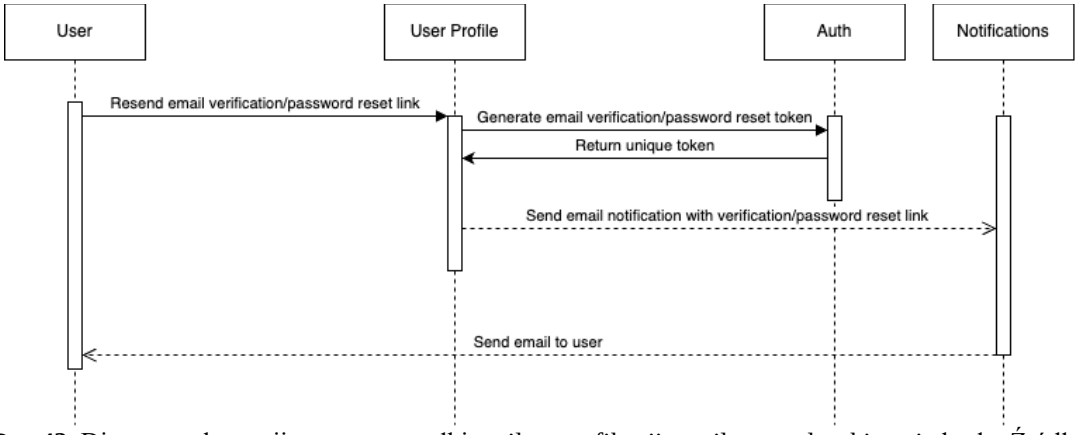
synchroniczną pomiędzy usługami, aby zachować jak najluźniejsze powiązania pomiędzy usługami. Na poniższej 41 ilustracji przedstawiono diagram przepływu komunikacji synchronicznej w aplikacji.



Rys 41. Przepływ komunikacji synchronicznej w systemie. Źródło: opracowanie własne

Usługa profilu użytkownika odpowiedzialna jest za proces rejestracji nowych użytkowników. W tym celu usługa wywołuje usługę uwierzytelniającą przekazując jej dane wymagane przez kontrakt.

Usługa uwierzytelniająca jest również wywoływana w funkcjonalności odzyskiwania hasła oraz weryfikacji adresu email, w celu wygenerowania unikalnego identyfikatora weryfikującego. W tych procesach usługa wykorzystuję identyfikator użytkownika w usłudze uwierzytelniającej, który jest zapisany w usłudze profilu użytkownika po pomyślnym procesie rekrutacji. Ze względu na podobieństwo procesów oba procesy zostały przedstawione na jednym diagramie sekwencji na rysunku 42. Strzałki oznaczone przerywana linią sygnalizują akcje asynchroniczną.



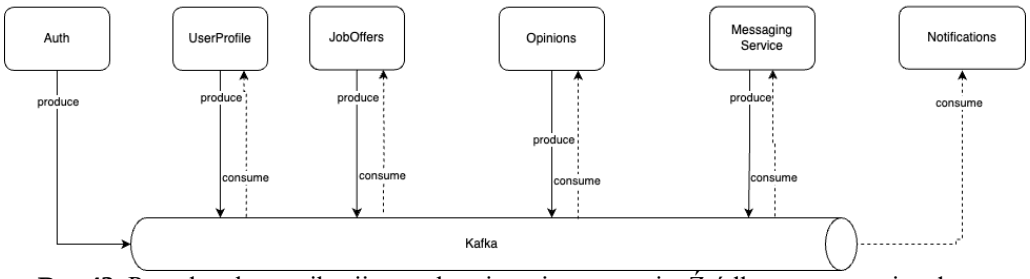
Rys 42. Diagram sekwencji procesu wysyłki maila weryfikacji email oraz odzyskiwania hasła. Źródło: opracowanie własne

Komunikacja usługi profilu użytkownika z usługą ofert zleceń odbywa się jednorazowo podczas dostosowania ofert programistów pod preferencje zleceniodawcy w celu pobrania

danych o ofertach zleceń wystawionych przez zleceniodawcę. Proces dostosowania ofert jest i tak czasochłonny, a duplikowanie wszystkich ofert zleceń nie miałoby sensu i mogłoby wpłynąć na wydajność aplikacji.

Obie usługi profilu użytkownika i ofert zleceń komunikują się z usługą danych statycznych w celu pobrania informacji o technologiach. Informacje te wykorzystywane są do algorytmu obliczania wskaźników popularności ofert zleceń oraz profilów programistów. Wskaźniki obliczane są raz dziennie, dlatego powiązanie nie ma znacznego wpływu na działanie aplikacji. Ponadto usługi akceptują ryzyko niepowodzenie się tej akcji. Ze względu na niską częstotliwość zmian technologii w systemie w przyszłości dane mógłby zostać zduplikowane pomiędzy usługami w celu uniknięcia powiązań pomiędzy usługami.

Komunikacja asynchroniczna została zaprojektowana wykorzystując zdarzenia asynchroniczne, które są dystrybuowane przez broker wiadomości. Jako broker wiadomości została wykorzystana technologia Apache Kafka [80]. Kafka jest rozproszonym systemem strumieniowania zdarzeń (ang. *event streaming*) pozwalająca na obsługę strumieni danych w czasie rzeczywistym. W systemie usługi przesyłają zdarzenia do brokera wiadomości, które następnie są konsumowane przez poszczególne usługi. W tabeli 1 przedstawiono wszystkie zdarzenia, które są obsługiwane w systemie. Na rysunku 43 przedstawiono przepływ komunikacji asynchronicznej w systemie.



Rys 43. Przepływ komunikacji asynchronicznej w systemie. Źródło: opracowanie własne

Tabela 1.

Nazwa zdarzenia	**Usługa publikująca**	**Usługa subskrybująca**	**Opis zdarzenia**
RegisterEvent	Usługa uwierzytelniająca	Usługa profilu użytkownika	Zdarzenie generowane w celu poinformowania o powodzeniu rejestracji
RegistrationFailed Event	Usługa profilu użytkownika	Usługa uwierzytelniająca	Zdarzenie generowane, gdy rejestracja po stronie usługi użytkownika nie powiedzie się w celu cofnięcia transakcji.
NotificationEvent	• Usługa profilu użytkownika • Usługa ofert zleceń • Usługa opinii • Usługa wiadomości	Usługa powiadomień	Zdarzenie generowane w celu przesłania powiadomienia w systemie
UserOpinionsDetails Event	Usługa opinii	Usługa profile użytkownika	Zdarzenie generowane w celu przesłania statystyk opinii po każdej aktualizacji danych
UserDetailsDataEvent	Usługa profilu użytkownika	• Usługa opinii • Usługa ofert zleceń • Usługa wiadomości	Zdarzenie generowane po każdej aktualizacji w celu synchronizacji danych użytkownika w systemie

5.2.5 Definiowanie granic transakcji

W tej sekcji opisane zostaną działania jakie zostały podjęte, aby zachować spójność w systemie podczas operacji modyfikujących rozprzestrzenionych pomiędzy różnymi usługi. Większość operacji realizowanych jest w ramach lokalnych transakcji usług, dlatego wyznaczanie granic transakcji oraz implementacja transakcji rozproszonych nie jest konieczna. Świadczy to o wysokim poziomie autonomii usługi oraz luźnych powiązaniach pomiędzy nimi.

Wyjątkiem jest obsługa procesu rejestracji, która wymaga objęcia w transakcję, aby zachować spójność w danych pomiędzy usługami. W tym celu wykorzystano wzorzec projektowy Saga, omówiony w części teoretycznej pracy. W przypadku niepowodzenia akcji po stronie usługi profilu użytkownika serwis przesyła zdarzenie `RegisterFailedEvent` na dedykowany temat usługi uwierzytelniającej. Następnie usługa usuwa dane dotyczące danego użytkownika, co skutkuje odrzuceniem całej operacji.

Dla procesów obsługujących wysyłkę wiadomości mailowych weryfikacyjnych oraz przypominających hasło, system umożliwia ręczne ponowienie wysyłki w przypadku niepowodzenia.

Podczas niepowodzenia transakcji w trakcie konsumpcji zdarzenia `NotificationEvent`, usługa powiadomień ponawia próbę przetworzenia operacji poprzez zdefiniowaną liczbę razy. W przypadku dalszego niepowodzenia, operacja jest ignorowana, jednakże powiadomienie może zostać nadal ponowione ręcznie poprzez interfejs API.

W przypadku innych komunikatów asynchronicznych system akceptuje ryzyko niepowodzenia operacji, wychodząc założenia, że system sam osiągnie stan spójności w pewnym momencie w przyszłości.

5.2.6 Zarządzanie zasobami pamięci i danych

W przypadku mikroserwisów, każda usługa posiada odrębną bazę danych, zlokalizowaną na dedykowanym serwerze bazodanowym. Izolacja baz danych oraz przypisanie dedykowanych serwerów dla każdej usługi zapewnia ich niezależność konfiguracyjną i operacyjną, co pozwala na optymalizację zasobów w zależności od wymagań i obciążenia poszczególnych usług.

Elasticsearch, stosowany w aplikacji monolitycznej do wyszukiwania ofert zleceń oraz użytkowników, został umieszczony w warstwie usług profilu użytkownika oraz ofert zleceń. Każdy z tych serwisów przechowuje indeksy danych wykorzystywanych do wyszukiwania na odrębnych instancjach bazy Elasticsearch.

Usługa profilu użytkownika oraz ofert zleceń korzystają z Redis'a jako rozproszonej pamięci podręcznej. Redis umożliwia szybki dostęp do danych oraz zapewnia spójność danych pomiędzy różnymi instancjami usługi.

Usługa danych statycznych wykorzystuje bibliotekę Caffeine [81] do przechowywania pamięci podręcznej. Caffeine przechowuję dane w pamięci podręcznej aplikacji, oznacza to, że każda instancja aplikacji będzie posiadała własny, odrębny stan pamięci podręcznej. Zastosowanie tej metody wynika z niskiego prawdopodobieństwa modyfikacji danych w tej usłudze oraz niskiego prawdopodobieństwa skalowania usługi do większej liczby instancji niż jedna. Decyzja ta zapewnia prostotę zarządzania i odpowiednią wydajność w kontekście specyficznych wymagań usługi danych statycznych.

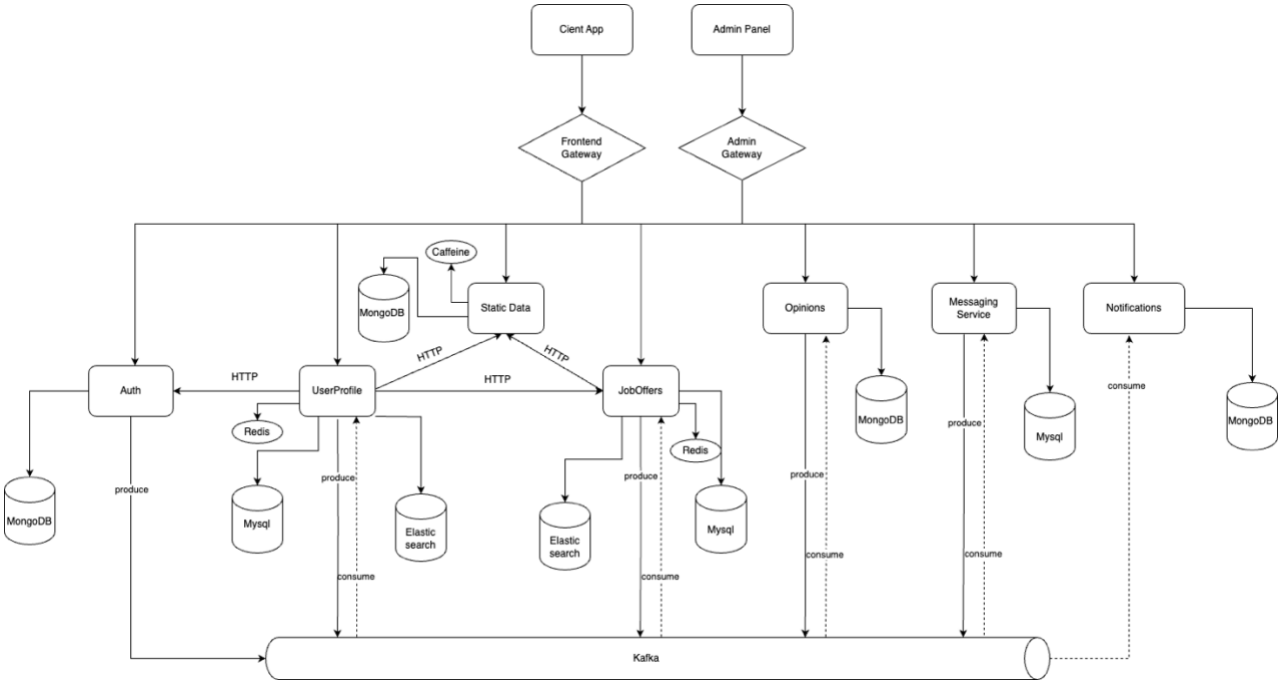
Podczas projektowania systemu w architekturze mikroserwisowej podjęto decyzję o rezygnacji z wykorzystania usługi Varnish jako serwera pośredniczącego pamięci podręcznej. Decyzja ta wynikała z niewystarczającej integracji Varnish z zintegrowanym środowiskiem Spring Framework. Dodatkowo zastosowanie serwera proxy mogłoby negatywnie wpłynąć na wyniki eksperymentu wydajnościowego, który zostanie omówiony w kolejnym rozdziale.

5.2.7 Komunikacja z interfejsem użytkownika

Serwer aplikacji komunikuję się z dwoma interfejsami, aplikacją główną oraz panelem administracyjnym. W tym celu wykorzystany został wzorzec BFF opisany we wcześniejszej części pracy. Każda aplikacja posiada oddzielny punkt wejściowy, którym jest brama API. Brama API jest usługą pośredniczącą, która odpowiada za kierowanie ruchem w aplikacji oraz zapewnia izolację interfejsu użytkownika od usług mikroserwisowych. Brama API umożliwia również maskowanie struktury API dostarczanej przez mikrousługi. Wykorzystanie oddzielnej bramy dla każdego interfejsu użytkownika zapewnia większą elastyczność oraz bezpieczeństwo, pozwalając na dostosowanie konfiguracji do specyficznych potrzeb każdego z interfejsów.

Brama API pozwala również na monitorowanie procesów biznesowych poszczególnych użytkowników wewnątrz systemu mikroserwisowego. W celu identyfikacji błędów w procesach biznesowych w poszczególnych zapytaniach w systemie, wprowadzono identyfikator korelacji zapytania, który jest generowany na poziomie bramy API i dostarczany do zapytań niższego poziomu do każdej powiązanej usługi. Identyfikator zapytania pozwala na śledzenie całej ścieżki zapytania przechodzącego przez różne mikroserwisy, co znacząco ułatwia diagnozowanie problemów.

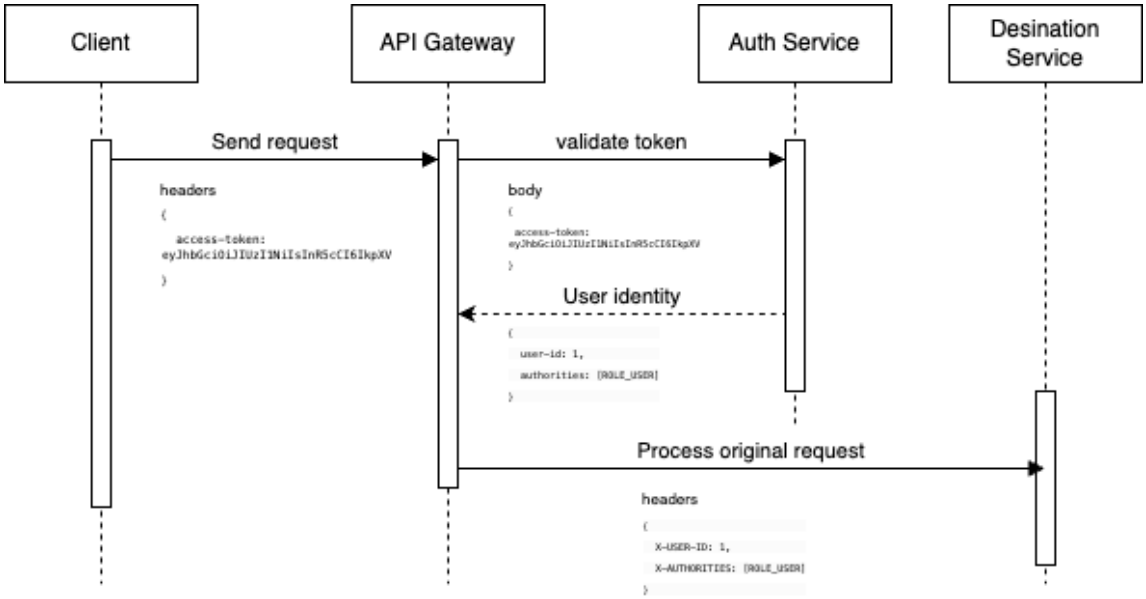
Schemat pełnej architektury aplikacji w architekturze mikroserwisowej zawierający wszystkie aspekty omówione w tym rozdziale dekompozycji aplikacji na mikroserwisy został przedstawiony na rysunku 44.



Rys 44. Schemat architektury mikroserwisowej aplikacji wspomagającej znajdowanie zleceń dla programistów. Źródło: opracowanie własne

5.3 Uwierzytelnianie i autoryzacja

Proces uwierzytelniania w architekturze mikroserwisowej został zaprojektowany na globalnym poziomie z wykorzystaniem bramy API. Brama API przechowuję listę ścieżek URI (ang. *whitelist*) dla każdego serwisu zezwalających na bezpośredni dostęp nieautoryzowanym użytkownikom. W przypadku zapytań do zasobów nieznajdujących się na tej liście, brama API wymaga podania tokenu JWT w nagłówku zapytania. W sytuacji braku tokenu, zapytanie zwróci status 401. Jeżeli token zostanie podany, brama API wywoła usługę uwierzytelniającą w celu jego walidacji. W przypadku pomyślnej walidacji, usługa zwróci biznesowy identyfikator użytkownika oraz identyfikator w systemie uwierzytelniającym wraz z przypisanymi rolami użytkownika. Następnie identyfikator biznesowy wraz z rolami użytkownika jest przekazywany w nagłówku w zapytaniu przekierowującym do usługi docelowej. Za proces autoryzacji zasobów odpowiedzialna jest każda usługa z osobna wykorzystując dane przekazane w nagłówku. Pozytywny przypadek procesu uwierzytelniania w systemie został zilustrowany na poniższym diagramie sekwencji.



Rys 45. Proces uwierzytelniający użytkownika w systemie mikroserwisowym. Źródło: opracowanie własne

5.4 Implementacja wybranych mikroserwisów

W niniejszym podrozdziale przedstawiona zostanie implementacja wybranych mikroserwisów, które stanowią kluczowe elementy w architekturze aplikacji. Omówione zostaną narzędzia oraz technologie wykorzystane w procesie tworzenia tych mikroserwisów, a także napotkane wyzwania i sposoby ich rozwiązania.

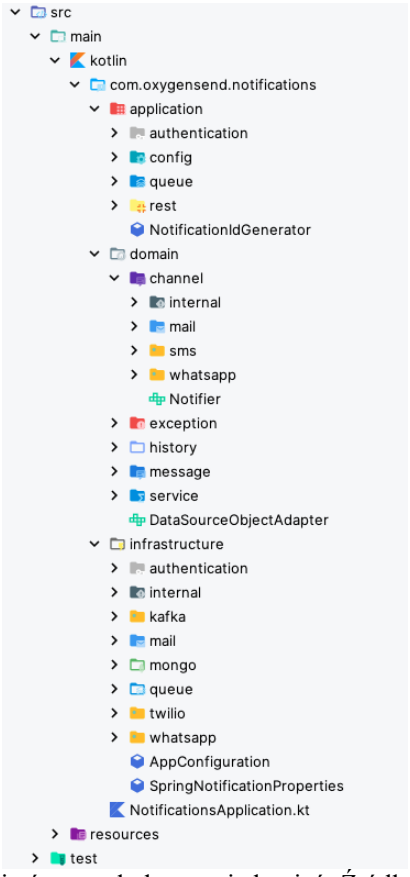
5.4.1 Serwis powiadomień

Usługa powiadomień została zaimplementowana w języku Kotlin. Usługa tak jak każda inna została napisana wykorzystując podejście DDD, z jasno zdefiniowaną strukturą katalogów, która oddziela domenę od warstw aplikacji oraz infrastruktury. Schemat katalogów w projekcie został przedstawiony na ilustracji 46.

Serwis jest uniwersalnym rozwiązaniem, który może zostać wykorzystany również w innych projektach. Umożliwia przesyłanie komunikatów mailowych, SMS-owych, wewnętrznych oraz za pośrednictwem WhatsApp. Powiadomienia SMS-owe oraz WhatsApp są przesyłane za pomocą zewnętrznego API Twilio [82]. Powiadomienia mailowe są wysyłane za pomocą biblioteki Agnus Mail [98], natomiast komunikaty wewnętrzne są zapisywane do bazy danych. Komunikaty mogą być przesyłane synchronicznie oraz asynchronicznie za pomocą REST lub asynchronicznie z wykorzystaniem brokera wiadomości Kafka.

Serwis powiadomień bazuję na profilach Spring, które umożliwiają uruchamianie poszczególnych funkcjonalności w zależności od włączonych profili. W przypadku

omawianego systemu, dla usługi powiadomień aktywowano funkcjonalności: komunikaty mailowe, komunikaty wewnętrzne oraz komunikację za pomocą Kafki.



Rys 46. Struktura pakietów w usłudze powiadomień. Źródło: opracowanie własne

Zarówno dane dostarczone za pomocą REST oraz Kafki mapowane są do wspólnego generycznego obiektu domenowego NotificationMessage, przedstawionego na listingu 2. Obiekty domenowe obsługiwane są przez interfejs NotificationMessageProcessor którego implementacja została zaprezentowana na listingu 3. Klasa NotificationConsumerRegistry odpowiada za załadowanie odpowiedniej implementacji interfejsu NotificationConsumer w zależności od typu powiadomienia. Wykorzystany został do tego wzorzec projektowy strategia. Logika przetworzenia powiadomienia w obu przypadkach jest taka sama, różnica polega na tym, że powiadomienia asynchroniczne dodawane są do kolejki FIFO (ang. *First In First Out*), a synchroniczne przetwarzane natychmiast.

Usługa przesyłania powiadomień jest przeznaczona do użytku wewnątrz systemu mikroserwisowego, dlatego w pierwszym kroku dokonywana jest autoryzacja usługi przesyłającej komunikat, jeżeli taka konfiguracja została włączona w ustawieniach startowych. Weryfikowany jest parametr login przekazywany w nagłówku powiadomienia, w celu weryfikacji z zaszyfrowanym kodem przekazanym na starcie aplikacji do maszyny wirtualnej.

Następnie weryfikowany jest identyfikator usługi wywołującej serviceId. Lista wspieranych usług jest również definiowana w pliku konfiguracyjnym. Po pomyślnej weryfikacji, powiadomienie zostaje przetworzone przez interfejs NotificationConsumer i zapisane w bazie danych, jeżeli taka opcja została skonfigurowana w parametrach aplikacji. Interfejs NotificationConsumer oraz jego implementacja dla powiadomień mailowych została przedstawiona na listingu 4.

Klasa QueueExecutor przedstawiona na listingu 5, została zaimplementowana z wykorzystaniem coroutines oraz wirtualnych wątków JVM. Każdy coroutine uruchamiany jest na nowym wirtualnym wątku. Coroutines są uruchamiane zgodnie z identyfikatorem przekazywanym w nagłówku powiadomienia, w przypadku omawianego systemu jest to identyfikator użytkownika. Jeżeli istnieje coroutine dla danego identyfikatora, kolejne powiadomienia są dołączane do kolejki FIFO. Dzięki temu powiadomienia dla danego użytkownika są przesyłane w odpowiedniej kolejności, nie wpływając na dostarczanie innych powiadomień w tym samym czasie.

Listing 2. Typ domenowego powiadomienia w jęzku Kotlin

```
data class NotificationMessage<T : NotificationPayload>(
   val headers: NotificationHeaders,
   val payload: T
)
data class NotificationHeaders(
   val id: String?,
   val type: String,
   val serviceId: String ,
   val timestamp: Instant? = null,
   val login: String? = null,
   val requestId: String? = null,
)
interface NotificationPayload {
   val recipients: Set<Recipient>
}
```

Listing 3. Implementacja interfejsu NotificationMessageProcessor w języku Kotlin

```
package com.oxygensend.notifications.infrastructure.queue

import com.oxygensend.notifications.application.authentication.AuthService
import com.oxygensend.notifications.application.queue.NotificationConsumerRegistry
import com.oxygensend.notifications.application.queue.QueueExecutor
import com.oxygensend.notifications.domain.message.NotificationMessage
import com.oxygensend.notifications.domain.message.NotificationMessageProcessor
import com.oxygensend.notifications.domain.message.NotificationPayload
import org.slf4j.LoggerFactory
import org.springframework.stereotype.Component

@Component
internal class DomainNotificationMessageProcessor(
    private val consumerRegistry: NotificationConsumerRegistry,
    private val authService: AuthService,
    private val queueExecutor: QueueExecutor
) : NotificationMessageProcessor {
    companion object {
        private val LOGGER = LoggerFactory.getLogger(this::class.java)
    }

    override fun <T : NotificationPayload> process(message: NotificationMessage<T>) {
        val consumer = consumerRegistry.get(message.payload)
        if (consumer == null) {
            LOGGER.info("Undefined consumer for: {}", message.headers.type)
            return
        }

        queueExecutor.execute(message.headers.id) {
            authService.authenticate(message.headers)
            consumer.consumeSafely(message);
        }
    }

    override fun <T : NotificationPayload> processSynchronously(message:
NotificationMessage<T>) {
        val consumer = consumerRegistry.get(message.payload)
        if (consumer == null) {
            LOGGER.info("Undefined consumer for: {}", message.headers.type)
            return
        }

        authService.authenticate(message.headers)
        consumer.consumeSafely(message);
    }
}
```

Listing 4. Kod źródłowy przedstawiający interfejs NotificationConsumer oraz jedną z jego implementacji w jęzku Kotlin

```
package com.oxygensend.notifications.domain.message

import org.slf4j.LoggerFactory

interface NotificationConsumer<T : NotificationPayload> {
    companion object {
        private val LOGGER = LoggerFactory.getLogger(this::class.java)
    }


    fun consume(message: NotificationMessage<T>)


    fun consumeSafely(message: NotificationMessage<T>) {
        val loggingContext = loggingContext(message)

        LOGGER.info("Consuming message: {} ", loggingContext)

        try {
            consume(message)
        } catch (ex: Exception) {
            LOGGER.error("Error: {}", loggingContext, ex)
        }

    }

    private fun loggingContext(message: NotificationMessage<T>): String {
        return "[id=${message.headers.id}, type=${message.headers.type},
consumer=${this::class}, recipients=${message.payload.recipients}"
    }
}
package com.oxygensend.notifications.application.queue.consumer

import com.oxygensend.notifications.domain.history.NotificationHistoryRepository
import com.oxygensend.notifications.domain.channel.Notifier
import com.oxygensend.notifications.domain.channel.mail.Mail
import com.oxygensend.notifications.domain.message.NotificationConsumer
import com.oxygensend.notifications.domain.message.NotificationMessage
import org.springframework.stereotype.Component

@Component
final class MailNotificationConsumer(
    private val notifier: Notifier<Mail>,
    private val notificationHistoryRepository: NotificationHistoryRepository
) : NotificationConsumer<Mail> {

    override fun consume(message: NotificationMessage<Mail>) {
```

```
        val progress = notifier.notify(message.payload)
        notificationHistoryRepository.save(message, progress)
    }
}
```

Listing 5. Implementacja klasy QueueExecutor w języku Kotlin

```
package com.oxygensend.notifications.application.queue

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.ExecutorCoroutineDispatcher
import kotlinx.coroutines.asCoroutineDispatcher
import kotlinx.coroutines.launch
import org.springframework.stereotype.Component
import java.util.*
import java.util.concurrent.ConcurrentHashMap
import java.util.concurrent.ConcurrentLinkedQueue
import java.util.concurrent.ConcurrentMap
import java.util.concurrent.Executors

@Component
class QueueExecutor {
    private val tasks: ConcurrentMap<String, Queue<() -> Unit>> = ConcurrentHashMap()
    private val executor: ExecutorCoroutineDispatcher =
        Executors.newThreadPerTaskExecutor(Thread.ofVirtual().name("queue-",
0).factory()).asCoroutineDispatcher()

    fun execute(key: String?, runnable: () -> Unit) {
        if (key == null) {
            CoroutineScope(executor).launch {
                runnable()
            }
            return
        }

        tasks.computeIfAbsent(key) { ConcurrentLinkedQueue() }.apply {
            add(runnable)
            if (size == 1) {
                CoroutineScope(executor).launch {
                    queueExecution(key)
                }
            }
        }

    }

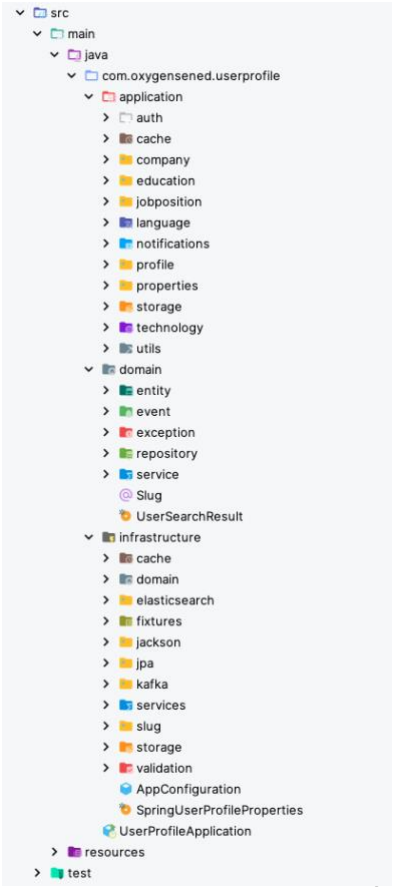
    private suspend fun queueExecution(key: String) {
        tasks[key]?.apply {
            while (isNotEmpty()) {
                poll()?.invoke()
```

```
        }
        tasks.remove(key)
    }
  }

}
```

Do przechowywania danych został wykorzystany nierelacyjny silnik bazy danych MongoDB [83]. Aplikacja posiada dokumentacje parametrów startowych w pliku w formacie Yaml, oraz dokumentacje API udostępnioną za pomocą OpenAPI. Do implementacji wykorzystany został menadżer wersji Maven [84] oraz narzędzie Docker Compose do w celu instalacji narzędzi MongoDB, Kafki, Zookeeper oraz Mailhoga [85] do testowania komunikacji mailowej.

5.4.2 Serwis profilu użytkownika



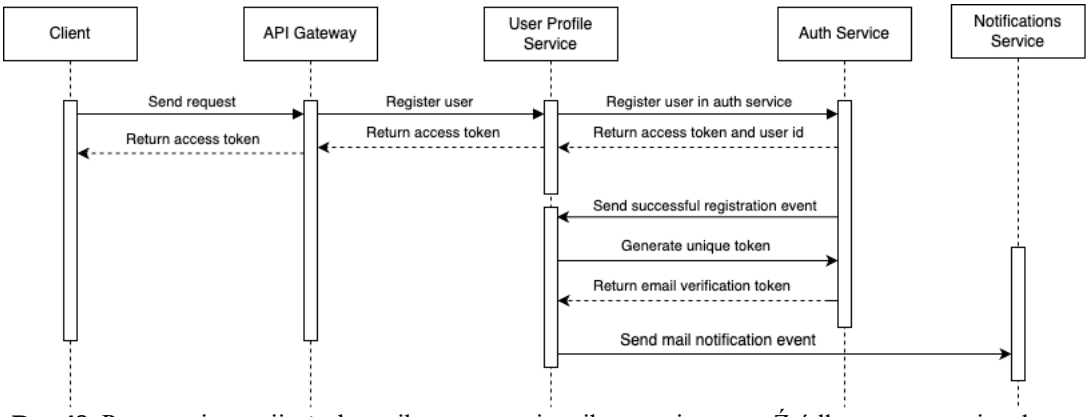
Rys 47. Struktura pakietów w usłudze profilu użytkownika. Źródło: opracowanie własne

Usługa profilu użytkownika została zaimplementowana w języku Java. Analogicznie do usługi powiadomień wykorzystane zostało podejście DDD, które umożliwia organizowanie kodu wokół dziedzinowych modeli, zapewniając większość przejrzystość. Profil użytkownika stanowi większy kontekst niżeli powiadomienia, jako że zawiera w sobie kilka kontekstów ograniczonych opisanych w procesie dekompozycji. W efekcie projekt jest bardziej

skomplikowany, co zadecydowało o jego implementacji w języku, który jest lepiej znany autorowi. Schemat katalogów w projekcie przedstawiono na ilustracji 47.

W warstwie aplikacji zaimplementowane zostały operacje CRUD w połączeniu z logiką biznesową funkcjonalności projektu.

Jak wspomniano wcześniej usługa profilu użytkownika zintegrowana jest z usługą uwierzytelniania, ponieważ odpowiada za rejestracje użytkowników w systemie oraz przesyłkę komunikatów mailowych związanych ze zmianą hasła oraz weryfikacji adresu mailowego. Komunikacja z usługą odbywa się zarówno synchronicznie, jak i asynchronicznie. Po pomyślnej rejestracji usługa uwierzytelniania natychmiast przesyła zdarzenie, które jest odczytywane z Kafki. Następnie, po odpowiedniej weryfikacji i wygenerowaniu unikalnego tokenu, nowe zdarzenie zostaje przesłane na topic usługi powiadomień, w celu wysłania e-maila z prośbą o weryfikację adresu. Jest to kolejna integracja usługi profilu użytkownika, dokładny przebieg procesu został zaprezentowany na rysunku 48.



Rys 48. Proces rejestracji użytkownika w systemie mikroserwisowym. Źródło: opracowanie własne

Aplikacja subskrybuje również topic udostępniony przez usługę opinii w celu pobrania statystyk opinii dla użytkowników. Usługa opinii po każdej aktualizacji opinii dla danego użytkownika przelicza wskaźnik opinii oraz liczbę opinii i przesyła dane na dedykowany topic. Usługa profilu użytkownika subskrybuje ten topic, aby uniknięcia bezpośrednich powiązań pomiędzy usługami przy pobieraniu danych. Dodatkowo usługa profilu użytkownika przesyła na dedykowany topic zaktualizowane dane dotyczące profilu użytkownika po każdej aktualizacji encji. Usługa komunikuję się również w sposób synchroniczny z usługą danych statycznych oraz usługą ofert pracy. Komunikacja synchroniczna w każdym przypadku została skonfigurowana za pomocą biblioteki Feign [86] z modułu Spring Cloud, która pozwala na tworzenie klienta dla usługi wykorzystując interfejs z adnotacjami. Na poniższym listingu 6 zaprezentowano implementacje interfejsu klienta Feign.

Listing 6. Implementacja klienta Feign dla usługi uwierzytelniającej w języku Java

```
package com.oxygensened.userprofile.infrastructure.services.auth;

import com.oxygensened.userprofile.infrastructure.services.auth.dto.AuthRegisterRequest;
import com.oxygensened.userprofile.infrastructure.services.auth.dto.AuthResponse;
import com.oxygensened.userprofile.infrastructure.services.auth.dto.CreateUserRequest;
import com.oxygensened.userprofile.infrastructure.services.auth.dto.TokenResponse;
import com.oxygensened.userprofile.infrastructure.services.auth.dto.UserIdRequest;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

public interface AuthClient {

    @RequestMapping(method = RequestMethod.POST, value = "/v1/auth/register")
    AuthResponse register(@RequestBody AuthRegisterRequest request);

    @RequestMapping(method = RequestMethod.POST, value =
"/v1/users/generate_email_verification_token")
    TokenResponse generateEmailVerificationToken(@RequestBody UserIdRequest request);

    @RequestMapping(method = RequestMethod.POST, value =
"/v1/users/generate_password_reset_token")
    TokenResponse generatePasswordResetToken(@RequestBody UserIdRequest request);
}
```

Aplikacja wykorzystuję Redis do przechowywania pamięci wewnętrznej, co umożliwia pozyskiwanie danych z różnych instancji serwisu ze względu na swoją rozproszoną strukturę. Dane w pamięci podręcznej serializowane są do postaci binarnej za pomocą biblioteki Kryo [87] oraz kompresowane przy użyciu biblioteki Snappy [88] w celu zaoszczędzenia pamięci i optymalizacji. Kryo jest wysokowydajną biblioteką do serializacji, przeznaczoną do szybkiej deserializacji i serializacji obiektów POJO w strumień bajtów. Snappy jest biblioteką do kompresji danych stworzoną przez Google. Cechuję się wysoce szybkim procesem kompresji oraz dekompresji danych, przez co idealnie sprawdza się do zastosowań wymagających szybkiego przetwarzania danych. W pamięci podręcznej przechowywane są za równo odpowiedzi usług jaki i dane drugiego poziomu np. dane ofert programistów przetworzone przez algorytm rekomendacji. Czas przechowywania danych w pamięci podręcznej konfigurowany jest w startowym pliku konfiguracyjnym.

Podobnie jak w wersji monolitycznej, aplikacja wykorzystuje Elasticsearch do efektywnego przeszukiwania danych. Elasticsearch zawiera jeden indeks, który jest wypełniany lub odświeżany (w zależności od konfiguracji) danymi użytkownika podczas startu aplikacji

oraz aktualizowany przy każdej zmianie w encji użytkownika. Spring Boot zapewnia prostą integrację z Elasticsearch poprzez moduł spring-boot-starter-data-elasticsearch.

Usługa danych korzysta z relacyjnej bazy danych MySQL, wykorzystując bibliotekę Hibernate oraz moduł Spring JPA do komunikacji z bazą danych. Hibernate [89] jest biblioteką ORM dla języka Java, natomiast Spring JPA zapewnia integrację z JPA (ang. *Java Persistence API*), upraszczając pracę z bazami danych. Do przechowywania historii migracji relacyjnej bazy danych wykorzystano Liquibase [90].

Serwis zapisuje i odczytuje dane z lokalnego dysku. Operacje wejścia oraz wyjścia dysku wykorzystywane są w przypadku zdjęć profilowych użytkowników. Podjęto decyzję o używaniu lokalnej przestrzeni dyskowej zamiast chmury lub oddzielnego serwisu przechowującego dane, ponieważ w przypadku zdjęć profilowych kluczowe jest szybkie załadowanie ich na stronie interfejsu, co ma bezpośredni wpływ na jakość aplikacji (ang. *user experience*). Zdjęcia dodawane przez użytkowników formatowane są do formatu webP oraz skalowane do mniejszych rozmiarów w celu optymalizacji zasobów. W przypadku działania wielu instancji aplikacji w klastrze, spójność danych jest zapewniana dzięki funkcjonalności PersistentVolume, która pozwala na wskazanie hosta przechowującego dane w formacie NFS. Dzięki temu aplikacje nadal korzystają z tego samego miejsca do odczytu i zapisu danych.

Usługa profilu użytkownika wykonuje powtarzalne zadania w tle za pomocą operacji CRON. Mechanizm CRON umożliwia zaplanowanie powtarzalnych zadań, które są realizowane jako oddzielne procesy w systemie operacyjnym. Framework Spring oferuje adnotację @Scheduled do planowania takich zadań. Jednak w przypadku systemów rozproszonych, gdy aplikacja może mieć więcej niż jedną instancję, każda z instancji realizowałaby te same zadania jednocześnie. Aby uniknąć tego problemu, wykorzystano bibliotekę ShedLock [90], która umożliwia izolację zadania dla jednej instancji aplikacji. Dzięki temu zadanie jest wykonywane tylko raz w tym samym czasie. ShedLock używa połączenia z bazą danych do zapisywania informacji o zablokowaniu zasobów.

Podczas implementacji wykorzystano narzędzie Docker Compose do instalacji niezbędnych zasobów, takich jak MySQL, Kafka, Redis, Elasticsearch, Zookeeper, na potrzeby lokalnego testowania. Podobnie jak w przypadku usługi powiadomień oraz innych usług, jako menadżer pakietów wykorzystano Maven, a dokumentacja API została udostępniona za pomocą OpenAPI. W trakcie implementacji usługi nie napotkano żadnych problemów, poza tymi wynikającymi z natury rozproszonej architektury, takimi jak wybór lidera podczas operacji CRON, synchronizacja danych oraz operacje wejścia/wyjścia na dysku.

5.5 Infrastruktura systemu mikroserwisowego

5.5.1 Wykrywanie usług

System mikroserwisowy wykorzystuję Kubernetes [91] do rejestracji usług w systemie. Kubernetes jest platformą do zarządzania kontenerami, która umożliwia automatyzację wdrożeń, skalowanie oraz przeprowadzanie operacji w aplikacjach kontenerowych. Środowisko Spring Cloud pozwala na integrację z funkcjonalnością wykrywania usług dostarczoną przez Kubernetes, dzięki bibliotece spring-cloud-kubernetes-client-discovery. Integracja ta opiera się na rekordach DNS. Kubernetes automatycznie tworzy rekordy DNS dla wszystkich usług w klastrze, co umożliwia wykrywanie usług wewnątrz przestrzeni nazw w klastrze. Spring Cloud, wykorzystując API Kubernetesa, pobiera dostępne usługi wraz z ich instancjami i wykorzystuje te informacje do kierowania ruchem. URI do poszczególnych serwisów w bramie API konfigurowane są na podstawie nazwy serwisu w Kubernetesie. Poniższy listing 7 prezentuje konfigurację ścieżki w bramie API dla usługi opinii, która w klastrze Kubernetesa ma nadaną nazwę opinions-service.

Listing 7. Konfiguracja bramy API dla ścieżki w języku YAML.

```
spring:
 cloud:
  gateway:
   routes:
    - id: opinions
      uri: lb://opinions-service
      predicates:
        - Path=/opinions/**, /users-opinion-details/**
      filters:
        - RewritePath=/opinions(?<segment>.*),/api/v1/opinions$\{segment}
        - RewritePath=/users-opinion-
details/(?<segment>.*),/api/v1/users/$\{segment}/opinions-details
        - name: AuthPrefilter
          args:
            excludedPaths: [ "GET,/api/v1/opinions", "GET,/api/v1/opinions/.*/comments",
"GET,/api/v1/users/.*/opinions-details" ]
```

5.5.2 Równoważenie obciążenia

Kubernetes udostępnia wbudowane mechanizmy do konfiguracji load balancera dla usług w klastrze, natomiast ze względu na niezależność systemu od platformy kubernetes oraz dostawców chmurowych zdecydowano się na wykorzystanie rozwiązania dostarczonego przez Spring Cloud. Domyślnym algorytmem wykorzystywanym przez Spring Cloud Load Balancer jest RoundRobin [92]. Algorytm RoundRobin polega na rozdzielaniu żądań równomiernie pomiędzy dostępne instancje w cykliczny sposób. Każde kolejne zapytanie jest przekazywane

do następnej instancji na liście, a po dojściu na koniec listy cykl zaczyna się od początku. Lista dostępnych usług i ich instancji pobierana jest z rejestratora usług, co zostało opisane we wcześniejszym podrozdziale.

5.5.3 Konfiguracja klastra

Aplikacja oparta na architekturze mikroserwisowej została zaprojektowana do działania w klastrze Kubernetes. Dla każdej usługi tworzony jest obraz Docker, który następnie uruchamiany jest jako serwis w dedykowanej przestrzeni nazw. W ramach tego projektu wykorzystano domyślną przestrzeń nazw. Każda z usług ma zdefiniowane podstawowe zasoby Service oraz Deployment.

Service stanowi abstrakcję, która umożliwia udostępnienie podów usługi w sieci oraz identyfikację usługi w przestrzeni nazw. Może być on również definiowany jako load balancer, który równomiernie rozdziela ruch sieciowy pomiędzy pody usługi.

Deployment konfiguruje aplikację, określając liczbę jednocześnie działających instancji usługi. pozwala również na konfigurację wolumenów oraz montowanie zewnętrznych dysków, co jest szczególnie przydatne w przypadku operacji wejścia i wyjścia na dysku lokalnym. Funkcjonalność ta została wykorzystana w celu synchronizacji danych przechowywanych na dysku pomiędzy instancjami. Dodatkowo, zastosowano ConfigMap, która umożliwia przekazywanie parametrów konfiguracyjnych używanych przez aplikację do klastra. ConfigMap umożliwia centralne zarządzanie konfiguracją, sprawiając, że proces aktualizacji aplikacji jest bardziej elastyczny.

W celu umożliwienia komunikacji pomiędzy Spring Cloud, a API Kubernetesa potrzebne jest zdefiniowanie konta usługa (ang. *Service Account*) z odpowiednimi rolami (ang. *Cluster Roles*) dla usług bramy API w konfiguracji Kubernetesa. Service Account umożliwia uwierzytelnianie i autoryzację aplikacji działających w klastrze do operacji na zasobach udostępnionych przez Kubernetes. Nazwa konta musi być podana podczas definicji Deployment bramy API, aby odpowiednie role zostały przypisane do usługi.

Usługi są dostępne wyłącznie wewnątrz klastra i nie mają dostępu z zewnątrz. Nie ma potrzeby, aby wszystkie usługi były widoczne, jeśli cała komunikacja przechodzi przez bramę API. Kubernetes dostarcza funkcjonalność Ingress, która umożliwia eksponowanie wybranych serwisów na zewnątrz klastra. Natomiast na potrzeby przeprowadzanego badania konfiguracja Ingress nie jest wymagana, ponieważ aplikacja nie jest eksponowana poza sieć lokalną.

Na listingu 8 przedstawiona została pełna konfiguracja dla bramy API interfejsu użytkownika, aby zaprezentować sposób konfiguracji usługi w klastrze Kubernetes.

Listing 8. Konfiguracja usługi bramy API interfejsu użytkownika w klastrze Kubernetes w jęzku YAML.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: gateway-config
data:
  config.yml: |
    spring:
      cloud:
        kubernetes:
          enabled: true
    eureka:
      client:
        enabled: false
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-service-account
  namespace: default
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: monitoring-cluster-role
rules:
  - apiGroups: [""]
    resources: ["nodes", "services", "pods", "endpoints"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get"]
  - apiGroups: ["extensions"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: monitoring-cluster-role-binding
roleRef:
  kind: ClusterRole
  name: monitoring-cluster-role
  apiGroup: rbac.authorization.k8s.io
subjects:
  - kind: ServiceAccount
```

```
    name: monitoring-service-account
    namespace: default
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway-service
  labels:
    app: gateway-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gateway-service
  template:
    metadata:
      labels:
        app: gateway-service
    spec:
      serviceAccountName: monitoring-service-account
      containers:
        - name: gateway-service
          image: shreddler/outsource_me-gateway:1.0.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
              protocol: TCP
          volumeMounts:
            - name: config-volume
              mountPath: /app/config
          livenessProbe:
            httpGet:
              port: 8080
              path: /manage/health
            initialDelaySeconds: 20
            timeoutSeconds: 15
          readinessProbe:
            httpGet:
              port: 8080
              path: /manage/health
            initialDelaySeconds: 5
            timeoutSeconds: 10
      volumes:
        - name: config-volume
          configMap:
            name: gateway-config
apiVersion: v1
kind: Service
metadata:
  name: gateway-service
spec:
```

```
ports:
  - targetPort: 8080
    port: 80
    protocol: TCP
selector:
  app: gateway-service
```

5.5.4 Zewnętrzne usługi

Zewnętrzne usługi oraz zasoby, które są wykorzystywane przez system mikroserwisowy uruchamiane są spoza klastra Kubernetes. Wszystkie usługi zostały poddane konteneryzacji z wykorzystaniem narzędzia Docker Compose. Konteneryzacja zapewnia izolację procesów minimalizując ryzyko problemów wynikających z różnic konfiguracyjnych systemu operacyjnego. Konteneryzacja pozwala również na uruchomienie wielu instancji tego samego zasobu na jednym serwerze, ponieważ tak jak zostało przedstawione na schemacie architektonicznym każda usługa wymaga połączenia z odizolowanym źródłem zarządzającym danymi. Poprzez rozdzielenie infrastruktury aplikacji od infrastruktury zewnętrznych usług system jest bardziej agnostyczny na zmiany i nie jest powiązany z jednym konkretnym dostawcą usług zewnętrznych. Ułatwia to przenoszenie infrastruktury systemu między różnymi środowiskami, ponieważ rekonfiguracja klastra nie jest wymagana, a jedynie podmiana źródła w pliku konfiguracyjnym aplikacji.

6 Testy

Aby porównać omawiane w temacie pracy architektury przeprowadzony został eksperyment badający wydajność aplikacji opisanych w dwóch poprzednich rozdziałach. W tym rozdziale przedstawione zostaną wyniki przeprowadzanej analizy badającej zachowanie obu aplikacji podczas różnego natężenia ruchu, uwzględniając różne konfiguracje aplikacji mikroserwisowej.

Testy zostały przeprowadzone na lokalnym środowisku korzystając z dwóch urządzeń z poniższymi parametrami:

Serwer 1: MacBook Pro 16:

- System operacyjny: macOS Sonoma 14.5,
- Processor: Apple M3 Pro 12 rdzeni, 12 wątków,
- Pamięć RAM: 36 GB,
- Pamięć masowa: 1 TB SSD.

Serwer 2: Lenovo Legion Y350-15ICH:

- System operacyjny: Linux Mint 21.2 Cinnamon,
- Processor: Intel Core i5-8300H, 4 rdzenie, 8 wątków,
- Pamięć RAM: 16 GB,
- Pamięć masowa: 256 GB SSD.

Aplikacja monolityczna wraz z wszystkimi zależnościami została w całości uruchomiona w kontenerach za pomocą narzędzia Docker Compose na serwerze 1. W trakcie testów wyłączono serwer pośredniczący Varnish, odpowiedzialny za obsługę pamięci podręcznej, aby wyniki odzwierciedlały rzeczywistą wydajność aplikacji monolitycznej, a nie serwera pośredniczącego, odpowiedzialnego za pamięć podręczną.

Aplikacja mikroserwisowa została uruchomiona na serwerze 1 w lokalnym klastrze Kubernetes. Do postawienia klastra wykorzystane zostało narzędzie kind [99]. Kind wykorzystuje kontenery Docker do tworzenia klastrów na lokalnych maszynach. Ze względu na architektoniczne założenie dotyczące izolacji zasobów, konieczne było uruchomienie zasobów na dwóch serwerach z uwagi na problemy wydajnościowe podczas korzystania

wyłącznie z serwera 1. Wszystkie zasoby zostały uruchomione wewnątrz kontenerów. Poniżej przedstawiono spis instancji zasobów uruchomionych na obu serwerach:

Serwer 1:

- Elasticsearch – usługa profilu użytkownika,
- MongoDB – usługa uwierzytelniania,
- Redis – usługa profilu użytkownika,
- Redis – usługa ofert zleceń.

Serwer 2:

- Kafka, Zookeeper,
- Mailhog,
- Elasticsearch – usługa ofert zleceń,
- MongoDB – usługa powiadomień,
- MongoDB – usługa danych statycznych,
- MongoDB – usługa opinii,
- MySQL – usługa wiadomości,
- MySQL – usługa ofert zleceń,
- MySQL – usługa profilu użytkownika.

Aplikacje w obu architekturach zostały załadowane taką samą ilością sztucznych danych. Dane te miały na celu stworzenie realistycznych warunków testowych, umożliwiając przeprowadzenie rzetelnych analiz wydajności i zachowania aplikacji w różnych scenariuszach użytkowania. Taka sama ilość danych pozwala na sprawdzenie jak różne podejścia architektoniczne radzą sobie z obciążeniem przy tych samych danych wejściowych.

Aplikacja monolityczna została uruchomiona na serwerze Nginx z domyślnymi ustawieniami.

Aplikacje mikroserwisowe zostały uruchomione na serwerze Apache Tomcat [96] z domyślnymi ustawieniami.

Do przeprowadzenia eksperymentu wykorzystano narzędzie JMeter [97], które umożliwia testowanie wydajności aplikacji internetowych.

Podczas wszystkich przeprowadzonych testów sprawdzono wpływ skalowania poziomego aplikacji w architekturze mikroserwisowej na jej wydajność. W tym celu przetestowano różne konfigurację mikroserwisów, obejmujące jedną, dwie oraz cztery repliki usług.

6.1 Testy obciążeniowe

Testy obciążeniowe mają na celu sprawdzenie jak system radzi sobie przy normalnym obciążeniu użytkowników. Pozwalają one określić jak dużo ruch system może obsłużyć przed spadkiem wydajności.

W ramach testów obciążeniowych przetestowano dwa scenariusze sprawdzające przepustowość dla zapytań GET oraz POST. Przewidywane obciążenie dla aplikacji monolitycznej, wspomniane w wymaganiach niefunkcjonalnych w pracy inżynierskiej [63] wynosi 1000 użytkowników na minutę.

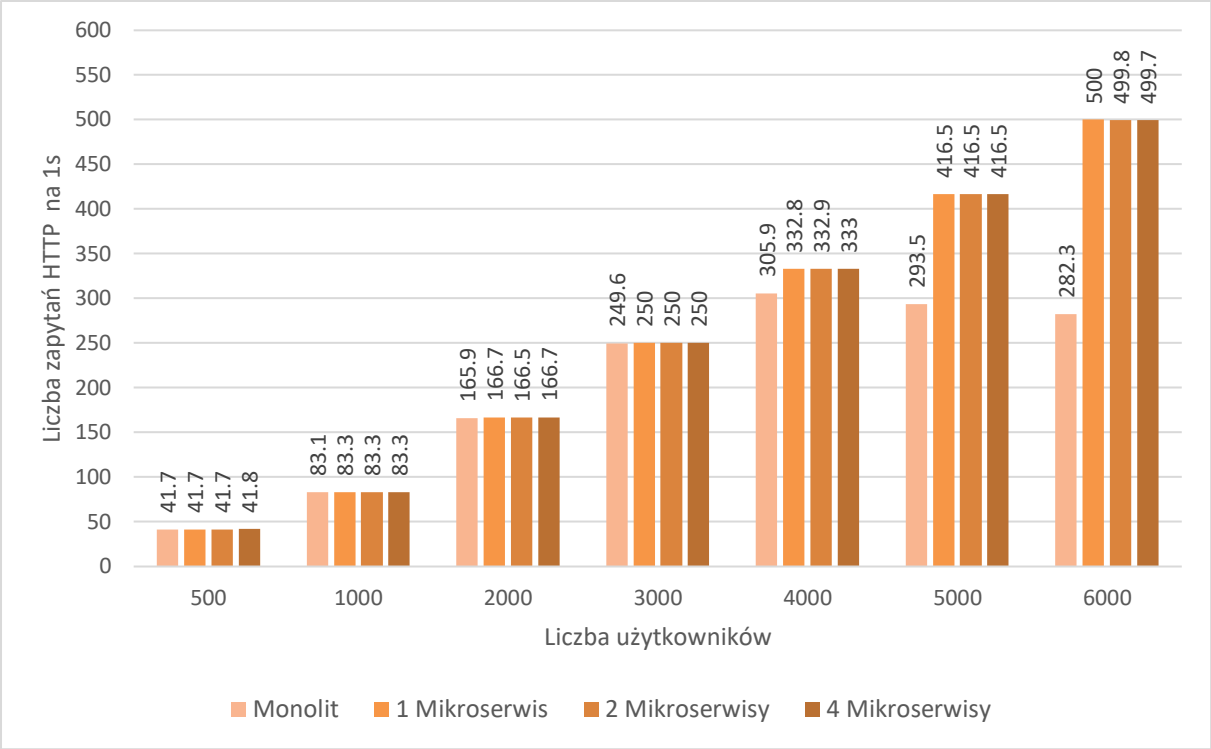
6.1.1 Testy pod stałym obciążeniem

Pierwszym rodzajem przeprowadzonych testów są testy pod stałym obciążeniem, które charakteryzują się tym, że liczba żądań wykonanych do serwera jest stała i zależna od ilości wątków uruchomionych w tym samym czasie.

Konfiguracja JMeter wykorzystana do testów pod stałym obciążeniem prezentuje się następująco:

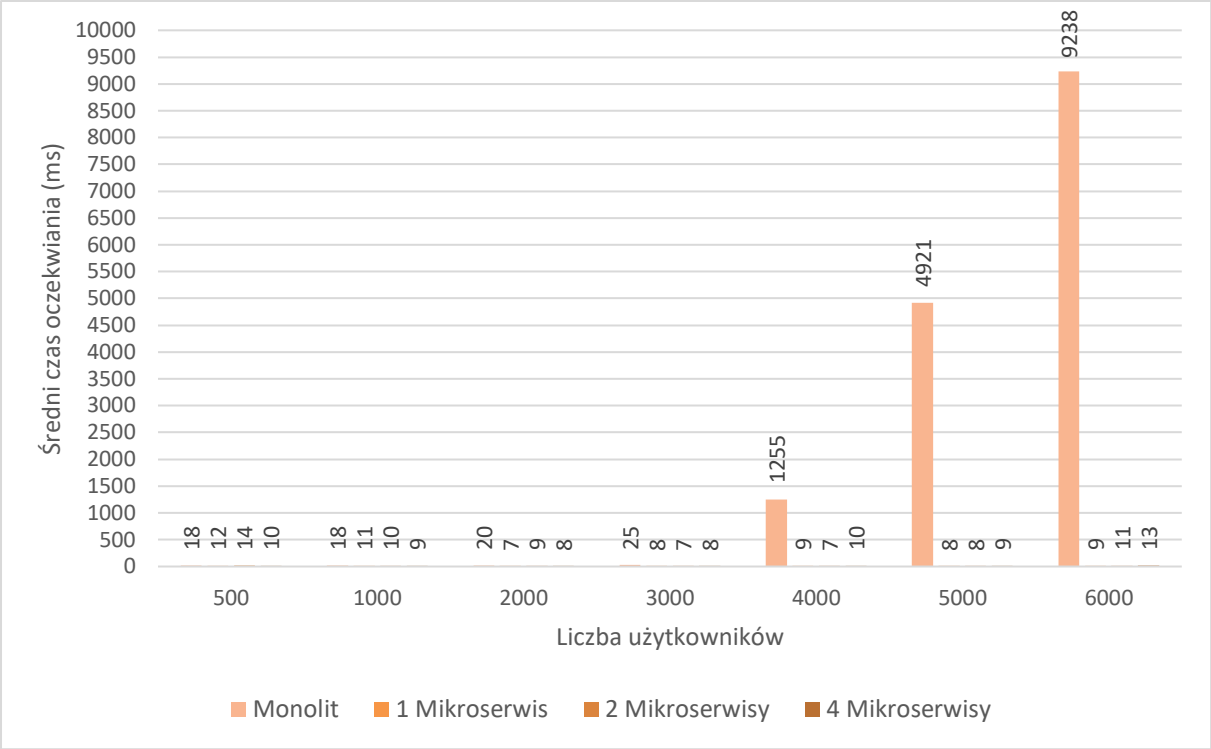
- Liczba użytkowników (wątków): 500 – 6000,
- Czas trwania: 60s,
- Liczba iteracji: 5.

Pierwszy scenariusz testowy zakłada sprawdzenie zachowania aplikacji w sytuacji, gdy ruch jest rozłożony na różne zasoby. W tym celu przygotowano test, który obejmuje wszystkie zasoby GET wykorzystywane w interfejsie użytkownika aplikacji, podczas którego użytkownik w każdej iteracji wybiera w sposób losowy odpowiedni zasób. Podczas testu zmierzono przepustowość oraz średni czas trwania oczekiwania na odpowiedz, uzyskane wyniki zostały przedstawione na wykresie 1 oraz 2.



Wykres 1. Wykres przedstawiający przepustowość aplikacji w stosunku do liczby użytkowników dla zasobów GET.

Jak można zauważyć na wykresie 1 wszystkie wersje aplikacji wykazują porównywalną wydajność przy obsłudze do 3000 użytkowników. W przypadku obciążenia na poziomie 4000 wątków, aplikacja monolityczna zaczyna wykazywać niższą wydajność w porównaniu do aplikacji opartych na mikroserwisach, co staję się jeszcze lepiej widoczne przy większym obciążeniu, gdzie prezentuję znacząco mniejszą przepustowość. Wszystkie konfiguracje aplikacji mikroserwisowych wykazały takie identyczne rezultaty.

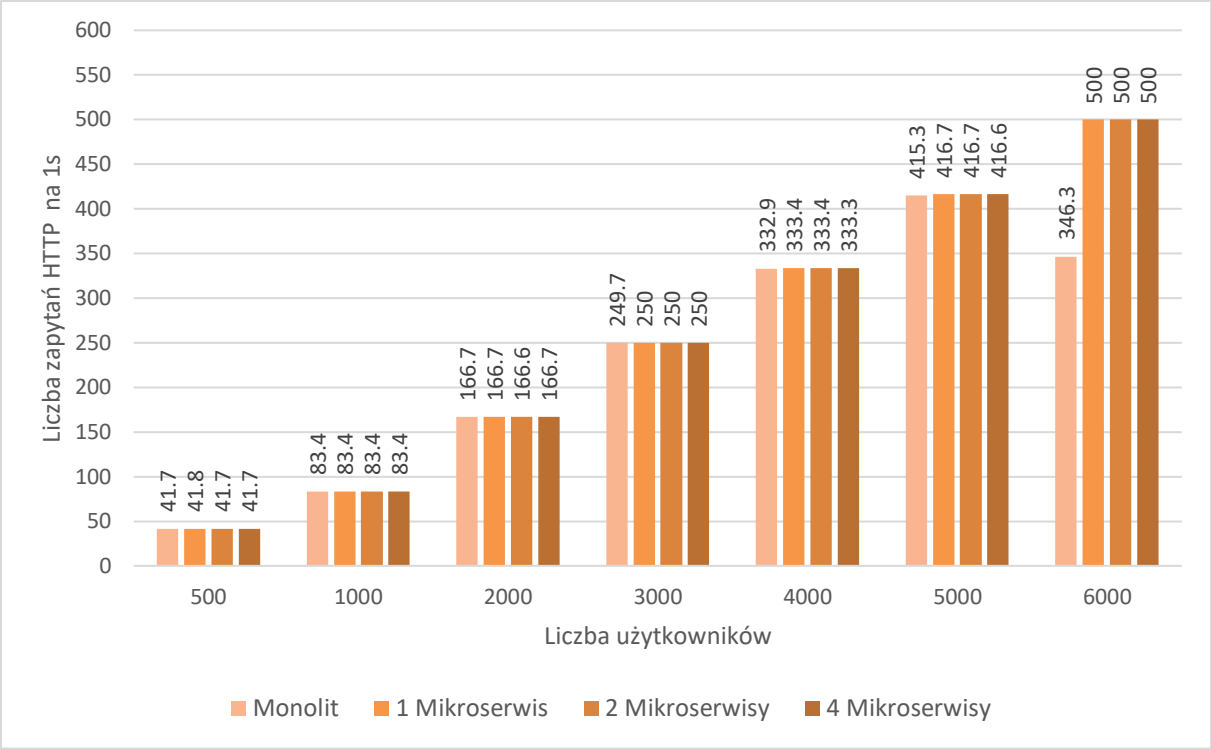


Wykres 2. Wykres przedstawiający średni czas oczekiwania na dane z zasobów GET w stosunku do liczby użytkowników.

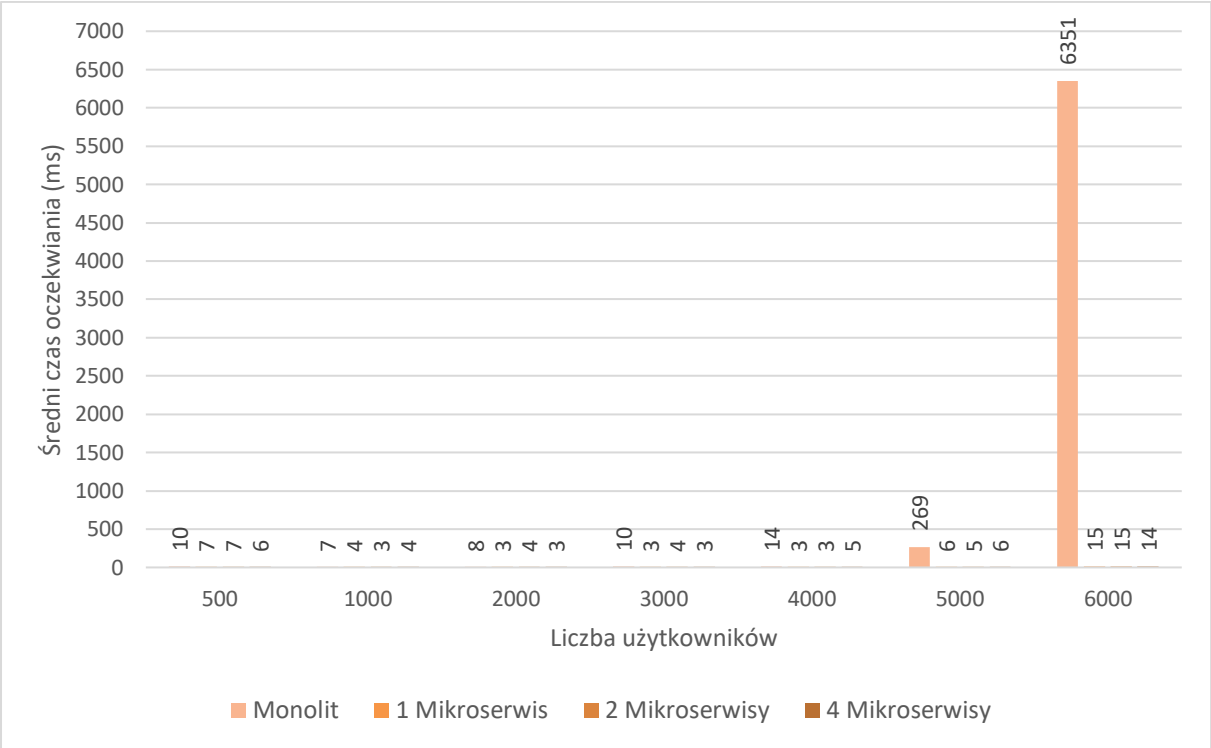
Przechodząc do czasów oczekiwania zaprezentowanych na wykresie 2, można zaobserwować podobny trend jak w przypadku przepustowości. Mianowicie wszystkie aplikacje utrzymują porównywalną wydajność do poziomu obciążenia wynoszącego 3000 użytkowników, przy czym dla aplikacji monolitycznej czas oczekiwania zaczyna wzrastać od poziomu 4000 użytkowników. Można zauważyć, że średni czas oczekiwania w przypadku aplikacji mikroserwisowych pozostaje niezmienny, niezależnie od wzrost obciążenia i jest on mniejszy o około 1000 razy w przypadku 6000 użytkowników.

Kolejny test został wykonany na stronie pojedynczej oferty zleceń, w celu weryfikacji jak rozłożenie ruchu w poprzednim teście wpłynęło na przepustowość aplikacji. Wyniki zostały przedstawiony na wykresie 3 oraz 4.

Jak można zauważyć na wykresie 3, przepustowość w przypadku pojedynczej strony użytkownika dla aplikacji monolitycznej przy zwiększonym ruchu jest wyższa niżeli w poprzednim teście. Wynika to prawdopodobnie z lepszej wydajności konkretnego zapytania w porównaniu z innymi, które negatywnie wpływały na wydajność aplikacji monolitycznej w pierwszym teście. Warto również zauważyć, że w tym teście liczba replik aplikacji mikroserwisowych również nie miała wpływu na przepustowość.



Wykres 3. Wykres przedstawiający przepustowość aplikacji w stosunku do liczby użytkowników dla strony pojedynczej oferty.

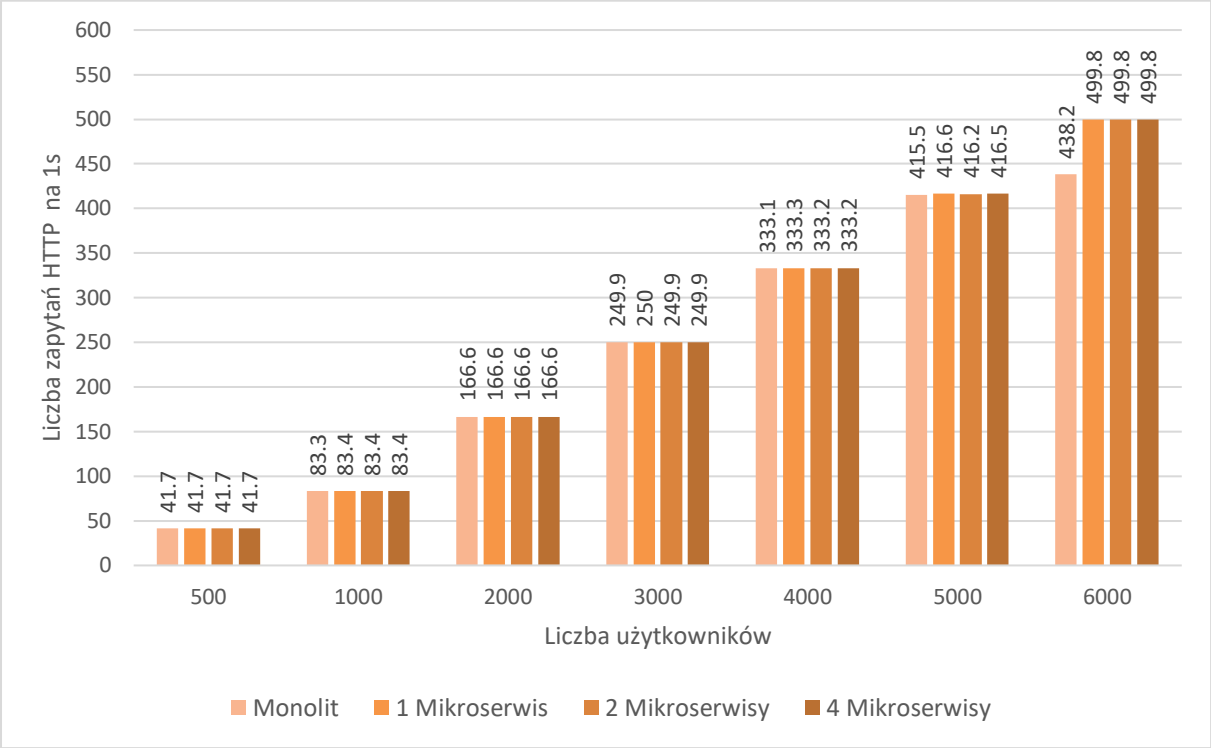


Wykres 4. Wykres przedstawiający średni czas oczekiwania na załadowanie strony pojedynczej oferty w stosunku do liczby użytkowników.

W przypadku tego testu korelacja pomiędzy przepustowością, a średnim czasem oczekiwania również została zachowana, ponieważ wydajność aplikacji monolitycznej spada

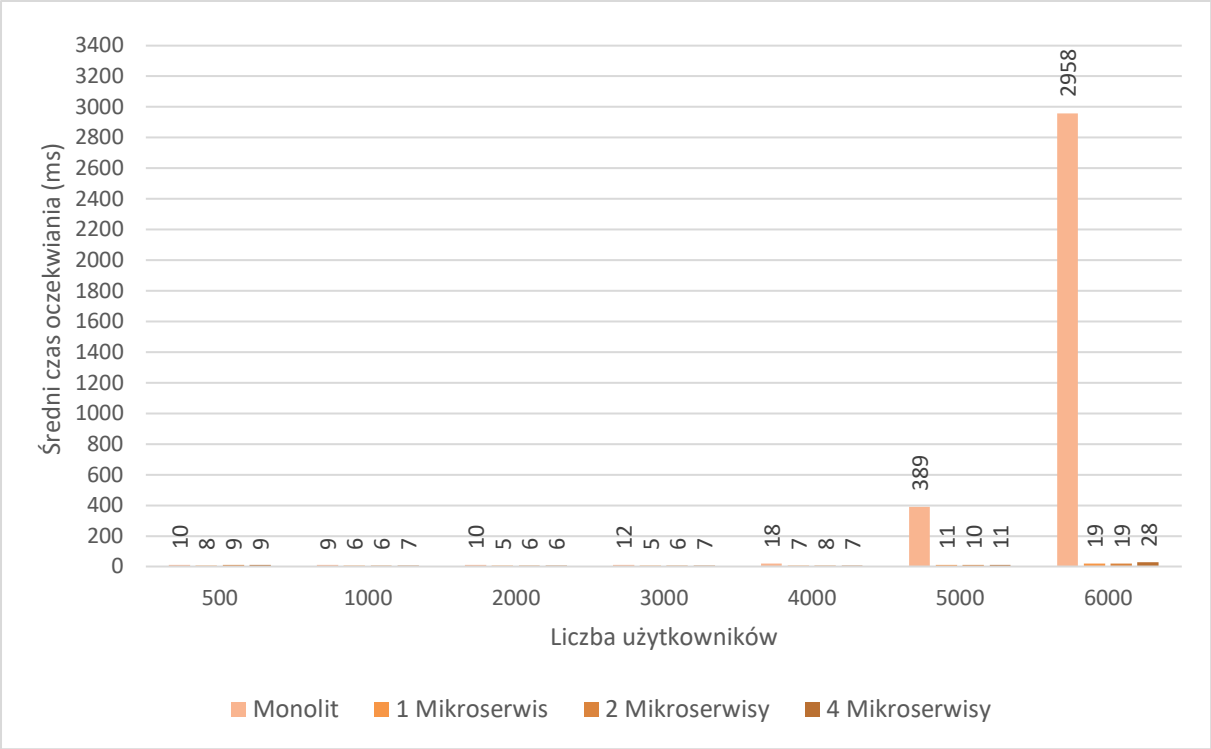
wraz ze wzrostem obciążenia. Jak można zaobserwować na wykresie 4, wydajność aplikacji monolitycznych pozostaje praktycznie na tym samym poziomie niezależnie od liczby zapytań. Średni czas oczekiwania na zapytanie dla 6000 użytkowników dla aplikacji monolitycznej sięga 6351 ms, co oznacza, że jest ona 423 razy mniej wydajna w porównaniu do aplikacji mikroserwisowych.

Ostatni scenariusz obejmuję proces logowania użytkownika w systemie, który odbywa się za pomocą zasobu POST. Proces logowania obejmuję zarówno operację odczytu jak i zapisu do bazy danych. Otrzymane rezultaty zostały przedstawiony na wykresie 5 oraz 6.



Wykres 5. Wykres przedstawiający przepustowość aplikacji w stosunku do liczby użytkowników dla procesu logowania.

Z wykresu 5 można odczytać, że przepustowość aplikacji mikroserwisowych niezależnie od poziomu obciążenia osiągnęła maksymalny pułap tak jak w dwóch poprzednich testach. Przepustowość aplikacji monolitycznej zmniejszyła się o 1 zapytanie na sekundę przy obciążeniu 5000 użytkowników oraz o 61 zapytań na sekundę przy obciążeniu 6000 użytkowników. Spadek ten jest również widoczny na wykresie 6 przedstawiającym średni czas oczekiwania, gdzie aplikacja monolityczna wykazuję o wiele dłuższy czas oczekiwania na odpowiedź. Dla aplikacji mikroserwisowych czas oczekiwania jest stabilny niezależnie od obciążenia.



Wykres 6. Wykres przedstawiający średni czas trwania procesu logowania w stosunku do liczby użytkowników.

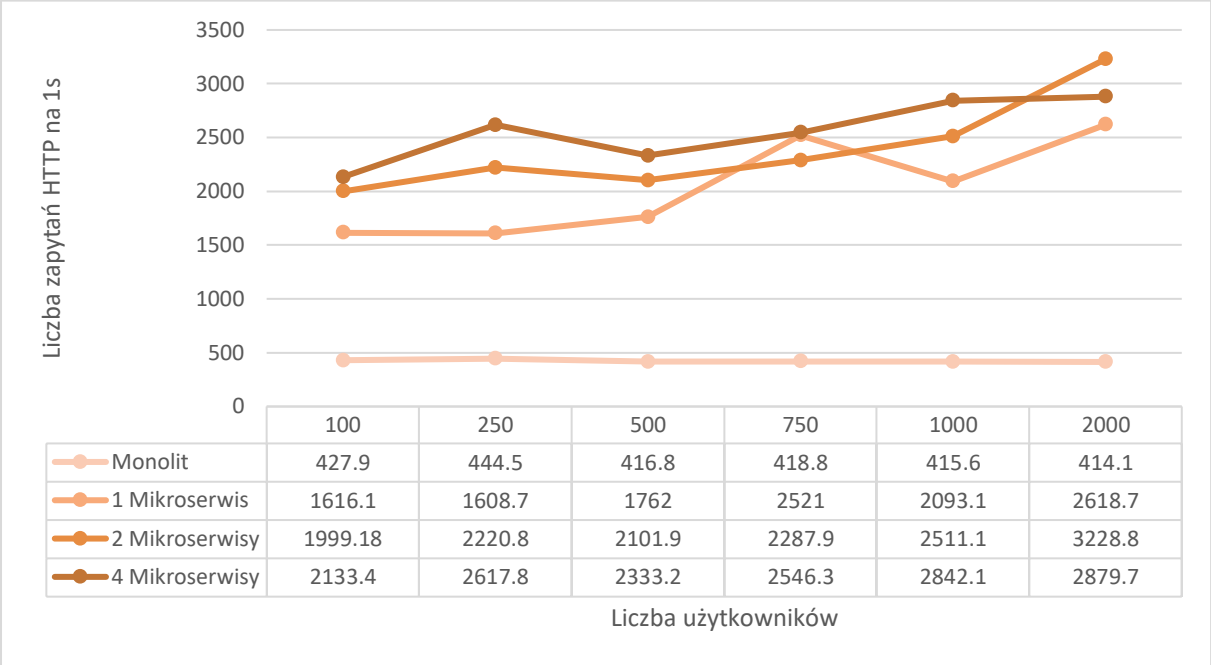
6.1.2 Testy pod zmiennym obciążeniem

Drugim typem przeprowadzonych testów obciążeniowych są testy pod zmiennym obciążeniem, w których liczba wysyłanych do serwera żądań HTTP nie jest z góry określona, a liczba użytkowników stale rośnie. W trakcie testu liczba użytkowników zwiększa się przez 30 sekund do ustalonego limitu (np. 100), po czym przez kolejne 30 sekund utrzymuje się na maksymalnym poziomie, generując maksymalną możliwą liczbę zapytań na jaką pozwala serwer. Następnie liczba wątków znowu wzrasta do osiągniecia kolejnego pułapu.

Konfiguracja JMeter wykorzystana do testów pod zmiennym obciążeniem prezentuje się następująco:

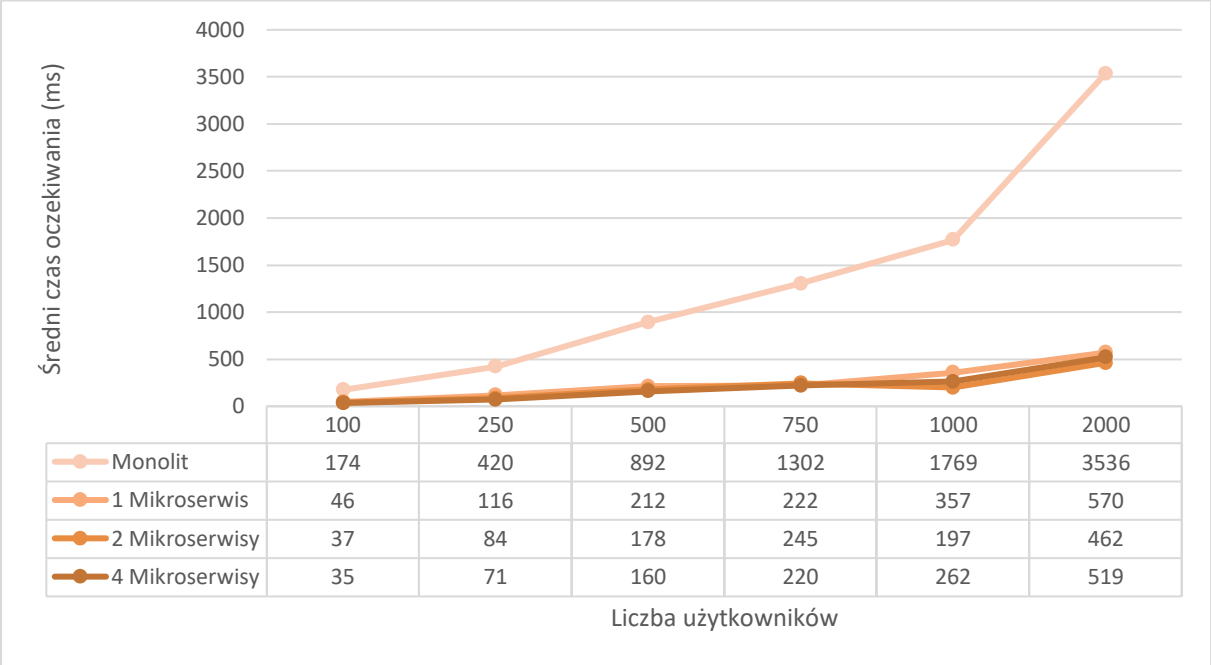
- Liczba użytkowników (wątków): 100 – 2000,
- Czas trwania: 30s,
- Liczba iteracji: nieskończoność,
- Czas życia wątku: 60s.

Pierwszy scenariusz weryfikuję wydajność aplikacji podczas zapytania GET. Do testu wykorzystany zostały zasób potrzebny do załadowania danych na pojedynczą stronę ofert zleceń. Uzyskane wyniki zostały zaprezentowane na wykresach 7 oraz 8.



	100	250	500	750	1000	2000
Monolit	427.9	444.5	416.8	418.8	415.6	414.1
1 Mikroserwis	1616.1	1608.7	1762	2521	2093.1	2618.7
2 Mikroserwisy	1999.18	2220.8	2101.9	2287.9	2511.1	3228.8
4 Mikroserwisy	2133.4	2617.8	2333.2	2546.3	2842.1	2879.7

Wykres 7. Wykres przedstawiający przepustowość aplikacji w stosunku do liczby użytkowników dla strony pojedynczej oferty.



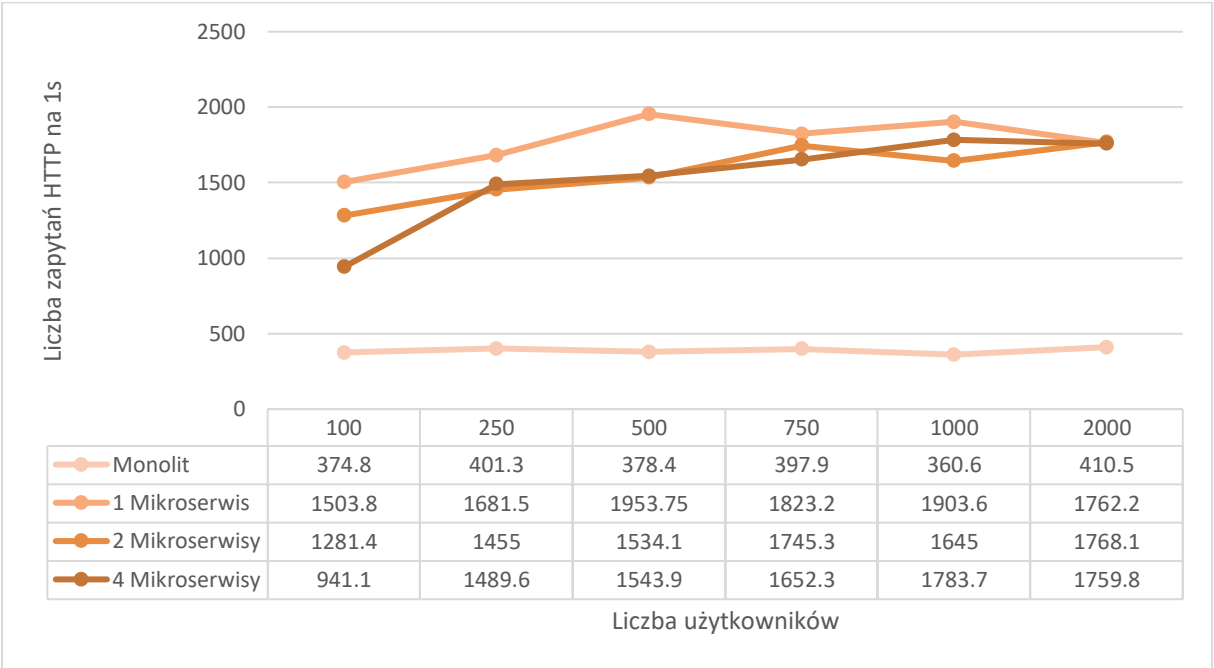
	100	250	500	750	1000	2000
Monolit	174	420	892	1302	1769	3536
1 Mikroserwis	46	116	212	222	357	570
2 Mikroserwisy	37	84	178	245	197	462
4 Mikroserwisy	35	71	160	220	262	519

Wykres 8. Wykres przedstawiający średni czas oczekiwania na załadowanie strony pojedynczej oferty w stosunku do liczby użytkowników.

Z analizy powyższych wykresów wynika, że zwiększenie ruchu nie powoduję spadku przepustowości, a nawet powoduje jej wzrost w przypadku aplikacji mikroserwisowych. Można również zauważyć, że zwiększenie liczby replik mikrousługi wpływa na zwiększenie przepustowość aplikacji, natomiast wyniki są dosyć porównywalne i nie wpływają istotnie na doświadczenie użytkownika. Średni czas oczekiwania na odpowiedź nie różni się znacząco pomiędzy konfiguracjami, a ewentualne opóźnienia są minimalne. Jak przewidywano, czas oczekiwania wzrasta wraz z liczbą użytkowników, aczkolwiek wyniki są akceptowalne.

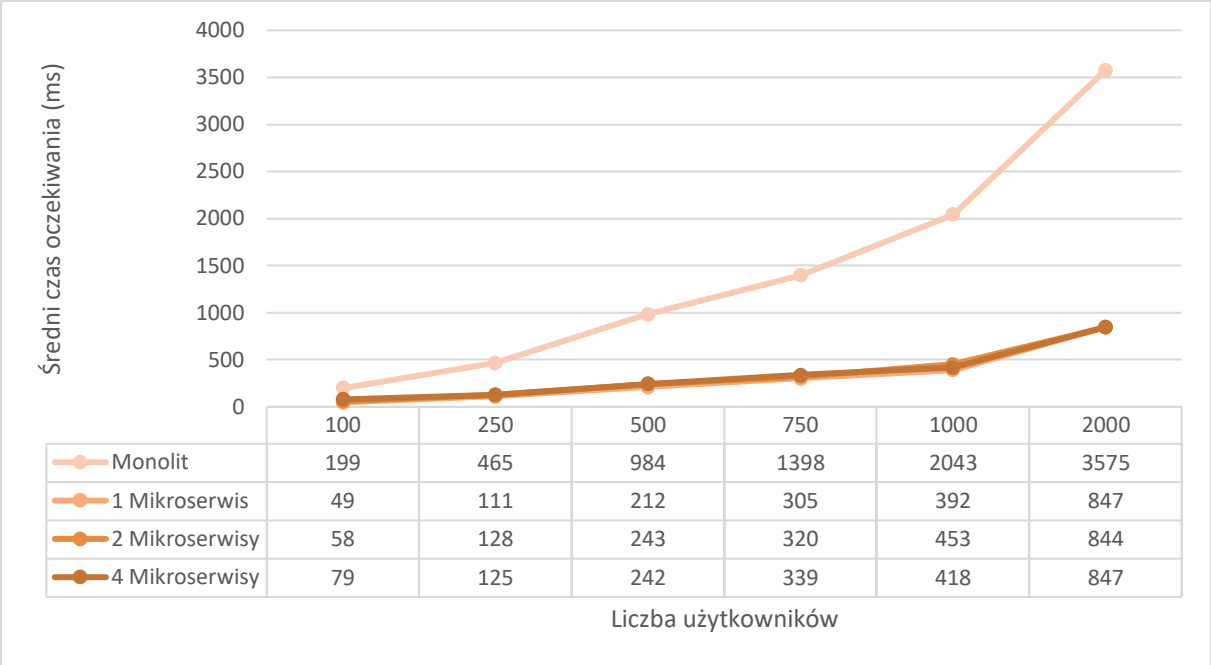
Przechodząc do aplikacji monolitycznej, można zauważyć, że nie jest ona w stanie obsłużyć więcej niż 444.5 zapytań na sekundę. Przepustowość aplikacji monolitycznej spada wraz ze wzrostem obciążenia. Średni czas oczekiwania dla aplikacji monolitycznej jest o wiele gorszy od aplikacji mikroserwisowej niezależnie od obciążenia. W sytuacji zwiększonej liczby użytkowników, wydłużony czas odpowiedzi może prowadzić do odczuwalnego dyskomfortu w korzystaniu z aplikacji przez użytkowników.

Drugi scenariusz sprawdza wydajność aplikacji podczas zapytania POST. Podczas tego testu sprawdzono wydajność procesu logowania w systemie, wykorzystując zasób generujący token dostępu. Uzyskane wyniki zostały zaprezentowane na wykresach 9 oraz 10.



	100	250	500	750	1000	2000
Monolit	374.8	401.3	378.4	397.9	360.6	410.5
1 Mikroserwis	1503.8	1681.5	1953.75	1823.2	1903.6	1762.2
2 Mikroserwisy	1281.4	1455	1534.1	1745.3	1645	1768.1
4 Mikroserwisy	941.1	1489.6	1543.9	1652.3	1783.7	1759.8

Wykres 9. Wykres przedstawiający przepustowość aplikacji w stosunku do liczby użytkowników dla procesu logowania.



	100	250	500	750	1000	2000
Monolit	199	465	984	1398	2043	3575
1 Mikroserwis	49	111	212	305	392	847
2 Mikroserwisy	58	128	243	320	453	844
4 Mikroserwisy	79	125	242	339	418	847

Wykres 10. Wykres przedstawiający średni czas trwania procesu logowania w stosunku do liczby użytkowników.

Można zauważyć, że również podczas testów zasobu POST aplikacja monolityczna prezentuję gorszą wydajność niż aplikacja mikroserwisowa. Przepustowość monolitu oscyluje w granicach 400 zapytań na sekundę i nie wykazuje istotnej korelacji z liczbą użytkowników. Średni czas odpowiedzi jest również znacznie dłuższy niż w przypadku aplikacji mikroserwisowych.

W kontekście aplikacji mikroserwisowych, test zasobu POST również nie wykazał istotnej korelacji między przepustowością, a liczbą użytkowników. Przepustowość utrzymuje się na podobnym poziomie we wszystkich konfiguracjach. Można również zauważyć, że maksymalna przepustowość dla zasobu POST jest niższa, niż w poprzednim teście. Wpływ na to ma zapewne czas oczekiwania na odpowiedź, który również jest wyższy niż w poprzednim scenariuszu. Czas oczekiwania w różnych konfiguracjach mikroserwisowych jest jednak zbliżony i nie zaobserwowano żadnych istotnych anomalii.

6.2 Testy przeciążeniowe

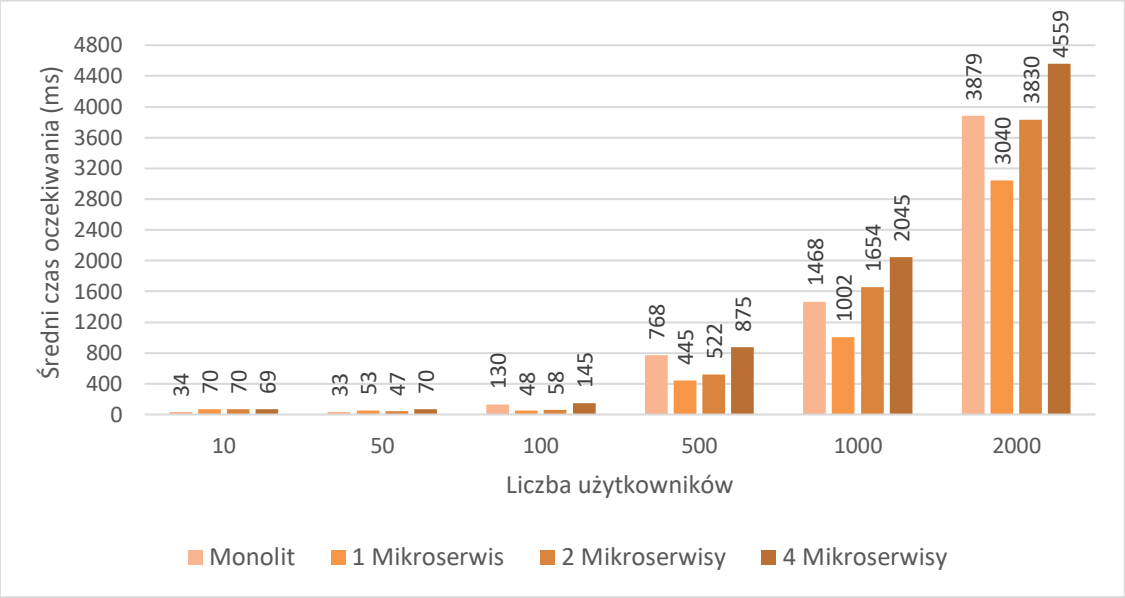
Testy przeciążeniowe mają na celu symulację zwiększonego ruchu w aplikacji, aby ocenić jej wydajność i stabilność w warunkach intensywnego użytkowania. Podczas tych testów analizuje się, jak aplikacja radzi sobie, gdy funkcjonalność jest równocześnie wykorzystywana przez dużą liczbę użytkowników. W ramach przeprowadzonych testów zmierzono czas przetwarzania żądań przy obciążeniu aplikacji od 10 do 2000 użytkowników na sekundę.

Konfiguracja JMeter wykorzystana do testów przeciążeniowych prezentuje się następująco:

- Liczba użytkowników (wątków): 10 – 2000,
- Czas trwania: 1s,
- Liczba iteracji: 1.

6.2.1 Operacje odczytu danych

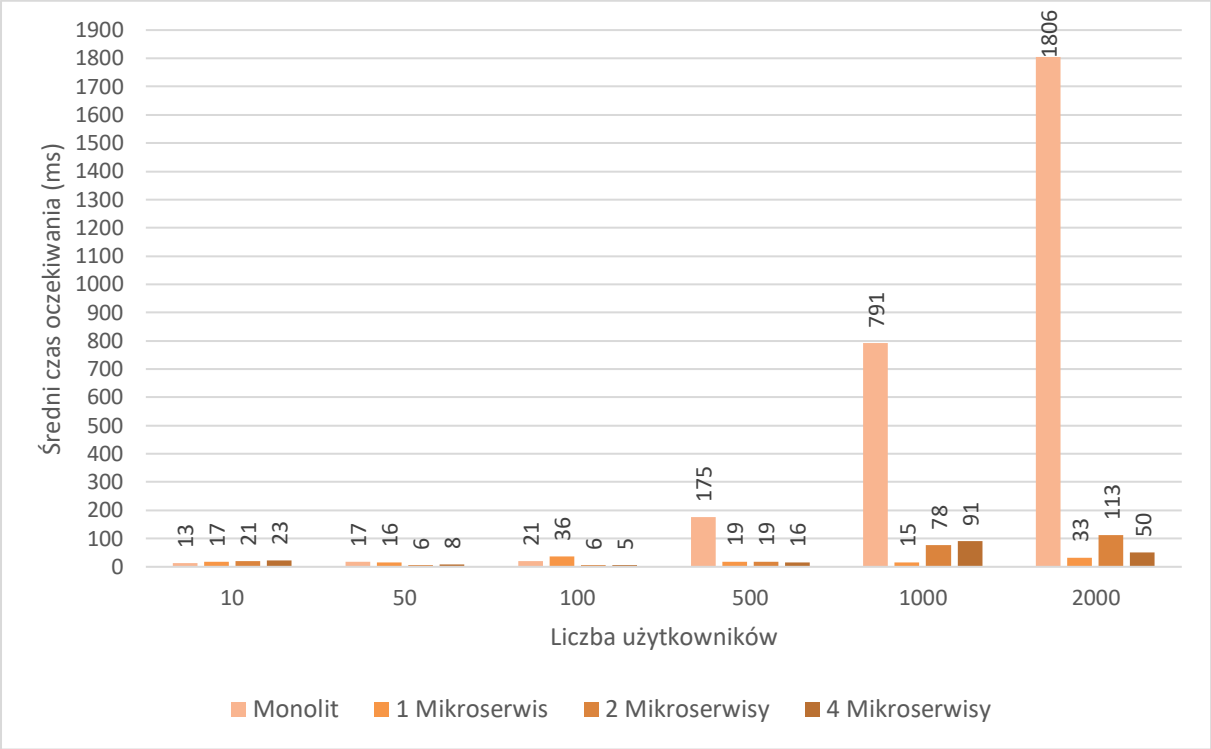
Na wykresie 1 przedstawione zostały uśrednione wyniki czasu ładowania wszystkich zasobów GET wykorzystywanych na stronie profilu użytkownika zleceniodawcy. W obu architekturach wywoływanych jest pięć zasobów GET w celu pobrania niezbędnych danych. W przypadku aplikacji mikroserwisowej dane pobierane są z trzech mikroserwisów: profilu użytkownika, opinii oraz ofert zleceń. Cała operacja nie wymaga uwierzytelnienia. Wyniki zostały zaprezentowane na wykresie 11.



Wykres 11. Wykres przedstawiający średni czas oczekiwania na załadowanie danych na stronie profilu użytkownika w stosunku do liczby użytkowników odwiedzających ją w czasie 1 sekundy.

Na podstawie analizy wykresu można stwierdzić, że wydajność wszystkich aplikacji jest zbliżona przy obciążeniu 10 i 50 użytkowników na sekundę. Jednakże, począwszy od obciążenia na poziomie 100 użytkowników na sekundę, konfiguracja z pojedynczymi replikami zaczyna wykazywać lepsze wyniki w porównaniu z aplikacją monolityczną, aczkolwiek prezentowane różnice nie są ogromne. Z kolei aplikacja monolityczna osiąga lepsze rezultaty niż aplikacja korzystająca z większej liczby instancji usług. Dodatkowo, zauważalny jest trend, że wraz ze wzrostem liczby równocześnie działających instancji, czas oczekiwania na załadowanie danych ulega wydłużeniu.

Kolejnym elementem objętym testami była strona pojedynczej oferty zleceń. W przypadku wersji mikroserwisowej, interfejs użytkownika integruje się wyłącznie z usługą ofert zleceń w celu pobrania niezbędnych danych do załadowania strony. Cała operacja odbywa się bez konieczności uwierzytelnienia. W obu wersjach aplikacji wywoływany jest jeden zasób GET. Wyniki uzyskane podczas tego eksperymentu zostały przedstawione na wykresie 12.

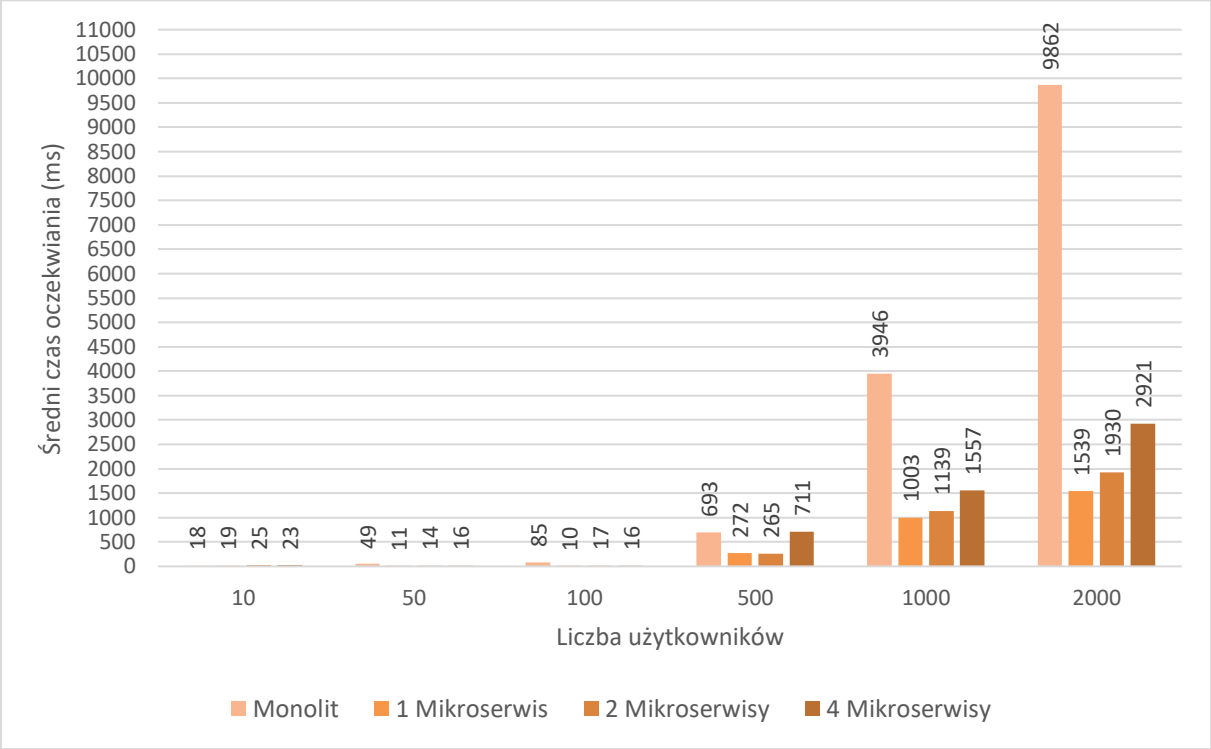


Wykres 12. Wykres przedstawiający średni czas oczekiwania na załadowanie danych na stronie oferty zleceń w stosunku do liczby użytkowników odwiedzających ją w czasie 1 sekundy.

Z wykresu można odczytać, że czas oczekiwania za równo dla aplikacji monolitycznej jak i mikroserwisowej jest porównywalny przy obciążeniu do 100 użytkowników na sekundę. Znaczący spadek wydajności aplikacji monolitycznej jest zauważalny przy obciążeniu na poziomie 500 użytkowników na sekundę, a dalszy wzrost obciążenia powoduje dalsze

pogorszenie wyników. W tym przypadku aplikacja mikroserwisowa wypada lepiej w każdej konfiguracji, zachowując stabilność nawet przy wzmożonym ruchu wynoszącym 2000 użytkowników na sekundę. Warto jednak zauważyć, że również w tym teście zaobserwowano tendencję do wydłużania się czasu oczekiwania na odpowiedź wraz ze wzrostem liczby instancji w aplikacji mikroserwisowej.

Następnie testom obciążeniowym została poddana pierwsza strona z ogłoszeniami ofert zleceń. W skład niej wchodzą wszystkie dane statyczne wykorzystywane do załadowania filtrów m.in. adresy, technologie, rodzaje ofert oraz pierwsza strona danych z ofertami zleceń. W wersji w architekturze mikroserwisowej interakcja zachodziła z usługą ofert zleceń oraz danych statycznych. W obu przypadkach zostały wywołane cztery zasoby GET. Ten proces również nie wymagał uwierzytelniania. Wyniki zostały zaprezentowane na wykresie 13.

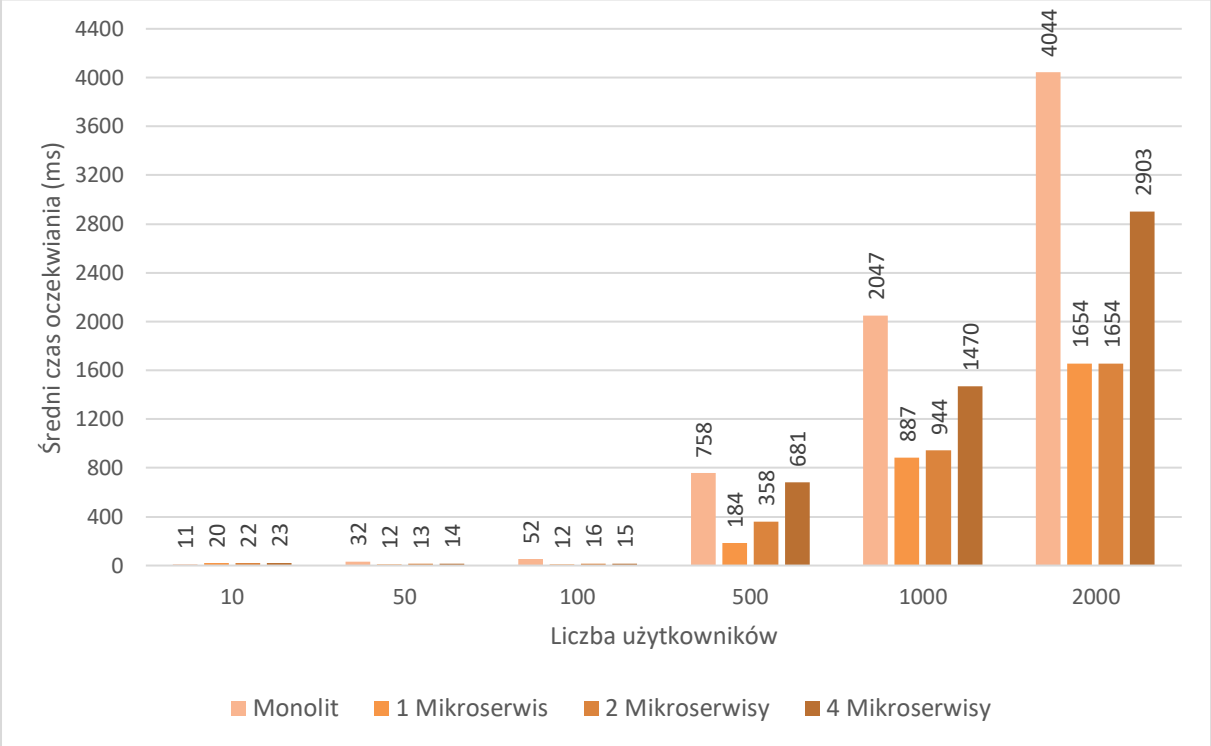


Wykres 13. Wykres przedstawiający średni czas oczekiwania na załadowanie danych na pierwszej stronie z ogłoszeniami ofert zleceń w stosunku do liczby użytkowników odwiedzających ją w czasie 1 sekundy.

Jak można zauważyć wydajność aplikacji mikroserwisowej jest znacząco lepsza od aplikacji monolitycznej, co potwierdzają również wyniki uzyskane przy większym obciążeniu, gdzie aplikacja mikroserwisowa w każdej konfiguracji radzi sobie lepiej. Ponadto w przypadku 1000 użytkowników na sekundę 6.49% zapytań z przesłanych 4000 nie zostało obsłużonych przez aplikacje monolityczną z powodu przekroczenia czasu oczekiwania, a przy 2000 użytkowników odsetek ten wzrósł do 32.23% przy 8000 zapytań. Eksperyment tak samo jak

poprzednie wskazał dodatnią korelację pomiędzy czasem oczekiwania na odpowiedź, a liczbą działających usług.

Kolejną przetestowaną stroną jest pierwsza strona z ogłoszeniami programistów. Strona tak samo jak strona ofert zleceń pobiera cztery zasoby GET, z tym, że zamiast interakcji z usługą ofert zleceń i danych statycznych, pobiera dane z usługi profilu użytkownika i danych statycznych. Proces nie wymaga uwierzytelniania. Osiągnięte wyniki zostały przedstawione na poniższym wykresie 14.

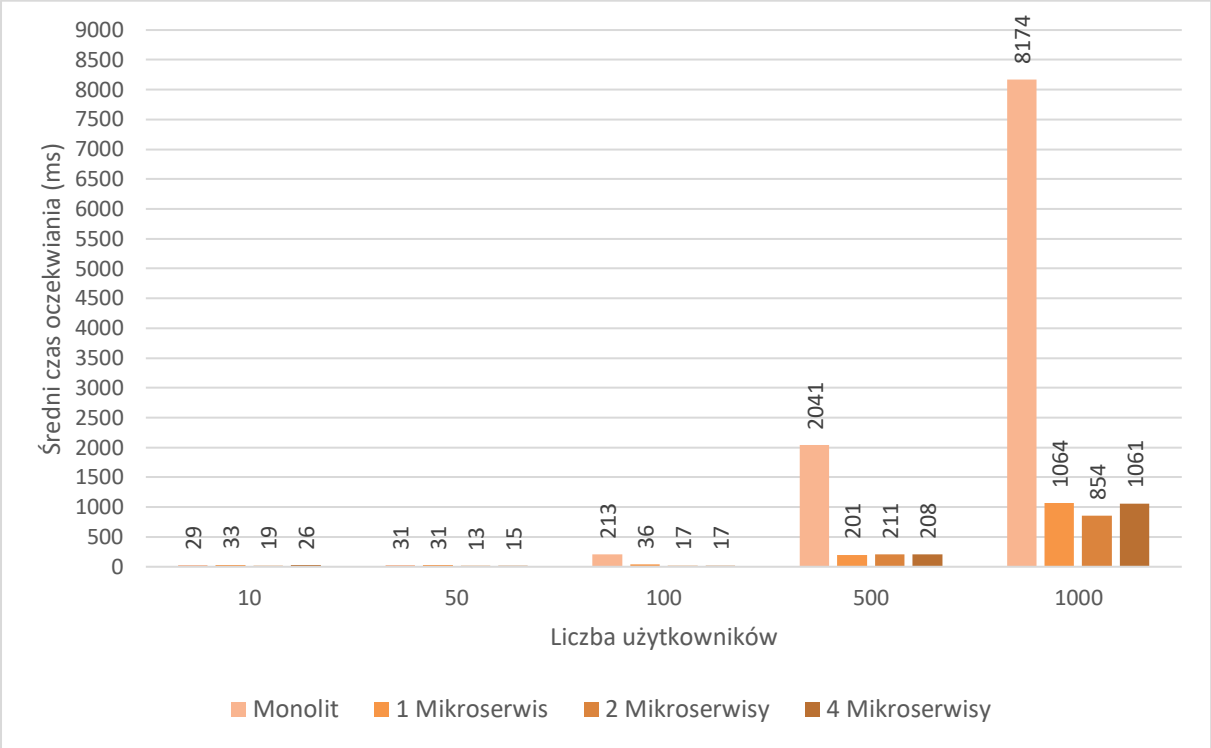


Wykres 14. Wykres przedstawiający średni czas oczekiwania na załadowanie danych na pierwszej stronie z ogłoszeniami programistów w stosunku do liczby użytkowników odwiedzających ją w czasie 1 sekundy.

Na podstawie wykresu można zauważyć, że wyniki dla obu architektur do 100 użytkowników na sekundę są na bardzo podobnym poziomie i nie odbiegają znacząco od siebie. Średni czas oczekiwania na dane dla tego obciążenia wynosi 21ms. Różnica w wydajności staje się widoczna dopiero przy obciążeniu na poziomie 500 użytkowników na sekundę, gdzie czas oczekiwania na dane w aplikacji mikroserwisowej z pojedynczymi replikami wynosi 184 ms, podczas gdy dla aplikacji monolitycznej 758 ms. Dodatkowo, przy obciążeniu 2000 użytkowników na sekundę, 20.33% zapytań z 8000 w aplikacji monolitycznej nie zostało przetworzonych poprawnie. Wykres prezentuję podobny czas oczekiwania dla konfiguracji z pojedynczymi oraz podwójnymi replikacji przy obciążeniu 1000 oraz 2000 użytkowników na sekundę. Natomiast w przypadku tego eksperymentu tendencja do wydłużania się czasu

oczekiwania na odpowiedź wraz ze wzrostem liczby instancji jest również widoczna, co można zaobserwować na podstawie wyników dla konfiguracji z czterema replikami.

Kolejnym elementem objętym testami była strona wyszukiwarki, odpowiadająca ze przeszukiwanie ofert zleceń oraz profili użytkowników na podstawie podanej frazy. Do pobrania danych interfejs użytkownika odwołuję się do dwóch zapytań GET. W przypadku aplikacji mikroserwisowej interakcja zachodzi z dwoma usługami, usługą ofert zleceń oraz profilu użytkownika. Proces nie wymaga uwierzytelnienia. Wyniki zostały zaprezentowane na wykresie 15.



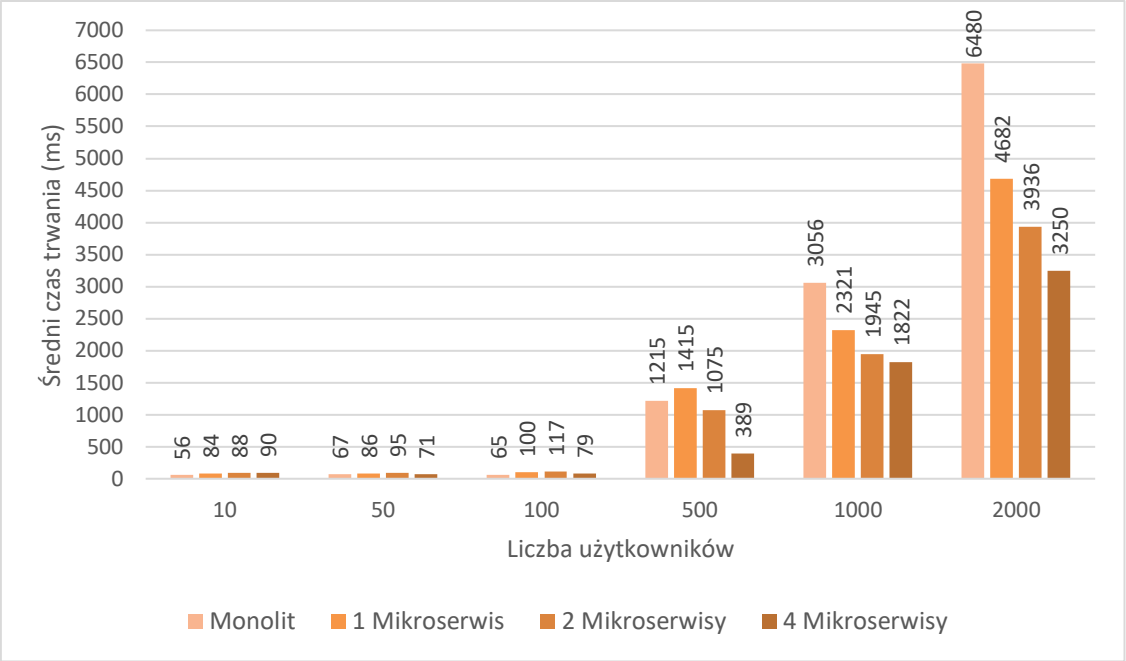
Wykres 15. Wykres przedstawiający średni czas oczekiwania na załadowanie danych na stronie wyszukiwarki w stosunku do liczby użytkowników odwiedzających ją w czasie 1 sekundy.

Analiza wykresu wskazuje na brak istotnych różnic w czasach odpowiedzi pomiędzy aplikacjami w obu architekturach przy obciążeniu do 50 użytkowników na sekundę. Przy obciążeniu 100 użytkowników na sekundę czas oczekiwania w aplikacji monolitycznej wzrasta do 213 ms, podczas gdy dla aplikacji mikroserwisowej pozostaje na stabilnym poziomie: 36 ms dla jednej instancji mikroserwisu, 13 ms dla dwóch instancji oraz 15 ms dla czterech instancji. Zależność podobieństwa wyników różnych konfiguracji aplikacji mikroserwisowej utrzymuję się do 1000 użytkowników na sekundę. Natomiast wydajność aplikacji monolitycznej nieporównywalnie spada osiągając czas oczekiwania na odpowiedź na poziomie 8500 ms przy obciążeniu 1000 użytkowników na sekundę Warto podkreślić, że za operacje wyszukiwania

odpowiedzialny jest Elasticsearch, który w architekturze mikroserwisowej posiada dwa indeksy rozdzielone pomiędzy różne instancje, podczas gdy w aplikacji monolitycznej oba indeksy znajdują się w jednej instancji, co może znacząco wpływać na wyniki testów. Należy również dodać, że próba przeprowadzenia testu dla 2000 użytkowników na sekundę w obu architekturach zakończyła się niepowodzeniem, prawdopodobnie z powodu zbyt dużego obciążenia Elasticsearcha przy domyślnej konfiguracji. Można również zauważyć, że w tym eksperymencie nie występuję dodatnia korelacja pomiędzy ilością instancji, a czasem oczekiwania, co może być związane z brakiem pobierania danych z bazy danych.

6.2.2 Operacje zapisu danych

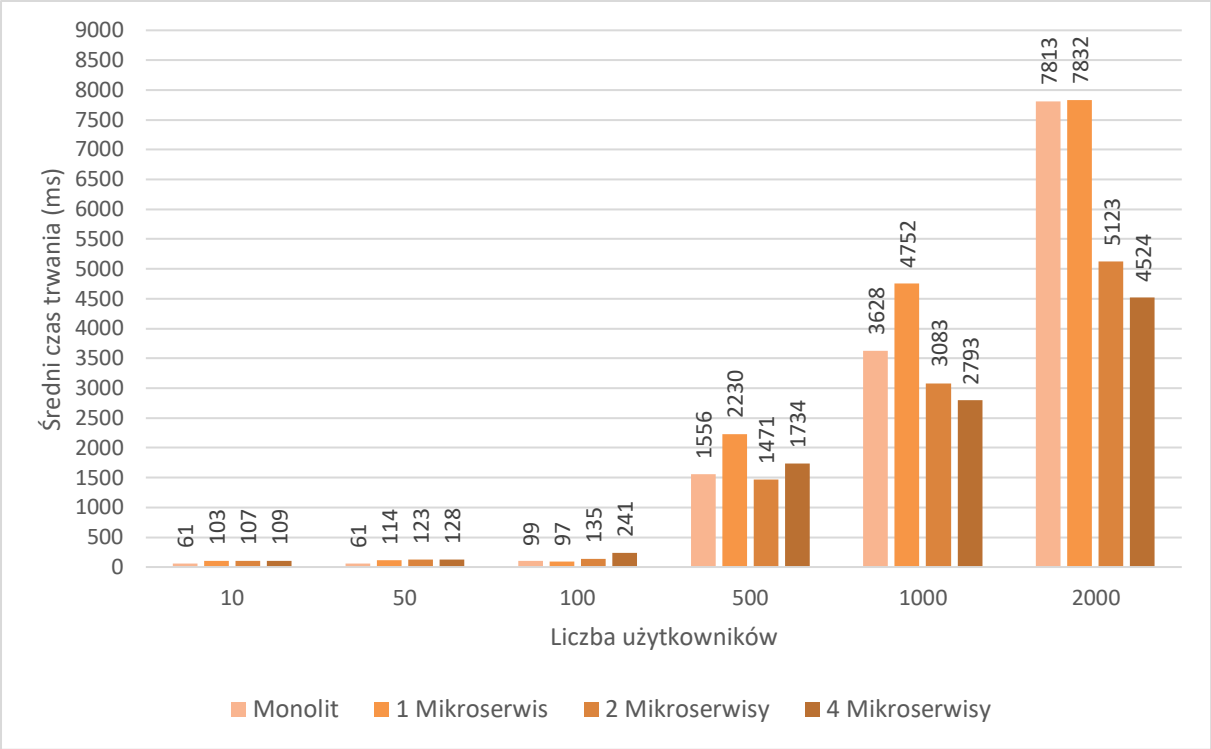
W kolejnych testach skupiono się na operacjach zapisu danych, rozpoczynając od publikacji nowego zlecenia. Aby opublikować nowe zlecenie, użytkownik musi być zalogowany oraz posiadać rolę zleceniodawcy, co wymaga przeprowadzenia procesu uwierzytelniania i autoryzacji. W trakcie testu każdy użytkownik loguje się do systemu w celu wygenerowania tokenu, który następnie jest przekazywany w nagłówku kolejnego żądania publikującego nowe zlecenie. Warto zaznaczyć, że w każdej iteracji testu wykorzystano innego użytkownika. W przypadku aplikacji monolitycznej wykonano żądania do dwóch zasobów POST, natomiast aplikacja mikroserwisowa angażuje trzy zasoby, z których jedno jest zapytaniem wewnętrznym pomiędzy bramą API a usługą uwierzytelniającą, co szczegółowo opisano w rozdziale dotyczącym aplikacji mikroserwisowej. Ostatecznie, proces w architekturze mikroserwisowej wymaga interakcji zarówno z usługą uwierzytelniania, jak i z usługą ofert zleceń. Otrzymane wyniki zostały zaprezentowane na poniższym wykresie 16.



Wykres 16. Wykres przedstawiający średni czas trwania procesu publikacji nowego zlecenia w stosunku do liczby użytkowników wykonujących tą czynność w czasie 1 sekundy.

Można zauważyć, że aplikacja monolityczna wykazuje lepszą wydajność w porównaniu ze wszystkimi konfiguracjami aplikacji mikroserwisowej przy obciążeniu do 100 użytkowników na sekundę. Prawdopodobnym czynnikiem wpływającym na to jest dodatkowe zapytanie w architekturze mikroserwisowej, które opóźnia cały proces. Jednakże, przy większym obciążeniu, aplikacje mikroserwisowe wykazują mniejsze opóźnienia. Zwiększenie liczby instancji usług w architekturze mikroserwisowej również znacząco poprawia wydajność aplikacji, co widać na przykładzie obciążenia 500 użytkowników na sekundę, gdzie czas odpowiedzi dla konfiguracji z jedną repliką wynosi 1415 ms, natomiast dla czterech replik spada do 389 ms.

Kolejnym elementem, który został przetestowany jest proces aplikacji na zlecenie. Podobnie jak w poprzednim przypadku, proces ten wymaga uwierzytelnienia i autoryzacji. Każda iteracja testu była przeprowadzana na innym użytkowniku, posiadającym wyłącznie rolę programisty. Oferty, na które aplikowali użytkownicy, również były inne dla każdego użytkownika. W przypadku aplikacji monolitycznej wykonano żądania do dwóch zasobów POST, natomiast w przypadku aplikacji mikroserwisowej wykonano trzy żądania do zasobów POST. Test ten obejmował interakcję z usługą ofert zleceń, usługą uwierzytelniającą oraz usługą powiadomień, przy czym interakcja z usługą powiadomień odbywała się w sposób asynchroniczny. Uśrednione wyniki uzyskane z przeprowadzonego badania zostały przedstawione na wykresie 17.

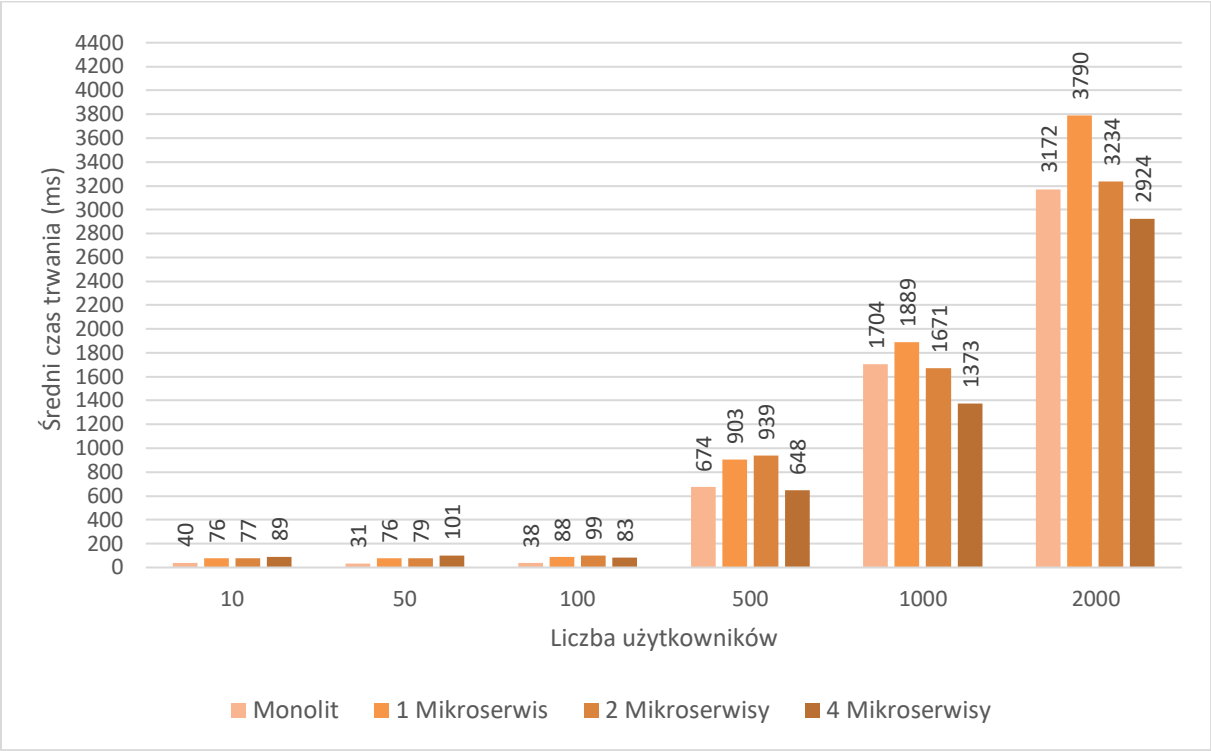


Wykres 17. Wykres przedstawiający średni czas trwania procesu aplikacji na zlecenie w stosunku do liczby użytkowników wykonujących tą czynność w czasie 1 sekundy.

Można zauważyć, że aplikacja monolityczna wykazuje krótszy czas oczekiwania na odpowiedź przy obciążeniu do 100 użytkowników na sekundę. Przy obciążeniu wynoszącym 500 użytkowników na sekundę wyniki są zbliżone do tych osiąganych przez aplikację mikroserwisową z dwoma replikami usług. Przy wyższych obciążeniach znacznie lepiej radzą sobie aplikacje mikroserwisowe z dwoma lub trzema replikami, przy czym ta druga konfiguracja osiąga nieco lepsze rezultaty. Warto podkreślić, że aplikacja mikroserwisowa z pojedynczymi instancjami wypada gorzej od monolitycznej w każdym z przeprowadzonych testów. W trakcie tego badania skalowanie poziome mikroserwisów pokazało, że przy obciążeniu wynoszącym 1000 użytkowników na sekundę system jest w stanie obsłużyć większą liczbę użytkowników przy mniejszym opóźnieniu.

Ostatnim elementem, który został poddany analizie jest proces rejestracji użytkownika w systemie. W obu aplikacjach zastosowano taką samą funkcję szyfrującą hasło użytkownika, w celu uniknięcia różnic wynikających z czasu szyfrowania. Rejestracja w aplikacji monolitycznej obejmuje jedno żądanie do zasobu POST. W przypadku aplikacji mikroserwisowej jest to żądanie POST do usługi profilu użytkownika i wewnętrzne żądanie POST do usługi uwierzytelniającej oraz dodatkowa komunikacja asynchroniczna z usługą

uwierzytelniania oraz powiadomień. Otrzymane rezultaty zostały przedstawione na poniższym wykresie 18.



Wykres 18. Wykres przedstawiający średni czas trwania procesu rejestracji użytkownika w stosunku do liczby użytkowników wykonujących tą czynność w czasie 1 sekundy.

Z wykresu wynika, że przy obciążeniu 10, 50 oraz 100 użytkowników na sekundę aplikacja monolityczna osiąga najmniejsze opóźnienia. Przy obciążeniu 500 użytkowników na sekundę aplikacja monolityczna uzyskuje wynik 674 ms, zbliżony do aplikacji mikroserwisowej z czterema instancjami, która osiągnęła 648 ms. Dla obciążenia 1000 i 2000 użytkowników na sekundę, konfiguracja z czterema replikami mikroserwisów osiąga najniższe opóźnienia. Natomiast pozostałe wersje mikroserwisów prezentują gorszy lub zbliżony poziom od aplikacji monolitycznej w każdym z testów. Przy obciążeniu 2000 użytkowników na sekundę, 26,70% zapytań do aplikacji monolitycznej zakończyło się błędem.

6.3 Wnioski

Z przeprowadzonych testów pod stałym obciążeniem wynika, że obie aplikacje prezentują bardzo dobrą wydajność dla założonego w wymaganiach niefunkcjonalnych obciążenia, a nawet przewyższają oczekiwane rezultaty. Obie aplikacje prezentowały przepustowość oraz opóźnienia na podobnym poziomie do momentu 15000/20000 zapytań na minutę. Natomiast przy wyższym obciążeniu, każdy z testów jednoznacznie wskazał, że aplikacja w architekturze

mikroserwisowej charakteryzuje się lepszą wydajnością i jest w stanie obsłużyć większy ruch nie zwiększając przy tym czasu oczekiwania na odpowiedź.

W testach przeprowadzonych pod zmiennym obciążeniem, gdzie aplikacje były testowane w warunkach stopniowego wzrostu obciążenia, symulując bardziej realistyczne scenariusze użytkowania, zaobserwowano znaczną różnicę w wydajności na korzyść architektury mikroserwisowej. Aplikacja mikroserwisowa była w stanie obsłużyć większą liczbę zapytań i wykazywała niższe opóźnienia w porównaniu do aplikacji monolitycznej.

Test przeciążeniowe wykazały, że przy gwałtownym wzroście obciążenia aplikacja mikroserwisowa w większości sytuacji radzi sobie lepiej od aplikacji monolitycznej. Różnice mogły być zaobserwowane przy poziomie 500 użytkowników na sekundę, od którego to momentu wydajność aplikacji monolitycznej drastycznie spadała. Podczas tych testów zauważono również, że w przypadku operacji odczytu, wraz ze wzrostem liczby instancji mikrousług, wydajność aplikacji spadała, natomiast w przypadku operacji zapisu wydajność rosła. Spadek wydajności w przypadku operacji odczytu może być przyczyną uruchomienia wszystkich instancji na jednym serwerze oraz korzystania z jednej bazy danych i pamięci cache przez wszystkie instancje, powodując kolizje zasobów.

Podsumowując przeprowadzony eksperyment potwierdza, że aplikacje w obu architekturach spełniają wymaganie niefunkcjonalne, założone w projekcie inżynieryjnym [63] o poprawnej obsłudze do 1000 użytkowników na minutę. Wszystkie rodzaje przeprowadzonych testów wykazały wyższość rozwiązania opartego na architekturze mikroserwisowej. Według badań, aplikacja mikroserwisowa jest wstanie obsłużyć znacznie więcej zapytań od aplikacji monolitycznej, bez względu na liczbę replik. Aplikacja mikroserwisowa poprawia również wydajność całego systemu przy wyższym obciążeniu zachowując niższe czasy oczekiwania na odpowiedź.

Wyniki porównania mogą nie być w pełni miarodajne, ze względu na to, że wszystkie mikrousługi zostały uruchomione na jednej, lokalnej maszynie, co spowodowało znaczne obciążenie serwera. W rezultacie, próby skalowania dodatkowo zwiększały obciążenie, przez co skalowanie mogło nie przynieść oczekiwanych rezultatów. Wpływ na otrzymane wyniki mogła mieć również zmiana stosu technologicznego z PHP na języki JVM. Dodatkowo domyślna konfiguracja serwera Nginx oraz PHP mogła przyczynić się do słabszych rezultatów aplikacji monolitycznej.

7 Podsumowanie

Celem pracy było przeprowadzenie analizy porównawczej pomiędzy dwoma podejściami do tworzenia oprogramowania: architekturą mikroserwisową i monolityczną na podstawie aplikacji wspomagającej znajdowanie zleceń dla programistów.

W rozdziale 1 określono cele i założenia pracy oraz przedstawiono jej strukturę wraz z opisem kolejnych rozdziałów.

W rozdziale 2 przeprowadzono przegląd obecnego stanu wiedzy na temat architektury oprogramowania. Przedstawiono w nim rolę architektury w inżynierii oprogramowania oraz ewolucję jej definicji na przestrzeni lat. Ponadto, zdefiniowano wzorce architektoniczne występujące w architekturze oprogramowania oraz omówiono prawo Conwaya, które jest powszechnie wykorzystywane w dziedzinie tworzenia oprogramowania mikroserwisowego.

Rozwiązania przedstawione w rozdziale 3 obejmowały przegląd obu architektur, w którym porównano ich charakterystykę, zalety oraz wady. Omówiono również wyzwania związane z implementacją mikroserwisów oraz modularny monolit jako alternatywę dla tradycyjnej architektury monolitycznej. Rozdział dostarcza kluczowych informacji potrzebnych do oceny, która architektura jest bardziej odpowiednia dla określonych projektów oprogramowania.

W rozdziale 4 zawarto streszczony opis realizacji projektu referencyjnego w architekturze monolitycznej powstałego w wyniku pracy inżynierskiej [63]. Rozdział przedstawia wymagania funkcjonalne oraz niefunkcjonalne aplikacji, projekt bazy danych. Opisano proces implementacji oraz konteneryzacje aplikacji monolitycznej. Zawarto również informacje o wykorzystanych technologiach.

W rozdziale 5 przedstawiono projekt aplikacji w architekturze mikroserwisowej. Przedstawiono proces dekompozycji aplikacji monolitycznej, diagram architektury systemu oraz wykorzystane technologie. Następnie skupiono się na implementacji systemu, w której opisano dwa mikroserwisy. W rozdziale zawarto również informacje na temat infrastruktury wykorzystywanej przez aplikacje mikroserwisowe.

Rozdział 6 poświęcono na przeprowadzenie badań wydajności obu architektur na podstawie projektu referencyjnego. Przeprowadzono testy obciążeniowe pod stałym obciążeniem i pod zmiennym obciążeniem, a także testy przeciążeniowe, które miały na celu zbadanie wydajności aplikacji pod ekstremalnym obciążeniem. Wyniki przeprowadzonego

badania pokazały, że przepustowość aplikacji opartej na architekturze mikroserwisowej przewyższa tę, którą oferuje aplikacja w architekturze monolitycznej. Ponadto, aplikacja mikroserwisowa lepiej radzi sobie z dużym obciążeniem.

7.1 Wnioski końcowe

Obie przedstawione architektury mają swoje zalety i wady, dlatego wybór odpowiedniej architektury powinien być zdefiniowany przez problem biznesowy, który ma zostać rozwiązany. Praca magisterska dostarcza kompleksowej analizy teoretycznej architektury monolitycznej i mikroserwisowej na podstawie przeglądu dostępnej literatury naukowej, co pozwala na lepsze zrozumienie ich mocnych i słabych stron oraz wspomaga proces podejmowania decyzji przy wyborze odpowiedniego podejścia. Dodatkowo, dzięki przeprowadzonej implementacji przykładowej aplikacji w obu architekturach, możliwe było zobrazowanie różnic w nakładzie pracy wymaganej do implementacji każdej z wersji.

Aplikacja monolityczna była zdecydowanie prostsza do implementacji, ponieważ cały kod znajdował się w jednym repozytorium, a wszystkie dane były zarządzane przez jedną bazę danych, co eliminowało konieczność obsługi synchronizacji danych oraz transakcji rozproszonych. Z kolei aplikacja mikroserwisowa wprowadza dodatkową złożoność, wymagając przede wszystkim precyzyjnego wyznaczenia granic kontekstu, aby mikroserwis miał tylko jedną odpowiedzialność w systemie. Proces dekompozycji aplikacji monolitycznej miał na celu przedstawienie, jak powinna wyglądać przykładowa dekompozycja aplikacji z zachowaniem luźnych powiązań między usługami, co jest kluczową cechą w celu zapewnienia ich niezależności.

Ważnym aspektem jest również koszt infrastruktury. Jak wiadomo idealna architektura mikroserwisowa zakłada oddzielną bazę danych dla każdej usługi, co wiąże się z większymi kosztami Jak zostało wspomniane podczas przeglądu architektury mikroserwisowej koszty infrastruktury mikroserwisowej są według jednego z przeprowadzonego eksperymentów 2x większe, dlatego warto pamiętać, że dodatkowe benefity wynikające z architektury rozproszonej oznaczają również dodatkowe koszty.

Przeprowadzone badania pozwoliły przyjrzeć się zachowaniu aplikacji w dwóch architekturach. Testy pokazały, że wydajność obu aplikacji jest porównywalna przy niższym obciążeniu, natomiast im większe obciążenie tym aplikacja mikroserwisowa wykazywała się lepszą wydajnością.

Natomiast przeprowadzone testy mogą nie być w pełni miarodajne, ze względu na to, że środowisko testowe miało charakter prowizoryczny, a wszystkie mikrousługi oraz ich repliki były uruchomione na tym samym serwerze. Dlatego w przypadku dalszego rozwoju pracy zaleca się przeprowadzenie dalszych badań w bardziej realistycznym środowisku z wykorzystaniem zwiększonych zasobów infrastrukturalnych. Taka konfiguracja pozwoliłaby na przetestowanie bardziej skomplikowanych scenariuszy pod większym obciążeniem, gdyż na obecnym lokalnym środowisku osiągnięcie poziomu 2000-2500 użytkowników na sekundę okazało się być limitem nie do przekroczenia dla obu aplikacji. Dodatkowo w celu lepszego porównania wydajności pomiędzy architekturami, warto rozważyć przepisanie aplikacji monolitycznej na język JVM, taki jak Java czy Kotlin z wykorzystaniem Spring Framework, aby wyeliminować wpływ technologiczny z przeprowadzonego porównania.

Podsumowując, praca magisterska dostarcza analizę porównawczą architektury monolitycznej i mikroserwisowej umożliwiającą lepsze zrozumienie mocnych i słabych stron obu architektur oraz dostarczającą rekomendacje, które mogą być pomocne przy wyborze odpowiedniego podejścia architektonicznego. **Wszystkie cele pracy zostały spełnione.**

8 Bibliografia

[1] R. Mishra, N. Jaiswal, R. Prakash, i P. N. Barwal, „Transition from Monolithic to Microservices Architecture: Need and proposed pipeline”, w *2022 International Conference on Futuristic Technologies (INCOFT)*, Belgaum, India: IEEE, lis. 2022, s. 1–6. doi: 10.1109/INCOFT55651.2022.10094556.

[2] F. P. Brooks i K. E. Iverson, *Automatic data processing: System/360 edition*. New York: Wiley, 1969.

[3] F. P. Brooks, *The mythical man-month: essays on software engineering*, Anniversary ed. with 4 new chapters, 39. printing. Boston, Mass.: Addison-Wesley, 2013.

[4] R. W. Schwanke, R. Z. Altucher, i M. A. Platoff, „Discovering, visualizing, and controlling software structure”, w *Proceedings of the 5th international workshop on Software specification and design - IWSSD '89*, Pittsburgh, Pennsylvania, United States: ACM Press, 1989, s. 147–154. doi: 10.1145/75199.75223.

[5] https://insights.sei.cmu.edu/documents/2544/2010_010_001_513810.pdf [Data dostępu: 12.03.2024)]

[6] P. B. Kruchten, „The 4+1 View Model of architecture”, *IEEE Softw.*, t. 12, nr 6, s. 42–50, lis. 1995, doi: 10.1109/52.469759.

[7] P. Kruchten i S. Waligórski, *Rational unified process od strony teoretycznej*. w Inżynieria Oprogramowania. Warszawa: Wydawnictwa Naukowo-Techniczne, 2007.

[8] L. Bass, P. Clements, i R. Kazman, *Software architecture in practice*, Fourth edition. w Sei series in software engineering. Boston: Addison-Wesley, 2021.

[9] „ISO/IEC Standard for Systems and Software Engineering - Recommended Practice for Architectural Description of Software-Intensive Systems”. IEEE. doi: 10.1109/IEEESTD.2007.386501.

[10] P. Clements, Red., *Documenting software architectures: views and beyond*, 2nd ed. w SEI series in software engineering. Upper Saddle River, NJ: Addison-Wesley, 2011.

[11] „ISO/IEC/IEEE Systems and software engineering -- Architecture description”. IEEE. doi: 10.1109/IEEESTD.2011.6129467.

[12] https://www.iso.org/obp/ui/es/#iso:std:iso-iec-ieee:42010:ed-2:v1:en [Data dostępu: (12.03.2024)]

[13] https://pl.wikipedia.org/wiki/Architektura_oprogramowania [Data dostępu: (12.03.2024)]

[14] http://project-media.pl/definicjeAO.php [Data dostępu: (12.03.2024)]

[15] M. Broy, „Yesterday, Today, and Tomorrow: 50 Years of Software Engineering”, *IEEE Softw.*, t. 35, nr 5, s. 38–43, wrz. 2018, doi: 10.1109/MS.2018.290111138.

[16] https://www.weetechsolution.com/blog/role-of-software-architecture-in-software-engineering [Data dostępu: (12.03.2024)]

[17] R. N. Taylor, N. Medvidović, i E. M. Dashofy, *Software architecture: foundations, theory, and practice*. Hoboken, NJ: John Wiley, 2010.

[18] P. Kuchana, *Software Architecture Design Patterns in Java*, 0 wyd. Auerbach Publications, 2004. doi: 10.1201/9780203496213.

[19] M. Richards, *Software architecture patterns*, Second edition. Sebastopol, CA: O’Reilly Media, Inc., 2022.

[20] Krasner et al., "A cookbook for using the model-view controller user interface paradigm in smalltalk-80", *J. Object Oriented Program.*, vol. 1, no. 3, pp. 26-49, sie. 1988..

[21] https://en.wikipedia.org/wiki/Event_(computing) [Data dostępu: (18.03.2024)]

[22] C. L. Hall, *Techniczne podstawy systemów klient-serwer*. Warszawa: Wydawnictwa Naukowo-Techniczne, 2011.

[23] https://learn.microsoft.com/pl-pl/azure/architecture/patterns/pipes-and-filters [Data dostępu: (22.03.2024)]

[24] R. Schollmeier, „A definition of peer-to-peer networking for the classification of peer-to-peer architectures and applications”, w *Proceedings First International Conference on Peer-to-Peer Computing*, Linkoping, Sweden: IEEE Comput. Soc, 2002, s. 101–102. doi: 10.1109/P2P.2001.990434.

[25] https://en.wikipedia.org/wiki/Service-oriented_architecture [Data dostępu: (22.03.2024)]

[26] D. Krafzig, K. Banke, i D. Slama, *Enterprise SOA: service-oriented architecture best practices*. w The Coad series. Indianapolis, IN: Prentice Hall Professional Technical Reference, 2005.

[27] G. Blinowski, A. Ojdowska, i A. Przybylek, „Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation”, *IEEE Access*, t. 10, s. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[28] M. Kyryk, O. Tymchenko, N. Pleskanka, i M. Pleskanka, „Methods and process of service migration from monolithic architecture to microservices”, w *2022 IEEE 16th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Lviv-Slavske, Ukraine: IEEE, luty 2022, s. 553–558. doi: 10.1109/TCSET55632.2022.9767055.

[29] https://pl.wikipedia.org/wiki/Aplikacja_monolityczna [Data dostępu: (01.06.2024)]

[30] R. Nakazawa, T. Ueda, M. Enoki, i H. Horii, „Visualization Tool for Designing Microservices with the Monolith-First Approach”, w *2018 IEEE Working Conference on Software Visualization (VISSOFT)*, Madrid: IEEE, wrz. 2018, s. 32–42. doi: 10.1109/VISSOFT.2018.00012.

[31] C.Richardson, „Monolithic Architecture pattern”, https://microservices.io/patterns/monolithic.html. [Data dostępu: 20.06.2024]

[32] R. Mishra, N. Jaiswal, R. Prakash, i P. N. Barwal, „Transition from Monolithic to Microservices Architecture: Need and proposed pipeline”, w *2022 International Conference on Futuristic Technologies (INCOFT)*, Belgaum, India: IEEE, lis. 2022, s. 1–6. doi: 10.1109/INCOFT55651.2022.10094556.

[33] J. Lewis and M. Fowler, „Microservices: A Definition of This New Architectural Term,” https://www.martinfowler.com/articles/microservices.html [Data dostępu: 20.06.2024]

[34] M. Bruce, P. A. Pereira, M. Rogulska, i M. Rogulski, Red., *Mikroserwisy w akcji*, Wydanie I. Warszawa: PWN, 2019.

[35] V. Benavente, L. Yantas, I. Moscol, C. Rodriguez, R. Inquilla, i Y. Pomachagua, „Comparative Analysis of Microservices and Monolithic Architecture”, w *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*, Al-Khobar, Saudi Arabia: IEEE, grudz. 2022, s. 177–184. doi: 10.1109/CICN56167.2022.10008275.

[36] https://www.redhat.com/en/topics/api/what-does-an-api-gateway-do [Data dostępu: (22.06.2024)]

[37] V. Velepucha i P. Flores, „A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges”, *IEEE Access*, t. 11, s. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.

[38] https://en.wikipedia.org/wiki/Service_discovery [Data dostępu: (22.06.2024)]

[39] https://pl.wikipedia.org/wiki/R%C3%B3wnowa%C5%BCenie_obci%C4%85%C5%BCenia [Data dostępu: (22.06.2024)]

[40] https://pl.wikipedia.org/wiki/Uniform_Resource_Identifier [Data dostępu: (22.06.2024)]

[41] https://pl.wikipedia.org/wiki/Internet [Data dostępu: (23.06.2024)]

[42] https://pl.wikipedia.org/wiki/SOAP [Data dostępu: (23.06.2024)]

[43] https://pl.wikipedia.org/wiki/Representational_state_transfer [Data dostępu: (23.06.2024)]

[44] https://kgb1001001.github.io/cloudadoptionpatterns/Microservices/Adapter-Microservice/ [Data dostępu: (24.06.2024)]

[45] https://openprairie.sdstate.edu/cgi/viewcontent.cgi?article=3974&context=etd [Data dostępu: (18.03.2024)]

[46] D. Surya Sai Venkatesh i S. Agarwal, „Data Access Pattern Recommendations for Microservices Architecture”, w *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, Barcelona, Spain: IEEE, lip. 2022, s. 241–243. doi: 10.1109/CLOUD55607.2022.00044.

[47] https://en.wikipedia.org/wiki/CI/CD [Data dostępu: (28.06.2024)]

[48] https://en.wikipedia.org/wiki/Infrastructure_as_code [Data dostępu: (28.06.2024)]

[49] https://www.melconway.com/research/committees.html [Data dostępu: (29.06.2024)]

[50] S. Newman, *Budowanie mikrousług: szybkie wprowadzenie : wykorzystaj potencjał architektury usług!* Gliwice: Helion, 2015.

[51] C. H. Jack, S. K. Teck, L. T. Ming, i D. Y. Hong, „An Overview Analysis of Authentication Mechanism in Microservices-Based Software Architecture: A Discussion Paper”, w *2023 IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS)*, Penang, Malaysia: IEEE, sie. 2023, s. 1–6. doi: 10.1109/ICSECS58457.2023.10256409.

[52] M. G. De Almeida i E. D. Canedo, „Authentication and Authorization in Microservices Architecture: A Systematic Literature Review”, *Applied Sciences*, t. 12, nr 6, s. 3023, mar. 2022, doi: 10.3390/app12063023.

[53] X. He i X. Yang, „Authentication and Authorization of End User in Microservice Architecture”, *J. Phys.: Conf. Ser.*, t. 910, s. 012060, paź. 2017, doi: 10.1088/1742-6596/910/1/012060.

[54] Y. J. Al-Houmaily i G. Samaras, „Two-Phase Commit”, w *Encyclopedia of Database Systems*, L. Liu i M. T. Özsu, Red., New York, NY: Springer New York, 2018, s. 4259–4265. doi: 10.1007/978-1-4614-8265-9_713.

[55] N. N. Sheyanov, M. P. Sinev, D. O. Neshko, D. V. Pashchenko, D. A. Trokoz, i A. I. Martyshkin, „METHODS TO SYNCHRONIZE DATA IN A MICROSERVICE ARCHITECTURE”, *Scientia Iranica*, t. 0, nr 0, s. 0–0, cze. 2022, doi: 10.24200/sci.2022.53164.3088.

[56] E. Daraghmi, C.-P. Zhang, i S.-M. Yuan, „Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture”, *Applied Sciences*, t. 12, nr 12, s. 6242, cze. 2022, doi: 10.3390/app12126242.

[57] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures”. Doctoral dissertation, 2000.

[58] M. Niswar, R. Arisandy Safruddin, A. Bustamin, i I. Aswad, „Performance evaluation of microservices communication with REST, GraphQL, and gRPC”, *International Journal of Electronics and Telecommunications*, s. 429–436, cze. 2024, doi: 10.24425/ijet.2024.149562.

[59] N. N. Nayim, A. Karmakar, M. R. Ahmed, M. Saifuddin, i Md. H. Kabir, „Performance Evaluation of Monolithic and Microservice Architecture for an E-commerce Startup”, w *2023 26th International Conference on Computer and Information Technology (ICCIT)*, Cox’s Bazar, Bangladesh: IEEE, grudz. 2023, s. 1–5. doi: 10.1109/ICCIT60459.2023.10441241.

[60] V. Vernon i R. Meryk, *DDD dla architektów oprogramowania*. Gliwice: Wydawnictwo Helion, 2016.

[61] J. Castro, N. Laranjeiro, i M. Vieira, „Techniques and Tools for Runtime Security Monitoring and Analysis of Microservices”, w *2023 53rd Annual IEEE/IFIP International*

Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S), Porto, Portugal: IEEE, cze. 2023, s. 191–193. doi: 10.1109/DSN-S58398.2023.00052.

[62] Kwan, M. Cataldo, i D. Damian, „Conway's Law Revisited: The Evidence for a Task-Based Perspective", *IEEE Softw.*, t. 29, nr 1, s. 90–93, sty. 2012, doi: 10.1109/MS.2012.3.

[63] S. Berdzik „Projekt aplikacji wspomagającej znajdowanie zleceń dla programistów", Praca inżynierska, Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie, 2023 [Data dostępu: 09.07.2024]

[64] https://yaml.org/ [Data dostępu: 09.07.2024]

[65] https://swagger.io/specification/ [Data dostępu: 09.07.2024]

[66] https://docs.docker.com/compose/ [Data dostępu: 09.07.2024]

[67] https://www.docker.com/ [Data dostępu: 09.07.2024]

[68] https://www.php.net/ [Data dostępu: 09.07.2024]

[69] https://www.nginx.com/ [Data dostępu: 09.07.2024]

[70] https://symfony.com/ [Data dostępu: 09.07.2024]

[71] https://api-platform.com/ [Data dostępu: 09.07.2024]

[72] https://www.elastic.co/ [Data dostępu: 09.07.2024]

[73] https://redis.io/ [Data dostępu: 09.07.2024]

[74] https://www.mysql.com/ [Data dostępu: 09.07.2024]

[75] https://varnish-cache.org/ [Data dostępu: 09.07.2024]

[76] https://pl.wikipedia.org/wiki/Wirtualna_maszyna_Javy [Data dostępu: 12.07.2024]

[77] https://www.java.com/pl/ [Data dostępu: 12.07.2024]

[78] https://kotlinlang.org/ [Data dostępu: 12.07.2024]

[79] https://spring.io/ Data dostępu: 14.07.2024]

[80] https://kafka.apache.org/ [Data dostępu: 14.07.2024]

[81] https://github.com/ben-manes/caffeine [Data dostępu: 15.07.2024]

[82] https://www.twilio.com/docs [Data dostępu: 23.07.2024]

[83] https://www.mongodb.com/ [Data dostępu: 24.07.2024]

[84] https://maven.apache.org/ [Data dostępu: 24.07.2024]

[85] https://github.com/mailhog/MailHog [Data dostępu: 24.07.2024]

[86] https://github.com/OpenFeign/feign [Data dostępu: 25.07.2024]

[87] https://github.com/EsotericSoftware/kryo [Data dostępu: 27.07.2024]

[88] https://github.com/google/snappy [Data dostępu: 27.07.2024]

[89] https://hibernate.org/ [Data dostępu: 27.07.2024]

[90] https://www.liquibase.com/ [Data dostępu: 28.07.2024]

[91] https://github.com/lukas-krecan/ShedLock [Data dostępu: 28.07.2024]

[92] https://kubernetes.io/pl/ [Data dostępu: 30.07.2024]

[93] https://pl.wikipedia.org/wiki/Round_robin [Data dostępu: 30.07.2024]

[94] https://www.geeksforgeeks.org/benefit-of-using-mvc/ [Data dostępu: 17.08.2024]

[95] https://en.wikipedia.org/wiki/Peer-to-peer [Data dostępu: 17.08.2024]

[96] https://tomcat.apache.org/ [Data dostępu: 17.08.2024]

[97] https://jmeter.apache.org/ [Data dostępu: 17.08.2024]

[98] https://eclipse-ee4j.github.io/angus-mail/ [Data dostępu: 25.08.2024]

[99] https://kind.sigs.k8s.io/ [Data dostępu: 25.08.2025]

[100] https://jwt.io/ [Data dostępu: 09.07.2024]

[101] L. De Lauretis, „From Monolithic Architecture to Microservices Architecture”, w *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Berlin, Germany: IEEE, paź. 2019, s. 93–96. doi: 10.1109/ISSREW.2019.00050. [27.08.2024]

[102] https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html [Data dostępu: 29.08.2024]